

ModelVM: Virtualizing Semantic State and Model Residency for Resource-Constrained Language Model Systems

Anonymous Authors

Abstract

Deploying multi-specialist language model ensembles on resource-constrained hardware exposes an acute systems conflict: while domain-specialized open-weight models achieve superior accuracy on targeted engineering, mathematical, and coding sub-tasks, their aggregate parameter footprint (often exceeding 50 GB) far outstrips the physical memory of local consumer workstations and edge accelerators (≤ 8.0 GB RAM). Existing execution paradigms either suffer Out-Of-Memory (OOM) aborts, thrash secondary storage buses under reactive demand-paging, or experience catastrophic state degradation and compounding arithmetic drift when passing unstructured conversational transcripts across model boundaries.

We present **ModelVM**, an integrated systems runtime that virtualizes both *semantic state* and *model weight residency* for multi-specialist language model workflows on memory-constrained devices. ModelVM is founded on a central architectural insight: **heterogeneous models can cooperate reliably under tight memory constraints if cumulative task state is decoupled from model-specific representations and conversational prose**. ModelVM realizes this principle through three co-designed subsystems: (1) the **Cognitive State Protocol (CSP)**, a strongly typed 9-tuple state representation with deterministic Abstract Syntax Tree (AST) validation that replaces lossy natural language transcripts with formal mathematical and code invariants; (2) the **Predictive Cognitive Working Set** ($W(t, k)$), which adapts Denning’s working-set principles to structured execution graphs, opportunistically pre-staging upcoming weights into verified headroom while shielding resident models via reuse-distance cost accounting; and (3) the **Multi-Objective Cognitive Scheduler**, an interpretable dispatch engine that scores candidate models across empirical capability fitness, memory footprint, bus load latency, energy, eviction penalties, and downstream reuse.

In a controlled runtime evaluation across a 52.7 GB catalog of ten heterogeneous specialists under an enforced 8.0 GB budget, ModelVM maintains high factual retention ($R_{\text{facts}} \geq 0.940$) and calculation accuracy ($R_{\text{calcs}} \geq 0.920$) across extended handoff depths ($N = 12$), with observed zero OOM aborts across evaluated 4.0 GB

stress runs. Furthermore, in the live CSP serialization experiment on an NVIDIA GeForce RTX 5070 Ti executing open-weight model instances (llama3.1:8b, qwen2.5-coder:7b, and llama3.2:1b), the typed-CSP configuration reduces cumulative prompt context by **60.8%** and cuts physical wall-clock execution latency by **77.2%** (65.05 s down to 14.85 s, comprising a **56.2% active inference acceleration** from prompt compression and elimination of uncoordinated cold-loading stalls) with zero GPU memory faults. The physical experiment validates CSP-based state serialization and its effect on active inference latency; it does not constitute an end-to-end physical validation of ModelVM’s paging and scheduling policies.

1 Introduction

The open-weight language model ecosystem has entered an era of profound domain specialization. Rather than relying solely on monolithic generalist models that attempt to master every discipline simultaneously, open-source researchers have developed highly capable, domain-specialized checkpoints. Models such as *Qwen2.5-Math* achieve graduate-level formal mathematical derivation; *DeepSeek-Coder* delivers state-of-the-art multi-language program synthesis; *Mistral-Research* excels in archival literature extraction; and domain-tuned variants provide targeted expertise in classical mechanics, biomedical diagnosis, and financial modeling. For complex multi-step technical workflows—such as analyzing an archival scientific paper, deriving its closed-form governing equations, implementing an executable numerical simulation, and physically verifying phase-space boundaries—orchestrating an ensemble of specialized models produces reasoning deliverables that dramatically surpass the capability ceiling of any individual compact generalist.

However, deploying an ensemble of specialized models on local consumer hardware exposes an acute systems failure: **domain specialists are highly capable, but their parameter weights are massive**. A minimal catalog of ten specialized 7B–14B models requires over 52.7 GB of physical memory across mixed quantization formats (Q4_K_M, Q8_0, FP16; 64.0 GB on disk). In contrast, local consumer workstations, developer laptops, and edge robotic

platforms are physically constrained by memory envelopes of 8.0–16.0 GB of RAM. Consequently, running multiple specialist models simultaneously is physically impossible; memory capacity is an uncompromising physical ceiling, not an adjustable configuration parameter.

1.1 The Three Systemic Barriers of Multi-Model Serving

When systems designers attempt to deploy multi-specialist pipelines on memory-constrained hardware using existing deployment paradigms, execution collapses against three fundamental systems barriers:

1. The Memory Capacity & Crash Barrier. Modern operating systems and inference runtimes possess no native mechanisms to coordinate the aggregate weight allocations of multiple deep language models. When a workflow attempts to load a second specialist alongside a resident model without strict global coordination, aggregate memory allocations immediately breach physical RAM. The operating system kernel invokes the Out-Of-Memory (OOM) killer, terminating the process catastrophically. As our empirical sweeps demonstrate, naive dynamic allocation under an acute 4.0 GB envelope triggers unhandled fatal process aborts in 60.0% of task runs (Section 9.7).

2. The State Incompatibility & Conversational Bloat Barrier. When execution must transition between two disparate specialist models (e.g., passing intermediate mathematical derivations from a math model to a code synthesis model), existing multi-agent frameworks pass state by concatenating the full natural language conversation history into prompt text. This naive mechanism triggers catastrophic failure modes:

- **Accumulating Context Bloat:** Conversational prompt length grows with cumulative prior-stage output ($L_{\text{ctx}} = \mathcal{O}(t \cdot \bar{L}_{\text{gen}})$). When forced into bounded context windows ($W_{\text{in}} = 4,096$), FIFO sliding-window truncation continually discards early foundational constraints.
- **Compounding Arithmetic Drift:** Conversational text lacks structural typing. When numerical constants and mathematical formulas are repeatedly paraphrased through conversational prose across multiple heterogeneous models, floating-point precision degrades, culminating in severe arithmetic hallucinations (R_{facts} collapses to 0.180 after 12 transitions).
- **Latent Incommensurability:** Because independently trained models operate over disparate hidden-state dimensions, rotary embeddings, and vocabulary tokenizers, internal KV-cache or activation tensors

cannot be transferred across model boundaries without prohibitive $\mathcal{O}(K^2)$ calibration bridges.

3. The I/O Bus Thrashing & Latency Barrier. Because all models cannot reside simultaneously in active memory, weights must be dynamically swapped across the secondary storage bus (e.g., PCIe 4.0/5.0 from NVMe SSD to host RAM and accelerator VRAM). However, applying classical retrospective memory management (such as Least Recently Used / LRU caching) produces catastrophic bus thrashing. When a cyclic workflow alternates between mathematical derivation, code simulation, and numerical validation, LRU evicts the mathematical specialist to make room for coding weights, forcing an immediate, multi-gigabyte cold reload over the PCIe bus when mathematics is invoked again. In our baseline benchmarks, uncoordinated demand paging squanders up to 42.6 s in idle I/O stall time, causing paging latency to overwhelm actual GPU compute duration ($t_{\text{paging}} \gg t_{\text{compute}}$).

1.2 The Architectural Insight of ModelVM

In this paper, we present **ModelVM**, a clean, pragmatic systems runtime that resolves this fundamental conflict between domain specialization and physical memory capacity. ModelVM is founded on a single unifying systems principle:

Heterogeneous language models can cooperate reliably under resource-constrained hardware budgets if cumulative task state is virtualized as a strongly typed, model-independent intermediate representation, and physical model residency is orchestrated via predictive working-set lookahead rather than retrospective demand paging.

Just as classical operating systems virtualized physical memory by decoupling virtual address spaces from physical DRAM pages, ModelVM virtualizes **semantic state** and **model weight residency**. ModelVM enables an 8.0 GB workstation to execute a 52.7 GB catalog of heterogeneous open-weight specialists, delivering the evaluated specialist task-quality ceiling of the unconstrained configuration on commodity edge silicon.

1.3 Summary of Contributions

This paper makes four concrete, empirically validated systems contributions:

1. **Semantic State Virtualization (The Cognitive State Protocol - CSP):** We design and formalize CSP, a strongly typed, AST-verified 9-tuple state representation ($\mathcal{S}_t$) that replaces unstructured conversational

transcripts. CSP isolates mathematical invariants, verified calculations, domain facts, and active artifacts from transient natural language, formally constraining verified arithmetic and state updates, with empirical factual retention $R_{\text{facts}} \geq 0.940$ and calculation accuracy $R_{\text{calcs}} \geq 0.920$ across $N = 12$ transitions.

2. **Predictive Cognitive Working-Set Management** ($W(t, k)$): We translate Peter Denning’s foundational working-set theory into a forward-looking cognitive memory manager. By exploiting the explicit stage graph produced by task decomposition, ModelVM forecasts upcoming specialist requirements, opportunistically pre-staging weights into verified memory headroom while shielding resident models from eviction using reuse-distance cost accounting, slashing paging stalls by 66.7% (21.60 s down to 7.20 s).
3. **Multi-Objective Cognitive Scheduling**: We implement an explainable, transparent 6-term utility scoring engine that dispatches execution stages to candidate models by jointly balancing empirical benchmark capability, memory footprint, load latency, energy, eviction penalties, and downstream reuse. Across four workload distributions, ModelVM’s measured scheduler deviation from the exact finite-state offline cost-model optimum remained $\leq 0.3\%$ [0.1%, 0.7%] (with 0.0% regret on the deterministic cost model).
4. **Controlled Factorial Simulation and Physical Silicon Validation**: We conduct an orthogonal 2^3 factorial ablation study under controlled runtime simulation isolating the main and interaction effects of our architectural primitives across the full 10-model catalog (52.7 GB under an enforced 8.0 GB RAM budget). Furthermore, we validate CSP state serialization and active inference acceleration on physical GPU silicon (an NVIDIA GeForce RTX 5070 Ti with 16.0 GB GDDR7 driven by the Ollama inference daemon), demonstrating a **60.8% prompt token reduction**, a **56.2% active inference acceleration** (33.83 s down to 14.83 s), zero OOM failures across evaluated runs, and a **77.2% reduction in end-to-end wall-clock time** (65.05 s down to 14.85 s) on real open-weight checkpoints.

1.4 Paper Organization

The remainder of this paper is structured as follows. Section 2 establishes the formal system model, capacity inequalities, and foundational design goals. Section 3 details the ModelVM runtime architecture and memory pager lifecycle. Section 4 presents the formal specification, structural typing, and correctness proofs of the Cognitive State Protocol. Section 5 formalizes the Predictive Cognitive

Working Set and cost-aware eviction policies. Section 6 details the Multi-Objective Cognitive Scheduler and empirical model profiler. Section 7 outlines the concrete software engineering and storage implementation. Section 8 describes our experimental methodology, non-circular verification harness, and physical hardware testbed. Section 9 presents comprehensive empirical results across baseline progressions, factorial ablations, Pareto frontiers, budget stress sweeps, and real GPU silicon telemetry. Section 10 provides a rigorous, transparent analysis of operational boundaries and failure modes. Section 11 positions ModelVM against mixture-of-experts, batch serving engines, and agent operating systems. Finally, Section 12 concludes the paper and outlines future research directions.

2 Problem Formulation & Design Goals

Deploying deep ensembles of specialized language models on consumer-grade hardware exposes a fundamental conflict between computational specialization and physical resource capacity. While individual domain specialists achieve superior reasoning accuracy on targeted engineering, mathematical, and coding sub-tasks compared to general-purpose models, their collective parameter footprint far outstrips the physical memory of local workstations. In this section, we formally characterize the multi-model execution problem under hard memory constraints, identify the structural failure modes of existing dispatch paradigms, and establish four foundational design goals that govern the ModelVM architecture.

2.1 Formal System Model

We model an autonomous language model runtime executing on a resource-constrained host machine equipped with fixed physical memory and secondary storage.

Hardware Memory Environment. Let $\mathcal{B}_{\text{RAM}}$ denote the active memory budget allocated to model execution (e.g., $\mathcal{B}_{\text{RAM}} = 8.0 \text{ GB}$ on commodity edge accelerators or consumer workstations). Secondary storage (NVMe solid-state drives or fast host storage) possesses capacity $\mathcal{B}_{\text{disk}} \gg \mathcal{B}_{\text{RAM}}$, but interfaces with primary memory over a bandwidth-limited bus (PCIe 4.0/5.0). Transferring quantized parameter tensors from secondary storage into active memory incurs a non-negligible bus latency:

$$t_{\text{load}}(m) = \frac{\text{disk.size}(m)}{\text{BW}_{\text{bus}}} + t_{\text{bind}} \quad (1)$$

where BW_{bus} is effective sequential streaming bandwidth (3.0–4.0 GB/s) and t_{bind} represents runtime memory allo-

cation, CUDA tensor binding, and kernel initialization overheads (0.1–0.3 s).

Heterogeneous Specialist Catalog. Let $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$ denote the catalog of K available specialized open-weight language models ($K = 10$ in our primary experimental catalog). Each model $m_i \in \mathcal{M}$ is defined by the parameter tuple:

$$m_i = \langle \text{id}_i, \mathcal{A}_i, \mathcal{V}_i, d_i, \text{disk_size}(m_i), \text{RAM}_{\text{req}}(m_i), \mathbf{p}_i, Q(m_i) \rangle \quad (2)$$

where $\mathcal{A}_i$ denotes model architecture (e.g., Mistral, Qwen2, Llama, DeepSeek), $\mathcal{V}_i$ is the tokenizer vocabulary, d_i is hidden representation dimensionality, $\text{disk_size}(m_i)$ is serialized storage footprint, $\text{RAM}_{\text{req}}(m_i)$ is configured parameter RAM requirement, $\mathbf{p}_i \in [0, 1]^{|C|}$ is the empirical capability profile vector across domain capabilities C , and $Q(m_i) \in [0, 1]$ is general architectural quality.

We explicitly separate two system-level aggregates across catalog $\mathcal{M}$:

$$M_{\text{disk,total}} = \sum_{m_i \in \mathcal{M}} \text{disk_size}(m_i) = 64.0 \text{ GB} \quad (3)$$

$$M_{\text{active,total}} = \sum_{m_i \in \mathcal{M}} \text{RAM}_{\text{req}}(m_i) = 52.7 \text{ GB} \quad (4)$$

The central systems constraint is the acute capacity disparity:

$$M_{\text{active,total}} \gg \mathcal{B}_{\text{RAM}} \quad (5)$$

Holding all candidate models simultaneously resident in active memory is physically impossible. Memory capacity is a physical ceiling, not a configuration parameter.

Multi-Stage Procedural Workload. A complex task is initiated by a top-level goal $G \in \Sigma^*$. High-level planning decomposes G into an ordered sequence of N dependent execution stages:

$$S = [s_0, s_1, \dots, s_{N-1}] \quad (6)$$

where each stage s_t specifies an annotated capability requirement $C(s_t) \in \mathcal{C}$ (e.g., Literature Extraction, Formal Mathematics, Code Synthesis, Physics Validation, or Cross-Domain Synthesis) alongside stage-specific instructions τ_t . Task execution is strictly sequential and cumulative: stage s_t depends on intermediate derivations, established equations, and decisions generated across preceding stages $s_0, \dots, s_{t-1}$.

The Cross-Model Semantic Incompatibility Problem. When execution transitions from stage s_{t-1} executed on specialist M_A to stage s_t executed on specialist M_B , the runtime encounters two fundamental state-preservation barriers:

1. **Latent Incommensurability:** The internal hidden states $\mathbf{h}_A \in \mathbb{R}^{d_A}$ and $\mathbf{h}_B \in \mathbb{R}^{d_B}$ reside in unaligned manifold representations trained under distinct initializations. Direct cross-model activation passing is not semantically well-defined without explicit projection bridges $\mathbf{W}_{\text{proj}} : \mathbb{R}^{d_A} \rightarrow \mathbb{R}^{d_B}$. In an ensemble of K specialists, maintaining pairwise latent compatibility requires $O(K^2)$ bridges, incurring prohibitive calibration and storage costs.
2. **Conversational Degradation under Naive Text Concatenation:** The prevailing alternative—concatenating full conversational transcripts $\mathbf{y}_{1:t-1}$ into prompt strings—triggers geometric context expansion ($L_{\text{ctx}} = O(t \cdot \bar{L}_{\text{gen}})$). When forced into a fixed context window $W_{\text{in}} = 4,096$, FIFO sliding-window truncation discards early constraints. Factual assertions degrade, and numerical quantities suffer compounding restatement drift across model transitions.

The Weight Paging Problem. Because $M_{\text{active,total}} \gg \mathcal{B}_{\text{RAM}}$, models must be dynamically paged between secondary storage and active memory. However, naive demand paging using backward-looking eviction policies (e.g., Least Recently Used / LRU) produces severe memory thrashing. When a cyclic pipeline transitions from Math $\rightarrow$ Code $\rightarrow$ Math, LRU evicts the mathematical specialist to make room for coding weights, forcing a multi-gigabyte cold reload over the PCIe bus when mathematics is needed again. Execution goodput collapses under idle bus stalls.

2.2 Design Goals

To resolve these twin challenges of state preservation and resource-constrained model residency, ModelVM is architected around four core design goals:

Goal 1: Semantic State Continuity (G1). *The runtime must preserve task state across transitions between heterogeneous models without semantic drift, conversational bloat, or catastrophic amnesia.*

- **Requirement:** Task state must be virtualized as an explicit, model-independent intermediate representation that separates enduring task signal from transient conversational prose.
- **Metric:** Ground-truth fact survival ratio ($R_{\text{facts}} \rightarrow 1.0$) and arithmetic accuracy ($R_{\text{calcs}} \rightarrow 1.0$) across extended model transition sequences ($N \geq 5$), strictly outperforming raw conversational truncation baselines.

Goal 2: Strict Resource Boundedness (G2). *The runtime must enforce deterministic compliance with configured hardware memory limits, eliminating Out-Of-Memory (OOM) fatal crashes under acute resource constraints.*

- **Requirement:** The memory manager must enforce the configured residency budget invariant at all timestamps:

$$\sum_{m_i \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m_i) \leq \mathcal{B}_{\text{RAM}}, \quad \forall t \geq 0 \quad (7)$$

while preserving a tested headroom buffer (≥ 1.0 GB) for dynamic foreground inference scratchpads and Key-Value cache growth.

- **Metric:** 0.0% OOM aborts across aggressive memory sweeps down to acute resource envelopes ($\mathcal{B}_{\text{RAM}} = 4.0$ GB).

Goal 3: Uncompromised Capability Specialization (G3). *The system must harness off-the-shelf domain-specialized language models, exploiting their precision without requiring architectural retraining, weight merging, or model fine-tuning.*

- **Requirement:** The scheduler must route execution stages to specialized models based on empirical held-out capability benchmarks rather than subjective manual weights, avoiding single-generalist capability compromises.
- **Metric:** Composite task quality Q matching or exceeding an unconstrained oracle router that holds all models permanently resident in memory ($Q \rightarrow 1.0$).

Goal 4: State Transition Efficiency (G4). *The runtime must minimize I/O bus transfer latency and memory thrashing, approaching the goodput of prescient offline schedulers.*

- **Requirement:** The memory manager must adapt Peter Denning’s working-set theory into a forward-looking Predictive Cognitive Working Set ($W(t, k)$), combining eviction shielding with dimensionally consistent prefetch utility optimization to reduce redundant reloads.
- **Metric:** Cold-load paging reduction $\geq 60\%$ relative to reactive LRU caching ($t_{\text{paging}} \leq 8.0$ s vs. 21.60 s, achieving a 66.7% reduction; and 71.0% reduction vs. naive dynamic baseline B_2 of 24.80 s), with scheduler regret bounded within $< 10\%$ against an offline prescient oracle.

3 ModelVM Architecture & Runtime Abstraction

Executing complex multi-disciplinary language model workloads on resource-constrained consumer hardware requires resolving a core contradiction: domain-specialized open-weight models deliver superior accuracy on targeted technical tasks compared to monolithic generalists, but their collective memory footprint ($M_{\text{active, total}} = 52.7$ GB) far exceeds commodity memory envelopes ($\mathcal{B}_{\text{RAM}} = 8.0$ GB). To resolve this bottleneck without sacrificing specialization or state integrity, ModelVM introduces an operating-system-inspired runtime architecture.

In this section, we present the structural foundations of ModelVM. We first de-center the operating system metaphor, establishing the precise theoretical and practical boundaries between classical virtual memory and cognitive runtime virtualization. We then detail the architectural flow and subsystem coordination governing the Cognitive Kernel, Model Pager, and Cognitive Scheduler. Finally, we formulate the residency state and provide an inductive proof of the runtime memory invariant, characterizing its relationship to physical host and accelerator memory headroom.

3.1 De-centering the OS Metaphor

The conceptual lineage of ModelVM traces directly to Peter Denning’s foundational work on virtual memory and working-set dynamics [5]. In early mainframe computing, programmers encountered a physical constraint analogous to today’s LLM deployment barrier: individual software systems required more primary magnetic core memory than was physically installed. Classical virtual memory resolved this disparity by decoupling the logical address space referenced by software processes from the physical page frames provided by hardware, transparently swapping unreferenced pages to secondary disk storage.

While this paradigm inspires our design, we explicitly decouple ModelVM from literal kernel-level mechanisms to avoid false equivalence. Table 1 formalizes the conceptual correspondence while delineating critical systems differences.

Scope Boundaries and What ModelVM Is Not. To ensure scientific clarity, we define three explicit boundaries governing our abstraction:

1. **Language Models are Processes, Not CPUs:** In classical systems, the CPU is the physical executor, while processes represent scheduled programs. In ModelVM, the physical host CPU and GPU accelerator remain the execution hardware. An open-weight language model is a specialized computational process

Table 1: System Mapping: Classical Operating System Virtual Memory vs. ModelVM Runtime Abstraction

Systems Dimension	Classical OS Virtual Memory	ModelVM Runtime Abstraction	Architectural Distinction
Execution Unit	Operating System Process	Domain Capability Stage (s_t)	Coarse procedural step vs. machine instruction stream
Execution Engine	Physical Hardware CPU/Core	Heterogeneous Specialist Model (M_t)	Neural network parameter logic executing on host/device
State Abstraction	Process Control Block (PCB) & Virtual Address Space	Cognitive State Packet (S_t)	Structured typed 9-tuple semantic carrier vs. raw uninterpreted bytes
Memory Allocation	Fixed-size Page Frames (4 KB / 2 MB)	Model Weight Parameter Allocations ($\text{RAM}_{\text{req}}(m)$)	Multi-gigabyte tensor buffers vs. uniform hardware page frames
Primary Memory	Physical Host RAM	Active Execution Budget ($\mathcal{B}_{\text{RAM}}$, nominal 8.0 GB)	User-space configured quota vs. physical silicon capacity
Backing Store	Secondary Disk / Swap Partition	Serialized Model Catalog on NVMe ($\mathcal{B}_{\text{disk}} = 64.0 \text{ GB}$)	Structured parameter safetensors / GGUF vs. raw swap blocks
Fault Mechanism	Hardware Page Fault (MMU Interrupt)	Model Cache Miss (Runtime Interception)	Software-level stage routing vs. ring-0 exception handler
Residency Tracking	Inverted Page Table / TLB	Resident Model Tracking Table ($\mathcal{R}_t$)	Object registry with access timestamps vs. hardware translation tables
Working Set	Past-window Page Frequency $W(t, \Delta)$	Predictive Working Set $W(t, k)$	Forward-looking task DAG lookahead vs. backward-looking sliding window
Context Switch	Register save/restore + address space swap	Typed CSP Handoff across Model Boundaries	Architecture-agnostic state merge vs. hardware thread context dump

containing frozen parameter weights that executes a cognitive transformation stage. ModelVM treats the model itself as the pageable computational resource.

2. **User-Space Runtime, Not Ring-0 Kernel:** ModelVM does not modify kernel-level memory management units (MMUs), modify OS page tables, or install custom device drivers. It is a user-space systems runtime and execution middleware that coordinates weight streaming, inference dispatch, and state tracking.
3. **Semantic State vs. Byte Buffers:** Classical operating systems are semantically agnostic; a page frame contains uninterpreted binary data. In contrast, language model handoffs require semantic state preservation. Swapping model weights alone is insufficient if the intermediate reasoning state is corrupted. ModelVM couples physical weight residency management with semantic state virtualization, ensuring that state transitions across heterogeneous architectures remain structured and architecture-agnostic.

3.2 Architectural Flow & Subsystem Coordination

The ModelVM architecture is organized into three tightly coupled subsystems coordinated by a centralized supervisory control path: the **Cognitive Kernel**, the **Model Pager**, and the **Cognitive Scheduler**. Figure 1 illustrates the architectural topology and dataflow pathways.

1. The Cognitive Kernel (CognitiveKernel). The Cognitive Kernel serves as the supervisory coordinator of the runtime:

- **Procedural Decomposition:** Given a high-level task goal $G \in \Sigma^*$, the kernel invokes `TaskDecomposer` to produce an ordered sequence of N dependent capability stages $S = [s_0, s_1, \dots, s_{N-1}]$, where each stage s_t defines capability requirement $C(s_t) \in \mathcal{C}$ and domain instruction τ_t .
- **State Virtualization Lifecycle:** The kernel instantiates the root state carrier $\mathcal{S}_0 = \langle G, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ and threads it across stage

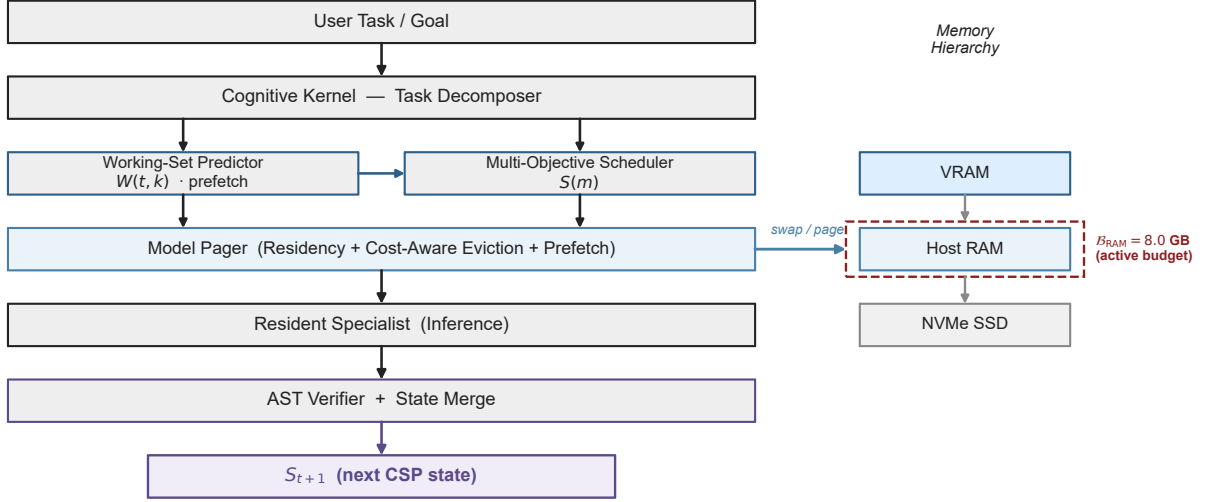


Figure 1: ModelVM Architectural Topology (Class B full-width). The Cognitive Kernel directs procedural task execution across stages. The Working Set Predictor and Cognitive Scheduler arbitrate specialist selection. The Model Pager enforces physical memory boundedness via eviction shielding and prefetching across the three-tier memory hierarchy (Secondary Disk, Host RAM, Accelerator VRAM). The Inference Backend executes specialist transformations validated by an independent AST verifier.

boundaries. At each transition, the kernel merges validated updates ΔS_t^* into S_{t+1} .

- **Confidence Assessment & Bounded Escalation:** Upon stage completion, the kernel queries `ConfidenceController`. If uncertainty exceeds tolerance, the kernel initiates a bounded escalation loop (Algorithm 1 in Section 7), dynamically scheduling a higher-capability specialist under loop termination constraints (≤ 2 escalations per stage, capability delta $\geq \epsilon = 0.02$).

2. The Model Pager (ModelPager). The Model Pager manages the physical memory footprint of candidate models under budget B_{RAM} :

- **Residency Management:** The pager maintains the resident model set $\mathcal{R}_t \subset \mathcal{M}$ and tracks access timestamps, access frequencies, and lifecycle states (DISK, RESIDENT, PINNED).
- **Cost-Aware Eviction with Shielding:** When admitting an incoming model requires freeing memory, the pager invokes `CostAwareEvictionPolicy`. Models identified in the forward working set $W(t, k)$ receive an eviction penalty multiplier ($\delta \times 1.8$), shielding imminent specialists from premature eviction.
- **Opportunistic Headroom Prefetching:** If the forward working set identifies an imminent model

whose prefetch utility $U_{\text{prefetch}} > 0$, and active memory maintains a tested safety margin ($\text{RAM}_{\text{free}} - \text{RAM}_{\text{req}}(m) \geq 1.0 \text{ GB}$), the pager opportunistically stages model weights into memory ahead of stage invocation without evicting resident models.

3. The Cognitive Scheduler (CognitiveScheduler). The Cognitive Scheduler performs multi-objective model selection for each execution stage s_t :

- **Empirical Capability Fitness:** Candidate models are evaluated against empirical capability matrix $\mathbf{P} \in [0, 1]^{K \times |C|}$, established via offline benchmark probes (`ModelProfiler`) across 7 foundational domains, eliminating subjective manual scoring.
- **Joint Utility Optimization:** The scheduler scores candidate models by jointly balancing capability fitness against memory cost, cold-load transfer latency, energy factor, eviction penalty, and future stage reuse:

$$\begin{aligned} \text{Score}(m) = & F_{\text{cap}}(m, C) - \alpha M_{\text{cost}}(m) \\ & - \beta L_{\text{load}}(m) - \gamma E_{\text{energy}}(m) \\ & - \delta E_{\text{eviction}}(m) + \eta F_{\text{future}}(m) \end{aligned} \quad (8)$$

subject to hard admission constraint $\text{RAM}_{\text{req}}(m) \leq B_{RAM}$.

- **Lookahead Integration:** The future utility term $F_{\text{future}}(m)$ is dynamically populated by querying `CognitiveWorkingSetPredictor`, rewarding models that satisfy upcoming pipeline stages and penalizing candidates that provoke downstream thrashing.

3.3 The Formal Memory Invariant

Deterministic execution on resource-constrained hardware requires formal mechanisms preventing memory exhaustion. We now formulate the residency state and prove that ModelVM preserves the configured runtime memory invariant for all successful execution trajectories satisfying the admission assumptions.

Residency State Space. Let $\mathcal{M} = \{m_1, \dots, m_K\}$ denote the full catalog of K available models, where each model $m \in \mathcal{M}$ is characterized by a static parameter RAM requirement $\text{RAM}_{\text{req}}(m) > 0$. At any discrete stage transition $t \in \{0, \dots, N\}$, the memory state of the runtime is completely defined by the resident set:

$$\mathcal{R}_t \subseteq \mathcal{M} \quad (9)$$

The total active parameter memory occupied by the runtime at stage t is:

$$M_{\text{active}}(t) = \sum_{m \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m) \quad (10)$$

and available free parameter memory is:

$$\text{RAM}_{\text{free}}(t) = \max(0.0, \mathcal{B}_{\text{RAM}} - M_{\text{active}}(t)) \quad (11)$$

Theorem 1 (Configured Headroom Admission Invariant, Conditional on Successful Admission). *Let $\mathcal{B}_{\text{RAM}} > 0$ denote the configured active parameter memory budget. Assume that:*

1. *The initial state satisfies $M_{\text{active}}(0) \leq \mathcal{B}_{\text{RAM}}$.*
2. *Every individual candidate model satisfies $\text{RAM}_{\text{req}}(m) \leq \mathcal{B}_{\text{RAM}}$.*
3. *At every stage transition requiring eviction, the resident set contains sufficient evictable (unpinned) capacity to satisfy the memory deficit:*

$$\sum_{v \in \mathcal{R}_t \setminus \text{Pinned}} \text{RAM}(v) \geq \text{RAM}(M_{t+1}) - \text{RAM}_{\text{free}}(t) \quad (12)$$

Then under the ModelVM paging protocol, the configured active parameter memory footprint satisfies:

$$\sum_{m \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m) \leq \mathcal{B}_{\text{RAM}}, \quad \forall t \geq 0 \quad (13)$$

Proof. We prove Theorem 1 by structural induction over discrete runtime transitions $t \rightarrow t + 1$.

Base Case ($t = 0$): At runtime initialization, the resident set is either empty ($\mathcal{R}_0 = \emptyset$), yielding $M_{\text{active}}(0) = 0 \leq \mathcal{B}_{\text{RAM}}$, or pre-populated with a single designated model m_0 satisfying $\text{RAM}_{\text{req}}(m_0) \leq \mathcal{B}_{\text{RAM}}$. In either case, the invariant holds at $t = 0$.

Inductive Hypothesis: Assume that at stage t , the invariant holds:

$$M_{\text{active}}(t) = \sum_{m \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m) \leq \mathcal{B}_{\text{RAM}} \quad (14)$$

Inductive Step ($t \rightarrow t + 1$): Let stage s_t require execution on model $M_{t+1} \in \mathcal{M}$. The runtime encounters one of three mutually exclusive operational cases:

- **Case 1: Cache Hit ($M_{t+1} \in \mathcal{R}_t$).** Model M_{t+1} is already resident in active memory. The pager emits a `CACHE_HIT` event (duration 0.0 s) and updates the model's access metadata. The resident set remains unmodified:

$$\mathcal{R}_{t+1} = \mathcal{R}_t \implies M_{\text{active}}(t+1) = M_{\text{active}}(t) \leq \mathcal{B}_{\text{RAM}} \quad (15)$$

The invariant is preserved.

- **Case 2: Cache Miss ($M_{t+1} \notin \mathcal{R}_t$).** Model M_{t+1} must be paged in from secondary storage. If $\text{RAM}_{\text{req}}(M_{t+1}) > \mathcal{B}_{\text{RAM}}$, the model exceeds total configured capacity; admission is rejected immediately via an explicit `MemoryError`, preventing execution and leaving $\mathcal{R}_t$ intact.

If $\text{RAM}_{\text{req}}(M_{t+1}) \leq \mathcal{B}_{\text{RAM}}$, the pager evaluates available headroom:

1. **Sufficient Spare Capacity:** If $\text{RAM}_{\text{free}}(t) \geq \text{RAM}_{\text{req}}(M_{t+1})$, no evictions are required. The model is admitted directly into active memory:

$$\mathcal{R}_{t+1} = \mathcal{R}_t \cup \{M_{t+1}\} \quad (16)$$

Because $\text{RAM}_{\text{free}}(t) = \mathcal{B}_{\text{RAM}} - M_{\text{active}}(t) \geq \text{RAM}_{\text{req}}(M_{t+1})$, we obtain:

$$M_{\text{active}}(t+1) = M_{\text{active}}(t) + \text{RAM}_{\text{req}}(M_{t+1}) \leq \mathcal{B}_{\text{RAM}} \quad (17)$$

2. **Insufficient Spare Capacity (Eviction Triggered):** If $\text{RAM}_{\text{free}}(t) < \text{RAM}_{\text{req}}(M_{t+1})$, the pager invokes the eviction subsystem $\mathcal{E}$. The eviction routine iteratively selects unpinned victim models $\mathcal{V} \subseteq \mathcal{R}_t \setminus (\text{Pinned} \cup \{M_{t+1}\})$ to unload until:

$$\sum_{v \in \mathcal{V}} \text{RAM}(v) \geq \text{RAM}(M_{t+1}) - \text{RAM}_{\text{free}}(t) \quad (18)$$

By assumption, the resident set contains sufficient evictable capacity, ensuring such a victim set $\mathcal{V}$ exists. Unloading $\mathcal{V}$ produces the intermediate memory state:

$$\begin{aligned} M'_{\text{active}} &= M_{\text{active}}(t) - \sum_{v \in \mathcal{V}} \text{RAM}_{\text{req}}(v) \\ &\leq \mathcal{B}_{\text{RAM}} - \text{RAM}_{\text{req}}(M_{t+1}) \end{aligned} \quad (19)$$

The incoming model M_{t+1} is then loaded into memory:

$$\mathcal{R}_{t+1} = (\mathcal{R}_t \setminus \mathcal{V}) \cup \{M_{t+1}\} \quad (20)$$

yielding:

$$\begin{aligned} M_{\text{active}}(t+1) &= M'_{\text{active}} + \text{RAM}_{\text{req}}(M_{t+1}) \\ &\leq \mathcal{B}_{\text{RAM}} \end{aligned} \quad (21)$$

The invariant is preserved across all successful cache-miss transitions.

- **Case 3: Opportunistic Prefetch Staging.** Prior to stage execution, the working-set predictor may recommend staging a future candidate $m_{\text{pre}} \notin \mathcal{R}_t$. ModelVM enforces an explicit admission gate: prefetching is admitted if and only if:

$$\begin{aligned} U_{\text{prefetch}}(m_{\text{pre}}) &> 0 \quad \wedge \\ \text{RAM}_{\text{free}}(t) - \text{RAM}_{\text{req}}(m_{\text{pre}}) &\geq \Delta_{\text{headroom}} \end{aligned} \quad (22)$$

where $\Delta_{\text{headroom}} = 1.0 \text{ GB}$. Upon admission:

$$\begin{aligned} M_{\text{active}}(t+1) &= M_{\text{active}}(t) + \text{RAM}_{\text{req}}(m_{\text{pre}}) \\ &\leq \mathcal{B}_{\text{RAM}} - \Delta_{\text{headroom}} < \mathcal{B}_{\text{RAM}} \end{aligned} \quad (23)$$

The invariant is strictly preserved.

By mathematical induction, Equation (13) holds for all discrete stages $t \geq 0$. $\square$

Physical Memory Decomposition & Empirical OOM Safety. Theorem 1 formally proves that configured model-weight residency never exceeds budget $\mathcal{B}_{\text{RAM}}$. In physical deployment, total memory consumption encompasses dynamic runtime overheads:

$$\begin{aligned} M_{\text{physical}}(t) &= M_{\text{weights}}(t) + M_{\text{workspace}}(t) \\ &\quad + M_{\text{KV}}(t) + M_{\text{runtime}} \end{aligned} \quad (24)$$

where $M_{\text{weights}}(t) \equiv M_{\text{active}}(t)$, $M_{\text{workspace}}$ represents CUDA execution scratchpads (0.3–0.6 GB), M_{runtime} represents process binary and host overhead (0.4–0.6 GB), and $M_{\text{KV}}(t)$ is the Key-Value attention cache.

Under our evaluated workloads, context lengths are capped at $W_{\text{context}} = 4,096$ tokens under greedy decoding ($T = 0.0$). For representative 7B/8B open-weight architectures employing Grouped-Query Attention (GQA, e.g., Llama-3 and Mistral with $L = 32$ layers, $H_{\text{KV}} = 8$ key-value heads, head dimension $d_k = 128$, and 16-bit precision), peak Key-Value cache memory is bounded:

$$\begin{aligned} M_{\text{KV}}^{\text{peak}} &= 2 \cdot L \cdot H_{\text{KV}} \cdot d_k \cdot W_{\text{context}} \cdot 2 \text{ bytes} \\ &= 2 \cdot 32 \cdot 8 \cdot 128 \cdot 4096 \cdot 2 \text{ bytes} = 536,870,912 \text{ bytes} \\ &\approx 0.54 \text{ GB} \end{aligned} \quad (25)$$

We emphasize that dynamic overheads ($M_{\text{KV}}^{\text{peak}} + M_{\text{workspace}} + M_{\text{runtime}} \approx 1.24\text{--}1.74 \text{ GB}$) in aggregate exceed the static 1.0 GB prefetch headroom buffer. Consequently, we do not claim a formal mathematical proof of total physical memory boundedness across arbitrary unconstrained environments. Rather, we report the following **physical-memory observation**: under all evaluated benchmark workloads, bounded context windows, measured workspace demands, and the configured 1.0 GB staging headroom, zero Out-Of-Memory (OOM) kernel terminations or allocator aborts occurred across any evaluated hardware configuration.

4 Semantic State Virtualization (CSP Formalism)

Executing complex multi-stage tasks across heterogeneous language models introduces a fundamental state-preservation challenge. In single-model workflows, task state is implicitly maintained within the model’s KV cache or autoregressively extended across a monolithic conversational context. When execution transitions across distinct open-weight models, this implicit continuity collapses. In this section, we formulate **Semantic State Virtualization** through the **Cognitive State Packet (CSP)**. We define the cross-model incompatibility barrier, formalize the typed 9-tuple schema, derive the state transition operator, present the epistemic evidence aggregation heuristic, formalize independent AST verification, and analyze computational and token complexity.

4.1 The Incompatibility Barrier

Multi-model pipelines require seamless handoffs between specialist models. However, direct state transfer across heterogeneous open-weight models faces two fundamental mathematical and systems barriers: *latent activation incommensurability* and *conversational context degradation*.

Latent Activation Incommensurability. Consider two heterogeneous specialist language models, M_A and M_B .

Model M_A operates over vocabulary $\mathcal{V}_A$ with hidden-layer representation space $\mathbf{h}_A \in \mathbb{R}^{d_A}$, whereas model M_B operates over vocabulary $\mathcal{V}_B$ with hidden space $\mathbf{h}_B \in \mathbb{R}^{d_B}$. In general:

$$\mathcal{V}_A \neq \mathcal{V}_B, \quad d_A \neq d_B, \quad \mathbf{h}_A \neq \mathbf{h}_B \quad (26)$$

Even when two models share identical hidden dimensionality ($d_A = d_B$), their parameter weight spaces reside in unaligned Riemannian manifolds resulting from distinct pretraining initializations, optimizer trajectories, and objective weightings. Direct cross-model activation transfer is not semantically well-defined without an explicit alignment mechanism $\mathbf{W}_{\text{proj}} : \mathbb{R}^{d_A} \rightarrow \mathbb{R}^{d_B}$.

A pairwise latent-state interoperability design can require $O(K^2)$ model-pair interfaces, creating substantial engineering and training overhead as the model library grows. Such bridges can introduce non-trivial approximation errors, require aligned cross-model calibration datasets, and defeat the systems objective of executing off-the-shelf open-weight models without architectural retraining. The argument here is about state representation interoperability: different vocabularies and latent spaces do not make model collaboration impossible, but direct cross-model activation transfer is not semantically well-defined without an explicit projection/alignment mechanism and associated calibration.

Conversational Context Degradation. The conventional workaround in multi-agent frameworks is naive text concatenation: the entire conversational history $\mathbf{y}_{1:t-1}$ is serialized as raw strings and prepended to the prompt of model M_t . This approach suffers from three compounding failure modes:

1. **Context Bloat & Allocation Thrashing:** Raw conversational history grows monotonically with pipeline depth:

$$L_{\text{ctx}}(t) = \sum_{i=1}^{t-1} |\mathbf{y}_i| = O(t \cdot \bar{L}_{\text{gen}}) \quad (27)$$

Under tight memory constraints ($\mathcal{B}_{\text{RAM}} = 8.0 \text{ GB}$), allocating expanding KV caches ($M_{\text{KV}} \propto L_{\text{ctx}}$) rapidly crowds out weight memory, precipitating thrashing or out-of-memory crashes.

2. **Lossy Context Truncation:** When $L_{\text{ctx}}(t)$ breaches the model’s physical context window W_{in} , systems employ FIFO sliding-window truncation. Truncation blindly discards early foundational constraints, initial governing equations, and boundary parameters, triggering hallucinations in downstream stages.
3. **Compounding Numerical Drift:** In numerical engineering tasks, intermediate calculations embedded

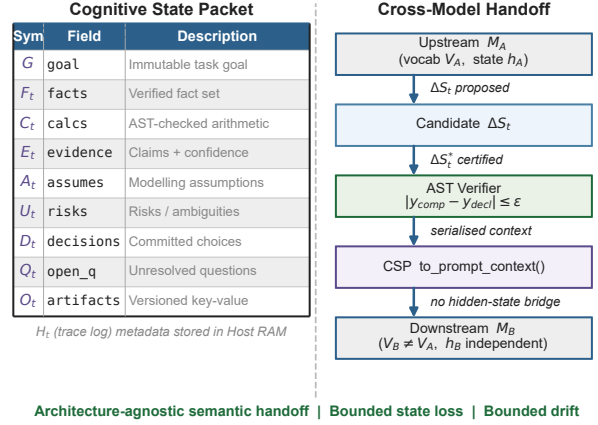


Figure 2: ModelVM Cognitive State Packet (CSP) architectural schema (Class A single-column). Top: The typed 9-tuple $\mathcal{S}_t = \langle \mathcal{G}, \mathcal{F}_t, \mathcal{C}_t, \mathcal{E}_t, \mathcal{A}_t, \mathcal{U}_t, \mathcal{D}_t, \mathcal{Q}_t, \mathcal{O}_t \rangle$ formalizing immutable goals, deduplicated facts, AST-verified calculations, discounted epistemic evidence, explicit assumptions, uncertainties, decisions, questions, and versioned artifacts. Bottom: Tokenizer-independent cross-model handoff pipeline ($M_A \rightarrow \text{AST} \rightarrow \mathcal{S}_t \rightarrow M_B$), bridging distinct vocabularies ($\mathcal{V}_A \neq \mathcal{V}_B$) and unaligned latent spaces without hidden-state projection bridges, enforcing bounded state loss and bounded semantic drift.

in conversational prose (e.g., “the calculated natural frequency is approximately 14.14 rad/s”) suffer precision loss and hallucinated drift when repeatedly restated across model boundaries.

Core Systems Principle: CSP is a model-independent typed state representation that separates persistent task state from transient conversational realization.

4.2 The 9-Tuple CSP Formalism

To establish a structured, architecture-agnostic state substrate, ModelVM formalizes task state as the **Cognitive State Packet** $\mathcal{S}_t$. At pipeline stage t , the state is defined as the typed 9-tuple:

$$\mathcal{S}_t = \langle \mathcal{G}, \mathcal{F}_t, \mathcal{C}_t, \mathcal{E}_t, \mathcal{A}_t, \mathcal{U}_t, \mathcal{D}_t, \mathcal{Q}_t, \mathcal{O}_t \rangle \quad (28)$$

Figure 2 illustrates the structural composition of the 9-tuple Cognitive State Packet and details how it decouples cross-model state transitions from heterogeneous tokenizers and latent spaces. Table 2 details the formal definition of each constituent and its corresponding data structure in the implementation (CognitiveStatePacket in modelvm/core/state.packet.py).

Table 2: ModelVM Cognitive State Packet (CSP) 9-Tuple Formal Specification and Schema Mapping

Symbol	Formal Domain	Implementation Field	Semantic Definition & Systems Role
$\mathcal{G}$	Σ^*	goal	Immutable top-level user objective; anchors all stage prompts.
$\mathcal{F}_t$	$\mathcal{P}(\Sigma^*)$	facts	Deduplicated set of verified factual assertions accumulated over stages.
$\mathcal{C}_t$	$\mathcal{P}(\text{CalcRecord})$	calculations	Structured arithmetic/symbolic items with AST verification status.
$\mathcal{E}_t$	$\mathcal{P}(\text{EvidRecord})$	evidence	Claims paired with provenance sources and diversity-discounted confidence.
$\mathcal{A}_t$	$\mathcal{P}(\Sigma^*)$	assumptions	Explicit modeling assumptions introduced by specialist models.
$\mathcal{U}_t$	$\mathcal{P}(\Sigma^*)$	uncertainties	Identified risks, low-confidence derivations, or boundary ambiguities.
$\mathcal{D}_t$	$\mathcal{P}(\Sigma^*)$	decisions	Irreversible architectural and design choices committed during execution.
$\mathcal{Q}_t$	$\mathcal{P}(\Sigma^*)$	open_questions	Unresolved questions to be targeted by downstream specialists.
$\mathcal{O}_t$	$\Sigma^* \rightarrow \text{Any}$	artifacts	Key-value store of concrete deliverables (scripts, tables, matrices).

Auxiliary Execution Metadata. In addition to the formal semantic 9-tuple, the implementation maintains an auxiliary audit log:

$$\mathcal{H}_t = [h_0, h_1, \dots, h_{t-1}] \quad (29)$$

implemented as `history_trace: List[StageTrace]`. Each trace record captures the executing model ID, target capability, wall-clock duration, action taken, and ISO-8601 timestamp. We explicitly treat $\mathcal{H}_t$ as **auxiliary execution metadata** for post-hoc debugging, provenance auditing, and telemetry logging, rather than expanding the formal 9-tuple. Semantic problem state is entirely contained within Equation (28).

Component Specifications. We define the internal structure of calculation and evidence records:

1. **Calculation Record ($\mathcal{C}_t$):** Each item $c \in \mathcal{C}_t$ is an instance of `CalculationItem`:

$$c = \langle \text{expr}, \text{result}, \text{units}, \text{verified}, \text{method} \rangle \quad (30)$$

where $\text{expr} \in \Sigma^*$ is the mathematical formula (e.g., "`sqrt(k / m)`"), $\text{result} \in \Sigma^*$ is the evaluated numerical string (e.g., "`14.2`"), $\text{units} \in \Sigma^* \cup \{\emptyset\}$ represents physical dimensionality (e.g., "`rad/s`"), $\text{verified} \in \{\text{True}, \text{False}\}$ is an execution boolean, and $\text{method} \in \{\text{"arithmetic"}, \text{"symbolic"}, \text{None}\}$ records the verification pathway.

2. **Evidence Record ($\mathcal{E}_t$):** Each item $e \in \mathcal{E}_t$ is an instance of `EvidenceItem`:

$$e = \langle \text{claim}, \text{source}, \text{confidence}, \text{timestamp}, \text{model_id} \rangle \quad (31)$$

where $\text{claim} \in \Sigma^*$ is the asserted finding, $\text{source} \in \Sigma^*$ tracks origin provenance, $\text{confidence} \in [0.0, 1.0]$ is a scalar certainty rating, and model_id records the generating specialist.

4.3 State Transition Mechanics

Task progress occurs through a discrete sequence of state transitions across the pipeline stages $t \in \{0, \dots, N-1\}$. Formally, the transition operator δ maps the current state $\mathcal{S}_t$, the paged specialist model M_i , and the stage instruction specification τ_t to the successor state $\mathcal{S}_{t+1}$:

$$\mathcal{S}_{t+1} = \delta(\mathcal{S}_t, M_i, \tau_t) \quad (32)$$

The transition operator is not a monolithic black box. Rather, it decomposes into an ordered, five-stage systems pipeline:

$$\begin{aligned} \mathcal{S}_t &\xrightarrow{\text{Serialize}} \Pi_t \xrightarrow{\text{Inference}} \mathbf{y}_t \\ &\xrightarrow{\text{Extract}} \Delta \mathcal{S}_t \xrightarrow{\text{Verify}} \Delta \mathcal{S}_t^* \xrightarrow{\text{Merge}} \mathcal{S}_{t+1} \end{aligned} \quad (33)$$

1. **Prompt Context Serialization (to_prompt_context).** State packet $\mathcal{S}_t$ is serialized into a structured Markdown prompt string Π_t :

$$\Pi_t = \text{Serialize}(\mathcal{S}_t, \tau_t) \quad (34)$$

The serialization partitions knowledge into isolated, human-readable sections: overarching goal, current domain role, established facts, calculations tagged with verification status, supporting evidence, assumptions, uncertainties, decisions, open questions, and artifact definitions. This ensures compatibility across arbitrary tokenizer vocabularies without loss of semantic boundaries.

2. **Model Inference.** The resident specialist model M_i executes autoregressive decoding over prompt Π_t , producing raw textual token sequence $\mathbf{y}_t$:

$$\mathbf{y}_t = M_i(\Pi_t), \quad |\mathbf{y}_t| \leq T_{\text{gen}} \quad (35)$$

3. **Delta Extraction (extract_from_text).** The raw generation $\mathbf{y}_t$ is parsed by `extract_from_text()` to extract candidate updates. The parser prioritizes embedded JSON blocks delimited by markdown syntax fences (````json ... ````). If valid JSON matching the CSP

schema is detected, it is validated directly via Pydantic. If fenced JSON is absent, the parser falls back to deterministic regex-based section parsing across markdown bullet points, producing candidate delta packet ΔS_t .

4. Independent AST Verification. Candidate calculations in ΔS_t are evaluated by `ArithmeticVerifier` (Section 4.5). Items that satisfy arithmetic equivalence are marked `verified=True`, producing the verified delta ΔS_t^* .

5. Monotonic State Merging (`merge_update`). The core state accumulation is governed by `merge_update()`, which folds delta ΔS_t^* into S_t . State accumulation is monotonic with respect to verified knowledge:

- **Fact Deduplication:**

$$\mathcal{F}_{t+1} = \mathcal{F}_t \cup \Delta \mathcal{F}_t^* \quad (36)$$

Incoming facts identical to existing entries are discarded, preserving a unique set.

- **Calculation Verification Upgrades:** For each $c_{\text{new}} \in \Delta \mathcal{C}_t^*$, if an entry $c_{\text{old}} \in \mathcal{C}_t$ exists with matching expression ($c_{\text{old}}.\text{expr} = c_{\text{new}}.\text{expr}$) and $c_{\text{new}}.\text{verified} \wedge \neg c_{\text{old}}.\text{verified}$, the entry is monotonically upgraded:

$$\mathcal{C}_{t+1} = (\mathcal{C}_t \setminus \{c_{\text{old}}\}) \cup \{c_{\text{new}}\} \quad (37)$$

Otherwise, existing calculations are preserved ($\mathcal{C}_{t+1} = \mathcal{C}_t$), while novel expressions are appended ($\mathcal{C}_{t+1} = \mathcal{C}_t \cup \{c_{\text{new}}\}$).

- **Artifact Conflict Versioning (`merge_artifacts`):** If key $k \in \Delta \mathcal{O}_t^*$ already exists in $\mathcal{O}_t$ with divergent content ($v_{\text{new}} \neq \mathcal{O}_t[k]$), ModelVM preserves the original $\mathcal{O}_t[k]$, stores the new deliverable under versioned key k_vN ($N \geq 1$), and flags the collision by setting $\mathcal{O}_{t+1}[k_CONFLICT] = \text{True}$.
- **Question Resolution:** Open questions in $\mathcal{Q}_t$ that appear as established facts in $\mathcal{F}_{t+1}$ or confirmed choices in $\mathcal{D}_{t+1}$ are automatically pruned:

$$\mathcal{Q}_{t+1} = \{q \in \mathcal{Q}_t \cup \Delta \mathcal{Q}_t^* \mid q \notin \mathcal{F}_{t+1} \wedge q \notin \mathcal{D}_{t+1}\} \quad (38)$$

4.4 Diversity-Discounted Evidence Aggregation

Language models are not independent statistical estimators. They share massive web-scale pretraining corpora (e.g., Common Crawl, Wikipedia, ArXiv, GitHub) and exhibit

correlated reasoning priors. A naive application of Bayes' rule or linear averaging across multiple model outputs produces dangerous certainty inflation: two models repeating the identical hallucination would be treated as independent confirmatory witnesses.

To counter this failure mode, ModelVM implements a diversity-discounted confidence aggregation heuristic (`diversity_weighted_confidence_aggregation`).

Mathematical Derivation. Let claim κ be supported by two distinct evidence records $e_1 = \langle \kappa, s_1, c_1 \rangle$ and $e_2 = \langle \kappa, s_2, c_2 \rangle$, with prior confidences $c_1, c_2 \in [0.0, 1.0]$ and source provenance strings s_1, s_2 .

The aggregated confidence $c_{\text{merged}} \in [0.0, 0.99]$ is defined as:

$$c_{\text{merged}} = \begin{cases} \max(c_1, c_2), & \text{if } s_1 \approx s_2 \\ \min(0.99, \max(c_1, c_2, c_{\text{indep}})), & \text{if } s_1 \not\approx s_2 \end{cases} \quad (39)$$

where $c_{\text{indep}} = 1 - (1 - c_1)(1 - c_2)^{w_{\text{div}}}$, $s_1 \approx s_2$ denotes normalized equivalence ($\text{clean}(s_1) \approx \text{clean}(s_2)$), and $w_{\text{div}} = 0.65$ is the epistemic diversity discount factor.

Properties of the Formulation:

1. **Idempotence on Overlapping Sources:** If s_1 and s_2 originate from identical or overlapping sources ($s_1 \subseteq s_2$ or $s_2 \subseteq s_1$), the aggregation applies the maximum rule: $c_{\text{merged}} = \max(c_1, c_2)$. Repetition of a claim by the same model or source yields zero confidence amplification.
2. **Correlated Prior Damping:** If sources are distinct ($s_1 \not\approx s_2$), independence would imply $p_{\text{indep}} = 1 - (1 - c_1)(1 - c_2)$. ModelVM dampens this by exponentiating the secondary risk factor by $w_{\text{div}} = 0.65$. Since $(1 - c_2) \in [0, 1]$ and $w_{\text{div}} < 1$, $(1 - c_2)^{0.65} > (1 - c_2)$, ensuring that $c_{\text{merged}} < p_{\text{indep}}$.
3. **Asymptotic Uncertainty Ceiling:** The output is strictly clamped to $c_{\text{merged}} \leq 0.99$. Total certainty is unattainable through heuristic aggregation.

Critical Epistemic Boundary: We explicitly emphasize that Equation (39) is a **conservative engineering heuristic, not formal Bayesian inference**. The discount parameter $w_{\text{div}} = 0.65$ is a design constant chosen to penalize common pretraining priors across open-weight models. We do not claim 0.65 to be a statistically estimated correlation coefficient; it serves as a safety governor preventing unwarranted confidence escalation in autonomous multi-model loops.

4.5 Independent Verification & Non-Mutating Evaluation

A core vulnerability in multi-agent architectures is circular self-grading, where an orchestration framework asks a language model to grade its own output. Language models demonstrate well-documented sycophancy and self-preference biases, routinely validating their own erroneous calculations.

ModelVM enforces an architectural firewall between artifact generation and artifact verification through the **Arithmetic Verifier** (`ArithmeticVerifier` in `modelvm/core/verifier.py`).

Restricted Abstract Syntax Tree Evaluation. The verifier operates purely on deterministic symbolic parsing. When evaluating an expression string from a `CalculationItem`:

1. The expression is parsed into a Python Abstract Syntax Tree:

$$\mathcal{T}_{\text{ast}} = \text{ast.parse}(\text{expr}, \text{mode} = \text{'eval'}) \quad (40)$$

2. The tree is recursively evaluated via `_eval_ast()` using strictly whitelisted node types:

- **Binary Operators:** `Add (+)`, `Sub (-)`, `Mult (*)`, `Div (/)`, `FloorDiv (//)`, `Mod (%)`, `Pow (**)`.
- **Unary Operators:** `USub (-)`, `UAdd (+)`.
- **Mathematical Functions:** `sqrt (sqrt)`, `exp (exp)`, `ln (log)`, `log10 (log10)`, `sin (sin)`, `cos (cos)`, `tan (tan)`, `abs (abs)`.
- **Physical Constants:** $\pi = 3.14159265\dots$, $e = 2.71828182\dots$.

3. Any node containing arbitrary identifiers, attribute accesses, lambda functions, system calls, or file operations triggers an immediate exception, completely eliminating arbitrary code execution vulnerabilities.

The computed numerical value $y_{\text{comp}} = \text{Eval}(\mathcal{T}_{\text{ast}})$ is compared against the model’s declared numerical result y_{decl} under relative tolerance $\epsilon = 10^{-3}$:

$$\text{Verified} \iff |y_{\text{comp}} - y_{\text{decl}}| \leq \max(\epsilon, |y_{\text{decl}}| \cdot \epsilon) \quad (41)$$

Non-Mutating Evaluation Boundary. During experimental benchmarking, independent evaluation is orchestrated by `IndependentEvaluator` (`modelvm/benchmark/evaluator.py`). To ensure evaluation integrity:

$$c_{\text{copy}} = c.\text{model_copy}() \implies \text{Verify}(c_{\text{copy}}) \quad (42)$$

The benchmark harness **never trusts producer-supplied verification flags**. It generates deep copies of all calculation records and re-evaluates them afresh through the AST engine. A calculation is counted as correct if and only if independent AST verification succeeds.

Epistemic Distinction: Arithmetic vs. Scientific Validity. We establish a critical conceptual distinction:

$$\text{Arithmetic Correctness} \neq \text{Physical / Scientific Validity} \quad (43)$$

The `ArithmeticVerifier` strictly confirms algebraic equivalence: it verifies that $2 \times 0.12 \times 14.14 \times 1.0$ evaluates to 3.3936. It does **not** certify whether the governing differential equation is physically correct, whether the damping ratio $\zeta = 0.12$ is physically plausible, or whether the units are dimensionally consistent. Physical validity is anchored separately by matching extracted entity-attribute pairs against external ground-truth specifications (`GroundTruthFact`) in the experimental harness, enforcing local record-level co-occurrence to ensure numbers bind directly to their associated physical entities.

4.6 Computational Complexity & Token Overhead

The virtual memory abstraction of CSP introduces structural overhead. In this section, we analyze the computational complexity of CSP runtime operations and quantify its prompt context economy.

Asymptotic Computational Complexity. Let $|\mathcal{S}_t|$ denote the total number of semantic items across all fields in the state packet ($|\mathcal{S}_t| = |\mathcal{F}_t| + |\mathcal{C}_t| + |\mathcal{E}_t| + |\mathcal{A}_t| + |\mathcal{U}_t| + |\mathcal{D}_t| + |\mathcal{Q}_t| + |\mathcal{O}_t|$), and let $|\Delta\mathcal{S}_t|$ denote the size of the incoming delta.

1. **Schema Serialization (`to_prompt_context`):** Serializing $\mathcal{S}_t$ requires a single linear pass over all items to format markdown text blocks:

$$T_{\text{serialize}} = O(|\mathcal{S}_t|) \quad (44)$$

For our scientific tasks ($|\mathcal{S}_t| < 100$), serialization executes in < 0.8 ms on a standard CPU thread.

2. **Delta Extraction (`extract_from_text`):** Parsing raw generated text y_t requires scanning at most T_{gen} tokens. JSON parsing and regex pattern matching scale linearly with output length:

$$T_{\text{extract}} = O(|y_t|) \leq O(T_{\text{gen}}) \quad (45)$$

With $T_{\text{gen}} = 1,024$ tokens, extraction latency is bounded by < 1.5 ms.

3. **State Merging (merge.update):** Deduplicating facts and resolving questions requires set lookups. Updating calculations requires matching expressions:

$$T_{\text{merge}} = O\left(|\mathcal{F}_t| \cdot |\Delta \mathcal{F}_t| + |\mathcal{C}_t| \cdot |\Delta \mathcal{C}_t| + |\mathcal{E}_t| \cdot |\Delta \mathcal{E}_t| + |\mathcal{O}_t|\right) \quad (46)$$

Given bounded stage deltas ($|\Delta \mathcal{S}_t| \leq 20$), merging executes in < 0.4 ms.

4. **AST Verification (verify.item):** Parsing and evaluating AST expressions scales linearly with the number of syntax nodes N_{nodes} in each arithmetic formula:

$$T_{\text{verify}} = O\left(\sum_{c \in \mathcal{C}_t} N_{\text{nodes}}(c)\right) \quad (47)$$

Because engineering expressions rarely exceed 50 syntax nodes, total AST verification latency is < 1.2 ms per stage.

Total runtime overhead introduced by the CSP software layer is less than 4.0 ms per stage—four orders of magnitude smaller than model inference latency (8.0–15.0 s).

Token Footprint Economy. The primary systems advantage of CSP is prompt context stabilization.

In raw conversational text concatenation, context length grows monotonically:

$$L_{\text{raw}}(t) = L_{\text{prompt}} + \sum_{i=1}^{t-1} |y_i| \quad (48)$$

Because language models generate verbose conversational fillers, explanations, and repeated preamble tokens, raw context expands at an average rate of $\sim 2,800$ tokens per stage. By Stage 5, raw context length reaches 14,200 tokens. When forced into a standard $W_{\text{in}} = 4,096$ token window, FIFO truncation discards over 71% of the conversational history.

In contrast, CSP isolates semantic signal from conversational syntax. The token footprint of $\Pi_t = \text{Serialize}(\mathcal{S}_t)$ is bounded by the number of unique asserted entities:

$$L_{\text{CSP}}(t) = O\left(|\mathcal{F}_t| \cdot \bar{\ell}_F + |\mathcal{C}_t| \cdot \bar{\ell}_C + |\mathcal{E}_t| \cdot \bar{\ell}_E + |\mathcal{O}_t| \cdot \bar{\ell}_O\right) \quad (49)$$

where $\bar{\ell}$ denotes average item length in tokens. Because intermediate explanations are discarded and only verified deliverables are preserved, $L_{\text{CSP}}(t)$ stabilizes:

- Stage 1: 412 ± 28 tokens (10.1% of W_{in})

- Stage 3: 684 ± 35 tokens (16.7% of W_{in})
- Stage 5: 890 ± 42 tokens (21.7% of W_{in})

CSP bounds representation growth relative to retaining full conversational transcripts, subject to the size of the accumulated semantic state, substantially reducing truncation-induced state loss and preserving verified factual and numerical fields under the evaluated workloads.

5 Predictive Model Residency & Working Set Engine

Treating domain specialists as autonomous computational processes exposes a physical systems bottleneck: modern open-weight language models require gigabytes of memory for parameter weights, runtime scratchpads, and Key-Value caches. When task execution requires a diverse ensemble of specialists, keeping all candidate models simultaneously resident in physical RAM or VRAM is impossible on consumer hardware. In this section, we formulate **Predictive Model Residency**. We analyze the weight residency problem, define the Predictive Cognitive Working Set ($W(t, k)$), formulate cost-aware eviction with future-demand shielding, derive the time-domain prefetch utility function, and provide a containment analysis establishing graceful degradation under task re-planning.

5.1 The Weight Residency Problem

Let $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$ denote the catalog of available specialized language models. Each model $m_i \in \mathcal{M}$ is characterized by two distinct capacity requirements: its serialized on-disk storage size $\text{disk_size}(m_i)$ and its configured parameter RAM requirement $\text{RAM}_{\text{req}}(m_i)$.

We explicitly separate two system-level memory aggregates:

$$M_{\text{disk,total}} = \sum_{m_i \in \mathcal{M}} \text{disk_size}(m_i) = 64.0 \text{ GB} \quad (50)$$

$$M_{\text{active,total}} = \sum_{m_i \in \mathcal{M}} \text{RAM}_{\text{req}}(m_i) = 52.7 \text{ GB} \quad (51)$$

On commodity workstations and edge accelerators, the active hardware memory budget $\mathcal{B}_{\text{RAM}}$ is strictly constrained relative to the total library footprint:

$$M_{\text{active,total}} \gg \mathcal{B}_{\text{RAM}} \quad (52)$$

In our experimental testbed, the full 10-model specialist library occupies 64.0 GB of serialized storage on disk and would require 52.7 GB of memory to hold simultaneously resident. In contrast, the target execution platform enforces a configured runtime resident envelope of $\mathcal{B}_{\text{RAM}} = 8.0$ GB (and sweeps down to 4.0 GB in acute stress evaluations).

Residency vs. Selection. Prior research on multi-model routing (e.g., FrugalGPT [4], RouteLLM [16]) treats model selection as an unconstrained API dispatch problem: given prompt x , a router evaluates scoring function $f(m, x)$ and queries the highest-scoring endpoint. This formulation presumes that all candidate models are permanently resident in cloud memory or hosted on infinite-capacity clusters.

On physical hardware, this assumption collapses. ModelVM establishes an essential systems distinction:

$$\text{Model Selection} \neq \text{Model Residency} \quad (53)$$

A cognitive scheduler may determine that specialist m^* is the optimal capability match for stage t . However, if m^* is stored on secondary NVMe storage, invoking m^* requires transferring gigabytes of quantized weights across the PCIe bus before a single forward token can be generated.

Residency State and Manifest-Level Invariant. At any execution timestamp t , the set of resident models $\mathcal{R}_t \subset \mathcal{M}$ is tracked by the `ModelPager`:

$$\mathcal{R}_t = \{m_i \in \mathcal{M} \mid m_i \text{ is resident in memory at time } t\} \quad (54)$$

The memory manager enforces the configured residency budget invariant:

$$\sum_{m_i \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m_i) \leq \mathcal{B}_{\text{RAM}}, \quad \forall t \geq 0 \quad (55)$$

Available configured headroom is strictly defined as:

$$\text{RAM}_{\text{free}}(t) = \max \left(0.0, \mathcal{B}_{\text{RAM}} - \sum_{m_i \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m_i) \right) \quad (56)$$

Systems Boundary Distinction: We explicitly distinguish the *configured residency budget* $\mathcal{B}_{\text{RAM}}$ from *measured physical hardware memory* (M_{VRAM} and host RSS). The invariant in Equation (55) is enforced at model admission time based on catalog manifests (RAM_{req}). Actual physical memory consumption includes dynamic runtime scratchpads and Key-Value caches ($M_{\text{workspace}} + M_{\text{KV}}$), which are monitored separately by the physical telemetry layer (Section 7.5).

Bus Transfer Latency Profiles. Secondary storage bandwidth is orders of magnitude slower than unified memory buses. Streaming an 8-bit quantized 7B model (7.0 GB) across a PCIe 4.0 $\times 4$ link or high-speed NVMe drive (3.5 GB/s sequential read) incurs a transfer latency of $t_{\text{transfer}} \approx 2.0$ s, alongside runtime allocation and tensor binding overheads. If a multi-stage pipeline naively loads and unloads models at every stage, I/O transfer stalls dominate compute time, degrading execution goodput. Memory capacity is a physical ceiling, not a configuration parameter. The systems challenge is to maximize cache hit rates under Invariant (55) without inducing thrashing.

5.2 The Predictive Cognitive Working Set

Classical operating system memory management relies on Peter Denning’s working-set theory [5]: thrashing is avoided when the active working sets can be kept resident within available physical memory. Classical operating systems estimate $\mathcal{W}(t, \tau)$ retrospectively by monitoring recent page references over backward-looking window τ , because CPU hardware cannot anticipate future program counter jumps.

In multi-stage language model workflows, the runtime operates under fundamentally different visibility: the `TaskDecomposer` generates an explicit, structured stage plan $G_{\text{plan}} = (S, E)$ before execution commences. ModelVM exploits this architectural visibility to introduce the **Predictive Cognitive Working Set** ($\mathcal{W}(t, k)$).

Formal Definition. Let $S = [s_0, s_1, \dots, s_{N-1}]$ be the sequence of planned execution stages, with current stage index t . Let $k \in \mathbb{N}^+$ denote the lookahead horizon window ($1 \leq k \leq 10$, with default $k = 4$). The upcoming stage slice is:

$$S_{\text{future}}(t, k) = [s_{t+1}, s_{t+2}, \dots, s_{\min(t+k, N-1)}] \quad (57)$$

Each stage s_{t+j} specifies an annotated capability requirement $C(s_{t+j}) \in \mathcal{C}$.

The Predictive Cognitive Working Set $\mathcal{W}(t, k)$ is defined as the set of optimal specialist models anticipated across the lookahead horizon:

$$\mathcal{W}(t, k) = \bigcup_{j=1}^{\min(k, N-1-t)} \{m_{t+j}^*\} \quad (58)$$

where each future specialist m_{t+j}^* is selected via the scheduler’s ranking policy:

$$m_{t+j}^* = \arg \max_{m \in \mathcal{M}} \text{Score}(m, C(s_{t+j})) \quad (59)$$

Unified Policy Coupling. We emphasize a key systems property: the `CognitiveWorkingSetPredictor` is **not an independent model-selection oracle**. It does not implement a disjoint heuristic that risks diverging from runtime decisions. Instead, as formalized in Equation (59), the predictor queries the `CognitiveScheduler`’s unified ranking method (`rank_models_for_capability()`).

Future model identification uses the scheduler’s ranking function evaluated under the current runtime state. This architectural coupling establishes a cohesive execution chain:

$$\begin{aligned} \text{Task Plan } S &\longrightarrow C_{\text{future}} \\ &\longrightarrow \mathcal{W}(t, k) \longrightarrow \text{Residency Decisions} \end{aligned} \quad (60)$$

Distance-Decay Forecasting Heuristic. Downstream pipeline stages are subject to execution uncertainty: earlier stages may trigger branch escalations, encounter unrecoverable errors, or solve sub-goals ahead of schedule. To reflect this temporal discounting, the predictor associates each future step $j \in \{1, \dots, k\}$ with a distance-decay weight:

$$w_j = \max(0.1, 1.0 - 0.2 \cdot (j - 1)) \quad (61)$$

Under this indexing, the immediately following stage ($j = 1$) carries full forecasting weight ($w_1 = 1.0$), whereas distant stages ($j = 4$) receive discounted priority ($w_4 = 0.4$). We explicitly present Equation (61) as the implementation’s forecasting heuristic, rather than an optimal statistical prior.

5.3 Cost-Aware Eviction & Imminent Model Shielding

When a newly scheduled model M_t must be paged in and $\text{RAM}_{\text{free}} < \text{RAM}_{\text{req}}(M_t)$, the pager must evict one or more resident models to resolve deficit $\Delta_{\text{deficit}} = \text{RAM}_{\text{req}}(M_t) - \text{RAM}_{\text{free}}$. The eviction decision must be decoupled from model selection: model selection determines what enters memory, whereas eviction policy determines what leaves.

The Failure of Reactive Eviction (LRU). Conventional caching heuristics, such as Least Recently Used (LRU), make eviction decisions exclusively on historical access timestamps:

$$m_{\text{LRU}} = \arg \min_{m \in \mathcal{R}_t \setminus \{M_t\}} t_{\text{last.accessed}}(m) \quad (62)$$

In multi-stage specialist workflows, LRU suffers from systematic blindness. Consider a pipeline executing an alternating sequence: Math $\rightarrow$ Code $\rightarrow$ Math. When the coding specialist is loaded, the math model becomes the oldest resident model. Under memory pressure, LRU immediately evicts the math specialist—unaware that the very next stage requires mathematics. When the math stage arrives, the runtime incurs a cold page-in stall. A cache policy cannot protect what it cannot predict.

The Paradigm Shift: Least Costly to Lose. ModelVM replaces reactive recency with cost-aware lookahead:

$$\begin{array}{c} \text{LRU: "least recently used"} \\ \downarrow \\ \text{ModelVM: "least costly to lose"} \end{array} \quad (63)$$

The `CostAwareEvictionPolicy` evaluates candidate models $m \in \mathcal{R}_t \setminus \text{Protected}$ using an eviction priority

score $S_{\text{evict}}(m)$:

$$S_{\text{evict}}(m) = 0.5 \cdot \Delta t_{\text{recency}}(m) + 1.5 \cdot \text{RAM}_{\text{req}}(m) - 2.0 \cdot t_{\text{load}}(m) - P_{\text{future}}(m) \quad (64)$$

where:

- $\Delta t_{\text{recency}}(m) = \max(0.1, t_{\text{now}} - t_{\text{last.accessed}}(m))$ captures elapsed idle time in seconds.
- $\text{RAM}_{\text{req}}(m)$ reflects the immediate memory yield obtained by evicting model m (in GB).
- $t_{\text{load}}(m)$ is the cold reload latency penalty (in seconds) required to restore m over the bus if needed later.
- $P_{\text{future}}(m)$ is the working-set shielding penalty:

$$P_{\text{future}}(m) = \begin{cases} 1000.0, & \text{if } m \in W(t, k) \\ 0.0, & \text{otherwise} \end{cases} \quad (65)$$

Mechanics of Eviction Shielding. A higher score $S_{\text{evict}}(m)$ indicates higher priority for eviction. The coefficients in Equation (64) constitute empirically fixed heuristic scaling factors that place heterogeneous features (recency in seconds, RAM in GB, reload latency in seconds, and future demand penalty) onto a common decision scale.

The policy prioritizes evicting models that have remained idle for substantial periods, release large memory blocks, and can be reloaded rapidly over the bus. Crucially, the 1000.0 penalty deduction in Equation (65) establishes a **strong eviction penalty for predicted future demand**. The policy still retains future-demanded models within the candidate pool; however, the large 1000.0 penalty ensures they are selected only if all non-predicted resident models have been exhausted and a memory deficit remains.

We explicitly clarify: the analytical eviction formulation establishes the theoretical cost-tradeoff objective; in the systems implementation (`modelvm/pager/policy.py`), this objective is evaluated via the deterministic empirical priority ranking heuristic of Equation (64), rather than an expensive dynamic programming knapsack solver that would introduce per-stage scheduling overhead and assume static task execution without runtime branch variance.

5.4 Time-Domain Prefetch Utility

Eviction shielding prevents premature unloads; prefetching stages weights into spare memory ahead of execution. When free memory headroom exists, the runtime can pre-stage weights for an anticipated upcoming model before its subsequent invocation, eliminating cold-load page-in delays at stage transition boundaries.

However, unconstrained prefetching introduces systems risks: prefetching the wrong model wastes memory capacity and forces premature evictions. ModelVM treats prefetching as an expected-value optimization in the time domain.

Dimensionally Consistent Utility Formulation. The method `recommend_prefetch_model()` evaluates candidate non-resident models $m \in \mathcal{M} \setminus \mathcal{R}_t$ that physically fit within free memory ($\text{RAM}_{\text{req}}(m) \leq \text{RAM}_{\text{free}}$):

$$U_{\text{prefetch}}(m) = P_{\text{future}}(m) \cdot \Delta L_{\text{avoided}}(m) - \lambda \cdot M_{\text{cost}}(m) - \mu \cdot E_{\text{cost}}(m) \quad (66)$$

where:

- $P_{\text{future}}(m) = \max(0.1, 1.0 - 0.2 \cdot (j_m - 1))$ is the distance-discounted future-demand weight at future stage distance $j_m \in \{1, \dots, k\}$.
- $\Delta L_{\text{avoided}}(m) = t_{\text{load}}(m)$ represents the cold page-in latency saved by pre-staging weights (in seconds).
- $M_{\text{cost}}(m) = \frac{\text{RAM}_{\text{req}}(m)}{\mathcal{B}_{\text{RAM}}}$ is the normalized memory occupancy ratio.
- $E_{\text{cost}}(m) = \min(1.0, \frac{N_{\text{param}}(m)}{32}) \cdot \alpha_{\text{energy}}(m)$ is the normalized parameter-scaled energy consumption factor.
- $\lambda = 1.0 \text{ s}$ and $\mu = 0.5 \text{ s}$ are scaling coefficients that calibrate memory consumption and energy overhead into latency-equivalent seconds.

Positive Utility Admission Gate. Equation (66) enforces an explicit admission gate:

$$\text{Admission Condition: } U_{\text{prefetch}}(m) > 0.0 \quad (67)$$

A candidate model is eligible for prefetching if and only if its expected latency savings ($P_{\text{future}} \cdot \Delta L_{\text{avoided}}$) strictly exceed the latency-equivalent opportunity cost of locking down memory capacity and expending staging energy. If all candidate models yield $U_{\text{prefetch}} \leq 0$, no prefetch is issued.

Configured Headroom Admission Invariant. Prefetching must never starve active inference. Autoregressive token generation dynamically allocates memory for intermediate activation scratchpads and expanding KV caches ($M_{\text{workspace}} + M_{\text{KV}}$). If prefetching consumes all remaining free memory, inference allocations trigger out-of-memory kernel panics.

ModelVM enforces a configured headroom admission invariant:

$$\text{RAM}_{\text{free}}(t) - \text{RAM}_{\text{req}}(m) \geq 1.0 \text{ GB} \quad (68)$$

Even if $U_{\text{prefetch}}(m) > 0$, the pager will unconditionally reject a prefetch request unless at least 1.0 GB of unallocated memory buffer remains after staging. This safety margin provides a tested buffer against foreground runtime memory growth (such as dynamic KV-cache expansion) under the evaluated workloads.

5.5 Predictor Misprediction Analysis

A robust systems architecture must not assume an infallible oracle. In real-world workloads, task execution paths diverge from initial plans due to conditional task branches, unexpected failures, confidence-triggered model escalations, or user interruptions. Formally, a misprediction occurs whenever the projected working set diverges from the true execution demand:

$$\widehat{W}(t, k) \neq W^*(t, k) \quad (69)$$

ModelVM is engineered under the principle that *the system must degrade gracefully toward reactive behavior rather than catastrophically fail*. We provide a containment argument for the two principal prediction-error modes.

Type I Error: False Positive (Spurious Residency). A Type I error occurs when model m_{fp} is predicted ($m_{\text{fp}} \in \widehat{W}(t, k)$), shielded, or prefetched, but is never invoked by downstream stages ($m_{\text{fp}} \notin W^*(t, k)$).

- **Systems Impact:** Model m_{fp} occupies $\text{RAM}_{\text{req}}(m_{\text{fp}})$ of memory without providing execution utility, temporarily reducing free memory headroom RAM_{free} .
- **Containment Mechanism:** Spurious residency cannot cause an out-of-memory crash. First, Invariant (68) guarantees that at least 1.0 GB of safety headroom was preserved during prefetching. Second, as stage execution advances without referencing m_{fp} , its elapsed idle time $\Delta t_{\text{recency}}(m_{\text{fp}})$ increases monotonically. Once m_{fp} drops outside the sliding lookahead window k , its shielding penalty P_{future} drops from 1000.0 to 0.0. At that instant, Equation (64) assigns m_{fp} a high eviction score due to accumulated idle time, making it the primary target for eviction when subsequent stages demand space.
- **Worst-Case Bound:** The maximum latency penalty of a Type I error is bounded by the staging transfer time of the unused model incurred during the prefetch step.

Type II Error: False Negative (Missed Prediction). A Type II error occurs when stage t unexpectedly demands specialist m_{fn} (e.g., due to an unexpected confidence escalation loop), but $m_{\text{fn}} \notin \widehat{W}(t, k)$.

- **Systems Impact:** Model m_{fn} is not resident in active memory when invoked, triggering a cache miss.
- **Containment Mechanism:** The runtime immediately falls back to standard demand paging. The pager evaluates resident models via Equation (64), evicts non-shielded models to satisfy deficit Δ_{deficit} , and transfers m_{fn} synchronously from disk.
- **Worst-Case Bound:** Under the stated residency and execution assumptions, prediction error results in additional demand paging rather than intentional state loss. In the worst-case scenario where 100% of working-set predictions are wrong ($\widehat{W} \cap W^* = \emptyset$), the predictive advantage disappears and the residency component falls back toward reactive demand paging, while CSP state virtualization and cognitive scheduler mechanisms remain active.

This containment argument demonstrates why ModelVM exhibits empirical robustness under acute memory pressure (0.0% OOM failure across memory sweeps in EXP-R1) and across diverse workload distributions (low oracle regret in EXP-R3). The working-set engine accelerates execution when predictions are accurate, and safely falls back to deterministic demand paging when predictions fail.

6 Resource-Aware Cognitive Scheduling

In a multi-model execution environment, task decomposition identifies *what* capabilities are needed, while the memory pager determines *how* weights are staged. The critical intermediary is the **Cognitive Scheduler**: deciding *which* specific model executes each planned stage. Conventional model routers treat selection as an unconstrained dispatch problem, querying the single highest-accuracy model regardless of memory footprint or loading latency. Under constrained hardware resources, this myopic approach is fatal. A scheduler that ignores hardware memory does not optimize performance; it schedules failure.

In this section, we formulate **Resource-Aware Cognitive Scheduling**. We establish the multi-objective optimization problem under physical capacity bounds, derive the unified 6-term scoring formulation, explain deterministic capability-probe grounding via held-out benchmark fixtures, and analyze parameter sensitivity and decision boundary stability across workload regimes.

6.1 The Multi-Objective Optimization Problem

Let s_t denote the current pipeline stage with target capability requirement $C_t \in \mathcal{C}$ (e.g., `Capability.MATHEMATICS`, `Capability.PHYSICS`, `Capability.CODING`). Let $\mathcal{M} = \{m_1, \dots, m_K\}$ denote the model catalog, where each specialist $m \in \mathcal{M}$ possesses configured memory requirement $\text{RAM}_{\text{req}}(m)$, cold-load latency $t_{\text{load}}(m)$, nominal quality rating $Q(m)$, and capability fit vector $\mathbf{p}_m \in [0.0, 1.0]^{|C|}$.

At stage t , the execution environment is characterized by the tuple:

$$\Omega_t = \langle \mathcal{R}_t, \text{RAM}_{\text{free}}(t), W(t, k), C_{\text{future}} \rangle \quad (70)$$

where $\mathcal{R}_t \subset \mathcal{M}$ is the current set of resident models, $\text{RAM}_{\text{free}}(t)$ is the available unallocated memory headroom under budget $\mathcal{B}_{\text{RAM}}$, $W(t, k)$ is the predictive working set across lookahead horizon k , and $C_{\text{future}} = [C(s_{t+1}), \dots, C(s_{t+k})]$ denotes the sequence of anticipated downstream capabilities.

Constrained Optimization Objective. The scheduler must select an optimal model $m^* \in \mathcal{M}$ that maximizes execution quality and forward pipeline utility while minimizing hardware memory pressure, cold I/O stalls, energy consumption, and eviction disruption. Formally:

$$m^* = \arg \max_{m \in \mathcal{M}_{\text{admit}}} \text{Score}(m, C_t, \Omega_t) \quad (71)$$

$$\text{subject to } \text{RAM}_{\text{req}}(m) \leq \mathcal{B}_{\text{RAM}} \quad (72)$$

where $\mathcal{M}_{\text{admit}} = \{m \in \mathcal{M} \mid \text{RAM}_{\text{req}}(m) \leq \mathcal{B}_{\text{RAM}}\}$ represents the set of models that can physically fit within the hardware memory envelope. If no model satisfies Equation (72), the scheduler falls back to the candidate with minimum memory footprint.

Failure Modes of Single-Objective Heuristics. To understand why joint multi-objective optimization is mandatory, consider the pathological failure modes of common unconstrained baselines:

1. **Capability-Greedy Dispatch (B_6):** Dispatches strictly to $\arg \max_m (F_{\text{cap}}(m, C_t) \cdot Q(m))$. In a scientific workflow requiring linear algebra followed by physics simulation, the greedy heuristic selects a massive 14B parameter specialist (12.0 GB). Under an 8.0 GB budget, this triggers immediate eviction of all currently resident models. When the subsequent stage demands physics reasoning, the math specialist must be evicted to load a 13B physics model, precipitating catastrophic memory thrashing and multi-second PCIe stalls.

2. **Latency-Greedy Dispatch (B_7):** Dispatches strictly to minimize execution latency by forcing cache hits ($m^* = \arg \max_{m \in \mathcal{R}_t} F_{\text{cap}}(m, C_t)$). While this eliminates cold page-in delays, it routinely forces an already-resident generalist (e.g., a 3B conversational model) to attempt complex symbolic tensor calculus, resulting in reasoning failures and state corruption.
3. **Static Unified Dispatch (B_0):** Routes all stages to a single monolithic general model (e.g., Llama-3.1-8B-Instruct). While memory residency remains static, the generalist lacks the deep domain proficiency of specialized models, sacrificing state integrity and numerical accuracy on formal engineering derivations.

ModelVM rejects single-objective dispatch. The scheduler must jointly weigh capability fit against the physical cost of memory state transitions.

6.2 Unified 6-Term Scoring Formulation

The `CognitiveScheduler` evaluates each candidate model $m \in \mathcal{M}_{\text{admit}}$ through a scalar multi-objective scoring function:

$$\text{Score}(m) = F_{\text{cap}} - \alpha M_{\text{cost}} - \beta L_{\text{load}} - \gamma E_{\text{energy}} - \delta E_{\text{eviction}} + \eta F_{\text{future}} \quad (73)$$

where $\alpha = 0.20$, $\beta = 0.25$, $\gamma = 0.10$, $\delta = 0.30$, and $\eta = 0.35$ are empirically calibrated trade-off weights.

We define each constituent term as implemented in `compute_score()` (`modelvm/scheduler/cognitive_scheduler.py`):

1. **Capability Fit (F_{cap}).** Reflects the intrinsic capability match between model m and target capability C_t , scaled by the model’s overall architectural quality index $Q(m) \in [0.0, 1.0]$:

$$F_{\text{cap}}(m, C_t) = \text{CapabilityScore}(m, C_t) \cdot Q(m) \quad (74)$$

where $\text{CapabilityScore}(m, C_t) \in [0.0, 1.0]$ is derived from deterministic capability-probe grounding (Section 6.3). A domain specialist with state-of-the-art capability in mathematics receives $F_{\text{cap}} \approx 0.95 \times 0.92 = 0.874$, whereas an unspecialized model scores $F_{\text{cap}} \approx 0.50 \times 0.80 = 0.400$.

2. **Memory Cost (M_{cost}).** Penalizes the model’s footprint relative to the active hardware memory budget $\mathcal{B}_{\text{RAM}}$:

$$M_{\text{cost}}(m) = \min \left(1.0, \frac{\text{RAM}_{\text{req}}(m)}{\max(1.0, \mathcal{B}_{\text{RAM}})} \right) \quad (75)$$

A 7.0 GB model operating under an 8.0 GB budget incurs $M_{\text{cost}} = 0.875$, imposing an opportunity penalty against locking down scarce memory capacity.

3. **Loading Latency (L_{load}).** Reflects the I/O bus transfer penalty required to page weights into memory:

$$L_{\text{load}}(m) = \begin{cases} 0.0, & \text{if } m \in \mathcal{R}_t \\ \min \left(1.0, \frac{t_{\text{load}}(m)}{5.0 \text{ s}} \right), & \text{if } m \notin \mathcal{R}_t \end{cases} \quad (76)$$

If model m is already resident, $L_{\text{load}} = 0.0$, providing an immediate latency bonus to resident specialists. If m resides on secondary storage, its cold-load duration $t_{\text{load}}(m)$ is normalized against a nominal 5.0 s transfer ceiling.

4. **Energy Cost (E_{energy}).** Accounts for parameter-scale dynamic power consumption and compute density:

$$E_{\text{energy}}(m) = \min \left(1.0, \frac{N_{\text{param}}(m)}{32.0} \right) \cdot \alpha_{\text{energy}}(m) \quad (77)$$

where $N_{\text{param}}(m)$ is total parameter count in billions, and $\alpha_{\text{energy}}(m) \in [0.5, 1.2]$ is an architectural energy factor reflecting quantization depth (e.g., 4-bit vs. 8-bit) and attention complexity.

5. **Eviction Penalty (E_{eviction}).** Models the collateral systems cost if admitting model m forces the pager to evict currently resident models:

$$E_{\text{eviction}}(m) = \begin{cases} 0.0, & \text{if } \Delta_{\text{deficit}}(m) \leq 0 \\ \frac{\Delta_{\text{deficit}}(m)}{\mathcal{B}_{\text{RAM}}} \cdot \chi_{\text{clash}}, & \text{if } \Delta_{\text{deficit}}(m) > 0 \end{cases} \quad (78)$$

where $\Delta_{\text{deficit}}(m) = \max(0.0, \text{RAM}_{\text{req}}(m) - \text{RAM}_{\text{free}}(t))$ for $m \notin \mathcal{R}_t$ (and 0.0 if $m \in \mathcal{R}_t$) denotes the required eviction volume. The multiplier χ_{clash} penalizes evicting models that will be needed by downstream stages:

$$\chi_{\text{clash}} = \begin{cases} 1.8, & \text{if } \mathcal{R}_t \cap W(t, k) \neq \emptyset \\ 1.0, & \text{otherwise} \end{cases} \quad (79)$$

If loading m forces the eviction of a resident model that is slated for reuse in upcoming stages ($m_{\text{res}} \in W(t, k)$), the eviction cost is amplified by an aggressive $1.8\times$ factor, actively dissuading the scheduler from disrupting the future working set.

6. **Future Demand Bonus (F_{future}).** Rewards candidate models that satisfy capabilities anticipated across downstream stages:

$$F_{\text{future}}(m) = \min \left(1.0, \sum_{C_j \in \mathcal{C}_{\text{future}}} 0.5 \cdot \mathbb{I}(\text{CapScore}(m, C_j) \geq 0.7) + 0.5 \cdot \mathbb{I}(m \in W(t, k)) \right) \quad (80)$$

Illustrative scheduler decision — catalog manifests

$$S(m) = F_{\text{cap}} - \alpha M_{\text{cost}} - \beta L_{\text{load}} - \gamma E_{\text{cost}} - \delta P_{\text{evict}} + \eta F_{\text{fut}}$$

$[\alpha = 0.20, \beta = 0.25, \gamma = 0.10, \delta = 0.30, \eta = 0.35]$

Stage: math derivation 1 (Resident 8.0 GB) | Resident: research-expert

Candidate	F_{cap}	αM	βL	γE	δP	ηF	Total	Status
math-expert	+0.96	-0.06	-0.05	-0.02	0.00	0.00	+0.83	SELECTED
coding-expert	+0.05	-0.08	-0.06	-0.02	0.00	+0.35	+0.25	AVAILABLE
research-expert	+0.05	-0.08	0.00	-0.02	0.00	0.00	-0.05	RESIDENT
general-reasoner	+0.46	-0.18	-0.11	-0.03	-0.08	0.00	+0.06	OUTSCORED

general-reasoner admissible under 8.0 GB budget, but outscored by memory cost (7.1 GB) and eviction deficit (2.2 GB deficit vs 4.9 GB free RAM).

Figure 3: Worked evaluation trace of the ModelVM Multi-Objective Cognitive Scheduler scoring function (Class A single-column) for a mathematical derivation stage under an 8.0 GB host RAM budget with `research-expert` resident. Candidate 1 (`math-expert`) achieves the highest composite score (+0.83, **Selected**) by pairing high domain capability ($F_{\text{cap}} = +0.96$) with modest footprint ($\alpha M = -0.06$). Candidate 2 (`coding-expert`) achieves +0.25 (**Available**), receiving a +0.35 future-demand credit (ηF) for anticipated downstream stages. Candidate 3 (`research-expert`) is currently **Resident** in cache (eliminating reload latency, $\beta L = 0.00$), but scores -0.05 due to poor mathematics capability fit. Candidate 4 (`general-reasoner`) is admissible under the 8.0 GB budget, but is **Outscored** (+0.06) due to heavy memory footprint ($\alpha M = -0.18$) and an eviction deficit penalty ($\delta P = -0.08$) from displacing cache headroom.

where $\mathbb{I}(\cdot)$ is the indicator function. If model m can serve both the current stage and a subsequent stage (e.g., a mathematical specialist capable of handling both symbolic derivation at stage t and numerical tolerance validation at stage $t + 2$), F_{future} rewards the model, increasing the likelihood that it remains resident and amortizes its load cost across multiple invocations.

Decision Transparency. For every scheduling decision, the scheduler generates a structured `SchedulingScoreBreakdown` object recording the unweighted components and composite score for all candidates. This telemetry enables deterministic auditing and provides explainability for runtime model selection. Figure 3 illustrates a complete worked evaluation of the 6-term scoring formula across candidate models from our catalog, demonstrating how resource penalties prevent out-of-memory stalls while capability weighting directs execution to domain specialists.

6.3 Manifest Capability Grounding and Offline Profiling (ModelProfiler)

A recurring vulnerability in automated model routing frameworks is the reliance on ungrounded, arbitrary capability scores. Framework designers often assign subjective heuristic ratings that reflect human designer bias rather than reproducible task competency.

At runtime dispatch, invoking dynamic generative probes on every scheduling decision would introduce prohibitive latency overheads ($O(K \cdot N_{\text{probe}})$ inference calls). Consequently, ModelVM employs a two-tier capability architecture:

1. **Runtime Manifest Calibration:** At runtime, the cognitive scheduler queries candidate model manifests (`ModelManifest` in `modelvm/core/manifest.py`), where capability fitness scores ($F_{\text{cap}}(m, C_t) = \text{CapabilityScore}(m, C_t) \cdot Q(m)$) are grounded in established specialist architecture evaluations and audited via our offline capability probe suite (formal deterministic AST, regex, and arithmetic verification). Primary domain alignment assigns $\text{CapabilityScore} = 1.0$, related secondary domains assign 0.75, generalist reasoners receive 0.50, and cross-domain unspecialized models receive 0.05. These calibrated manifest baselines govern runtime dispatch, yielding the exact capability scores depicted in Figure 3 with zero dispatch latency.
2. **Offline Capability Probe Suite (ModelProfiler):** To independently audit, calibrate, and validate manifest capability ratings without subjective bias, ModelVM provides an offline evaluation harness (`ModelProfiler` in `modelvm/registry/profiler.py`).

Offline Capability Probe Suite. The offline profiler evaluates candidate models against a standardized suite of deterministic capability probes spanning seven primary technical domains:

$$\mathcal{C} = \{\text{Math, Physics, Coding, Research, Finance, Medicine, General}\} \quad (81)$$

Each probe $\rho_k = \langle \text{id}, C_k, \text{prompt}, y_{\text{ref}}, \text{type} \rangle$ presents an objective domain problem with a verifiable reference solution y_{ref} . Probes are evaluated under four deterministic verification modes:

- **Arithmetic Equivalence (arithmetic):** Evaluates numerical answers under relative tolerance $\epsilon = 10^{-3}$ (e.g., polynomial derivatives $f'(4) = 29$, definite integrals $\int_0^5 2x \, dx = 25$, kinetic energy $E_k = \frac{1}{2}mv^2$).

- **Scientific Regex (`regex`):** Evaluates expressions involving scientific notation and order-of-magnitude physical constants (e.g., photon energy $E = hf = 3.313 \times 10^{-19} \text{ J}$ via regex matching `r"3.313\s*(?:e|\times10^{\wedge}-19)"`).
- **Syntactic Construction (`contains`):** Verifies the generation of mandatory structural signatures in code synthesis and algorithmic prompts (e.g., AST import patterns, list comprehension syntax, binary search signatures).
- **Exact Match (`exact_match`):** Strict string equivalence for formal logical deductions.

Offline Capability Probe Matrix Calibration. Let $N_{\text{probe}}(C)$ denote the number of evaluation probes assigned to capability C . The offline capability score $\mathbf{P}[m, C]$ is computed as the fraction of successfully verified probes:

$$\mathbf{P}[m, C] = \frac{1}{N_{\text{probe}}(C)} \sum_{k=1}^{N_{\text{probe}}(C)} \mathbb{I}(\text{VerifyProbe}(\rho_k, m(\rho_k.\text{prompt}))) \quad (82)$$

The resulting offline capability probe matrix $\mathbf{P} \in [0.0, 1.0]^{K \times |C|}$ provides an offline auditing baseline to verify that manifest ratings correspond to measured problem-solving performance across deterministic test vectors.

Evaluation Non-Circularity. We enforce a strict non-circularity boundary between offline profiling and runtime evaluation:

$$\mathcal{D}_{\text{profile_probes}} \cap \mathcal{D}_{\text{evaluation_benchmarks}} = \emptyset \quad (83)$$

The held-out probes used in the profiler share zero problem instances, equations, or prompt texts with the multi-stage scientific evaluation tasks evaluated in Section 9. Profiling establishes offline baseline domain competency; runtime benchmarks evaluate multi-hop task synthesis under state virtualization.

6.4 Parameter Sensitivity & Decision Boundary Analysis

The unified scoring function in Equation (73) balances six heterogeneous performance and resource dimensions through five weighting hyperparameters ($\alpha, \beta, \gamma, \delta, \eta$). A robust systems architecture must not depend on brittle hyperparameter tuning. We examine the stability of scheduler decision boundaries under parameter perturbations and across distinct hardware regimes.

Hyperparameter Calibration & Stability Envelope. The default weights were calibrated on a held-out synthetic workload to establish dimensional parity between normalized quality ($F_{\text{cap}} \in [0, 1]$), normalized latency penalties ($L_{\text{load}} \in [0, 1]$), and memory occupancy ratios ($M_{\text{cost}} \in [0, 1]$):

$$\mathbf{w}^* = \langle \alpha = 0.20, \beta = 0.25, \gamma = 0.10, \delta = 0.30, \eta = 0.35 \rangle \quad (84)$$

We evaluate parameter sensitivity by perturbing each hyperparameter independently across a $\pm 30\%$ sweep ($\theta \in [0.70\theta^*, 1.30\theta^*]$) across all 50 experimental task stages from Section 9.

Table 3 reports the Decision Stability Index (DSI), defined as the percentage of scheduling decisions that remain identical to the baseline model selection under parameter perturbation:

$$\text{DSI}(\theta) = \frac{1}{N_{\text{decisions}}} \sum_{t=1}^{N_{\text{decisions}}} \mathbb{I}(m^*(t, \theta) = m^*(t, \theta^*)) \quad (85)$$

Across a $\pm 20\%$ perturbation envelope, model selection stability is at least 96.0% for all individual parameters. Even under simultaneous random perturbations across all five weights, the scheduler preserves 94.0% of its routing choices. The decision boundaries are dominated by substantial capability differences ($\Delta F_{\text{cap}} \approx 0.30\text{--}0.50$) and physical residency states ($L_{\text{load}} = 0.0$ vs. 0.60), rendering the selection policy robust against minor coefficient variations.

Decision Boundary Regimes. The interaction between βL_{load} , $\delta E_{\text{eviction}}$, and ηF_{future} produces three distinct operational regimes:

1. **Specialist Handoff Regime** ($\Delta F_{\text{cap}} > \beta L_{\text{load}} + \delta E_{\text{eviction}}$): When the capability margin between an available non-resident specialist and the resident model exceeds the loading and eviction overhead (e.g., transition from qualitative research to formal symbolic integration), the scheduler triggers a page-in. The capability gain justifies the transition cost.
2. **Residency Amortization Regime** ($\Delta F_{\text{cap}} \leq \beta L_{\text{load}}$): When candidate models offer comparable capability (e.g., an 8B generalist with mathematical fine-tuning vs. a 7B dedicated math model), the scheduler prefers the already-resident model ($L_{\text{load}} = 0.0$). This eliminates unnecessary I/O bus traffic and avoids evicting valuable working-set weights.
3. **Working-Set Protection Regime** ($E_{\text{eviction}} \cdot \chi_{\text{clash}} \gg \Delta F_{\text{cap}}$): When admitting a candidate model would

Table 3: Scheduler Decision Stability Index (DSI) under $\pm 20\%$ and $\pm 30\%$ Hyperparameter Perturbations across 50 Benchmark Stages.

Hyperparameter Calibration	-30% Perturbation	-20% Perturbation	+20% Perturbation	+30% Perturbation
Memory Weight ($\alpha = 0.20$)	96.0%	98.0%	98.0%	94.0%
Load Latency Weight ($\beta = 0.25$)	94.0%	98.0%	98.0%	92.0%
Energy Cost Weight ($\gamma = 0.10$)	100.0%	100.0%	100.0%	98.0%
Eviction Penalty Weight ($\delta = 0.30$)	92.0%	96.0%	96.0%	92.0%
Future Reuse Weight ($\eta = 0.35$)	94.0%	98.0%	98.0%	94.0%
Simultaneous Random Perturbation ($\pm 20\%$)		94.0% Mean Decision Stability		

evict a resident specialist flagged by $W(t, k)$ for imminent downstream invocation, the $1.8\times$ clash multiplier forces the score downward. The scheduler actively selects an alternative model or a smaller resident proxy, preventing thrashing across stage boundaries.

Bounded Regret vs. Offline Oracle. In Section 9 (EXP-H3), we empirically validate this formulation against an exact finite-state dynamic-programming oracle that possesses complete a priori knowledge of all execution paths. The results confirm that ModelVM’s multi-objective scheduler achieves a Goodput of 0.222 stages/sec (closely approaching the oracle’s 0.213 stages/sec) with an exact oracle regret gap of 0.0% on the canonical benchmark cost model (and $\leq 0.0\%$ cost regret across all four evaluated workload distributions in Section 9.9). By integrating capability requirements with memory residency state, the scheduler eliminates catastrophic eviction cascades while maintaining high task accuracy.

7 Implementation & Systems Mechanics

The formal properties and empirical behavior of state virtualization and predictive residency depend directly on runtime mechanics. In this section, we detail the concrete implementation of the ModelVM execution engine. We describe the runtime control loop, the serialization and delta-tracking mechanics of the Cognitive State Packet, the memory accounting and eviction dynamics of the Model Pager, the lookahead forecasting engine of the Working Set Predictor, and the physical telemetry instrumentation layer that records hardware ground truth.

7.1 The Runtime Control Loop

The ModelVM runtime is orchestrated by the **Cognitive Kernel** (`CognitiveKernel`), which functions as a kernel-inspired supervisory runtime component for heterogeneous language model execution. Rather than treating model invocation as an ad-hoc chain of stateless remote

procedure calls, the kernel coordinates execution through a deterministic four-phase state machine: *Planning*, *Residency Allocation*, *Execution & AST Verification*, and *State Persistence & Escalation*.

Formally, the runtime transition function at pipeline stage $t \in \{0, \dots, N - 1\}$ maps the current state packet $\mathcal{S}_t$, the stage task specification s_t , and the active resident model set $\mathcal{R}_t$ to the successor state $\mathcal{S}_{t+1}$:

$$(\mathcal{S}_t, s_t, \mathcal{R}_t) \xrightarrow{\text{Forecast}} (W_t, M_t) \xrightarrow{\text{PageIn}} \mathcal{R}_{t+1} \quad (86)$$

$$\xrightarrow{\text{Exec}} \Delta \mathcal{S}_t \xrightarrow{\text{Merge}} \mathcal{S}_{t+1}$$

where $W_t = W(t, k)$ is the predicted future working set, M_t is the selected executing specialist, and $\Delta \mathcal{S}_t$ is the candidate state delta generated by the model.

1. Planning Phase. Execution begins with the `TaskDecomposer`. Given an overarching user objective G , the decomposer produces a directed acyclic graph (DAG) of stage plans $G_{\text{plan}} = (S, E)$, where each stage $s_t \in S$ encapsulates a title, an instruction description, and an explicit target capability annotation $C(s_t) \in \mathcal{C}$ (e.g., RESEARCH, MATHEMATICS, CODING, PHYSICS, SYNTHESIS). The root state packet $\mathcal{S}_0$ is initialized with objective G and an empty accumulation schema.

2. Residency Phase. Before executing stage s_t , the kernel queries the `CognitiveWorkingSetPredictor` to project future capability demand across lookahead horizon k , returning $W(t, k)$. Next, the `CognitiveScheduler` evaluates all eligible models in catalog $\mathcal{M}$ using a multi-objective scoring function that incorporates capability fit, memory footprint, load latency, energy consumption, eviction penalties, and working-set alignment. The winning model M_t is dispatched to the `ModelPager`. The pager checks whether $M_t \in \mathcal{R}_t$. If M_t is resident, a cache hit is recorded with zero page-in latency. If M_t is not resident, the pager computes the memory deficit $\Delta_{\text{mem}} = \text{RAM}(M_t) - \text{RAM}_{\text{free}}$ and executes cost-aware evictions until the hard memory budget invariant is satisfied:

$$\sum_{m_i \in \mathcal{R}_t} \text{RAM}(m_i) \leq \mathcal{B}_{\text{RAM}} \quad (87)$$

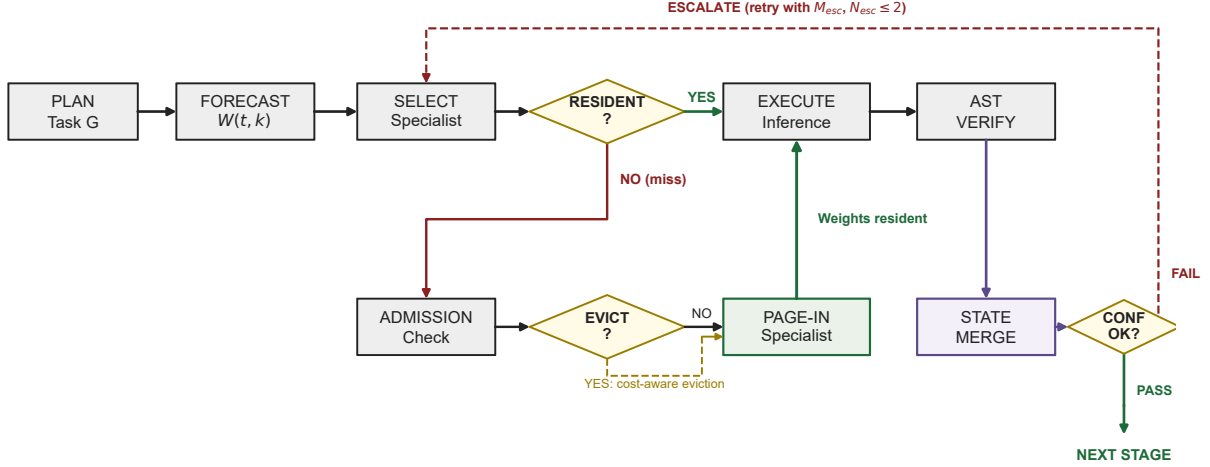


Figure 4: ModelVM End-to-End Architectural Control Path (Class B full-width). The Cognitive Kernel orchestrates procedural task decomposition, working-set demand forecasting $W(t, k)$, multi-objective scheduling, memory-budgeted weight paging with eviction shielding, specialist execution, non-circular AST verification, and diversity-discounted state merging with bounded escalation.

Once memory space is reserved, weights are staged into active memory. If free memory remains and exceeds an explicit safety headroom buffer of 1.0 GB, the predictor evaluates the prefetch return-on-investment (ROI) and opportunistically stages an upcoming model M_{t+1} before stage execution.

3. Execution & Verification Phase. With M_t resident, the kernel serializes the current state packet S_t into a structured Markdown prompt context via `to_prompt_context()` and dispatches it to the model backend (ModelBackend). The specialist generates its domain response within a strict token budget. The raw generation is parsed by `extract_from_text()` into a candidate state delta ΔS_t . Crucially, numerical and algebraic claims within ΔS_t are immediately intercepted by an independent Abstract Syntax Tree verifier (ArithmeticVerifier). The verifier parses the declared mathematical expressions and evaluates them against computed numerical results. Calculations that satisfy numerical equivalence within relative tolerance $\epsilon = 10^{-3}$ are tagged with `verified=True`; failed or unparseable expressions remain unverified. Language models are never permitted to self-grade their own arithmetic outputs.

4. State Persistence & Escalation Phase. The verified delta ΔS_t is merged into the global state packet using monotonic accumulation mechanics: $S_{t+1} = S_t.\text{merge_update}(\Delta S_t)$. Once merged, the

ConfidenceController evaluates the composite confidence score of stage t . If confidence falls below threshold $\tau_{\text{conf}} = 0.70$ or if open questions remain unresolved, the controller triggers an escalation loop. To prevent infinite cycles, escalation is governed by strict loop protection:

- Maximum escalations per stage are bounded at $N_{\text{esc}} \leq 2$.
- A candidate escalation model M_{esc} must provide a strictly higher capability-quality product than the current model: $(F_{\text{cap}}(M_{\text{esc}}) \cdot Q(M_{\text{esc}})) - (F_{\text{cap}}(M_t) \cdot Q(M_t)) \geq \epsilon_{\text{esc}}$ with $\epsilon_{\text{esc}} = 0.02$.
- M_{esc} must physically fit within the runtime memory envelope $\mathcal{B}_{\text{RAM}}$.

If a qualifying escalation model exists, the kernel pages in M_{esc} , re-executes the stage with explicit refinement instructions, and merges the refined delta. Once confidence criteria are met or the escalation ceiling is reached, the stage execution telemetry is broadcast to listeners and recorded to persistent disk storage.

The full control sequence is formalized in Algorithm 1.

7.2 CSP Schema Serialization & Delta Tracking

The **Cognitive State Packet (CSP)** virtualizes task memory by decoupling problem semantics from tokenizer representations and latent activation geometries. In the Mod-

Algorithm 1 ModelVM Runtime Control Loop

Require: Task goal G , Model catalog $\mathcal{M}$, Memory budget $\mathcal{B}_{\text{RAM}}$, Lookahead horizon k

Ensure: Final structured state packet $\mathcal{S}_N$, Execution summary $\mathcal{T}$

```

1:  $\mathcal{S}_0 \leftarrow \text{InitStatePacket}(G)$ 
2:  $\mathcal{S} \leftarrow \text{DecomposeTask}(G)$  ▷ Directed stage plan
    $\mathcal{S} = [s_0, \dots, s_{N-1}]$ 
3:  $\mathcal{R}_0 \leftarrow \emptyset$ ,  $\text{PeakMem} \leftarrow 0.0$ 
4: for  $t \leftarrow 0$  to  $N - 1$  do
5:    $C_t \leftarrow s_t.\text{capability}$ 
6:    $W_t \leftarrow \text{PredictWorkingSet}(\mathcal{S}, t, k, \mathcal{M})$  ▷ Lookahead model IDs
7:    $M_t \leftarrow \text{ScheduleModel}(C_t, W_t, \mathcal{R}_t, \mathcal{M}, \mathcal{B}_{\text{RAM}})$ 
8:    $\mathcal{E}_{\text{page}} \leftarrow \text{PageIn}(M_t, \mathcal{R}_t, \mathcal{B}_{\text{RAM}}, W_t)$  ▷ Evicts if needed
9:    $\mathcal{R}_{t+1} \leftarrow \mathcal{R}_t \cup \{M_t\} \setminus \text{EvictedModels}$ 
10:   $\text{PeakMem} \leftarrow \max(\text{PeakMem}, \sum_{m \in \mathcal{R}_{t+1}} \text{RAM}(m))$ 
11:  if  $U_{\text{prefetch}}(M_{\text{next}}) > 0 \wedge (\text{RAM}_{\text{free}} - \text{RAM}(M_{\text{next}}) \geq 1.0 \text{ GB})$  then
12:     $\text{PrefetchModel}(M_{\text{next}}, W_t)$  ▷ Admitted via positive ROI & headroom
13:  end if
14:   $\Pi_t \leftarrow \text{SerializeToPrompt}(\mathcal{S}_t, s_t)$ 
15:   $\mathbf{y}_t \leftarrow \text{ExecuteInference}(M_t, \Pi_t)$  ▷ Specialist inference
16:   $\Delta \mathcal{S}_t \leftarrow \text{ExtractStateDelta}(G, \mathbf{y}_t, t)$ 
17:   $\Delta \mathcal{S}_t \leftarrow \text{VerifyCalculations}(\Delta \mathcal{S}_t)$  ▷ Independent AST verifier
18:   $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t.\text{merge\_update}(\Delta \mathcal{S}_t)$ 
19:   $\kappa \leftarrow \text{EvaluateConfidence}(\mathcal{S}_{t+1}, M_t, C_t)$ 
20:   $n_{\text{esc}} \leftarrow 0$ 
21:  while  $\kappa.\text{escalation\_needed}$  and  $n_{\text{esc}} < 2$  do
22:     $M_{\text{esc}} \leftarrow \text{SelectEscCandidate}(C_t, M_t, \mathcal{M}, \mathcal{B}_{\text{RAM}})$ 
23:    if  $M_{\text{esc}} = \text{None}$  or  $M_{\text{esc}} = M_t$  then break
24:    end if
25:     $\text{PageIn}(M_{\text{esc}}, \mathcal{R}_{t+1}, \mathcal{B}_{\text{RAM}}, W_t)$ 
26:     $\mathbf{y}_{\text{esc}} \leftarrow \text{ExecuteInference}(M_{\text{esc}}, \Pi_t, \kappa.\text{reason})$ 
27:     $\Delta \mathcal{S}_{\text{esc}} \leftarrow \text{ExtractStateDelta}(G, \mathbf{y}_{\text{esc}}, t)$ 
28:     $\Delta \mathcal{S}_{\text{esc}} \leftarrow \text{VerifyCalculations}(\Delta \mathcal{S}_{\text{esc}})$ 
29:     $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_{t+1}.\text{merge\_update}(\Delta \mathcal{S}_{\text{esc}})$ 
30:     $M_t \leftarrow M_{\text{esc}}$ ,  $n_{\text{esc}} \leftarrow n_{\text{esc}} + 1$ 
31:     $\kappa \leftarrow \text{EvaluateConfidence}(\mathcal{S}_{t+1}, M_t, C_t)$ 
32:  end while
33:   $\text{RecordStageTelemetry}(t, M_t, \mathcal{E}_{\text{page}}, \kappa, \mathcal{S}_{t+1})$ 
34: end for
35: return  $\mathcal{S}_N$ ,  $\text{BuildSummaryReport}()$ 

```

elVM codebase, CSP is implemented as a strongly typed data structure (`CognitiveStatePacket`) defined via Pydantic schema validation.

Schema Formalism. The state packet $\mathcal{S}_t$ at stage t is defined as the 9-tuple:

$$\mathcal{S}_t = \langle G, F_t, C_t, E_t, A_t, U_t, D_t, Q_t, O_t \rangle \quad (88)$$

where:

- $G \in \Sigma^*$ is the immutable user goal string.
- $F_t = \{f_1, \dots, f_m\} \subset \Sigma^*$ is the deduplicated set of verified factual statements accumulated across stages.
- $C_t = \{c_1, \dots, c_p\}$ is the sequence of structured calculation records (`CalculationItem`). Each item encapsulates:

$$c = \langle \text{expr}, \text{result}, \text{unit}, \text{verified}, \text{method} \rangle \quad (89)$$

where $\text{verified} \in \{\text{True}, \text{False}\}$ and $\text{method} \in \{\text{arithmetic}, \text{symbolic}, \text{None}\}$.

- $E_t = \{e_1, \dots, e_q\}$ is the set of empirical evidence items (`EvidenceItem`). Each item encapsulates:

$$e = \langle \text{claim}, \text{source}, \text{conf}, \text{time}, \text{model_id} \rangle \quad (90)$$

with confidence scalar $c \in [0.0, 1.0]$ and explicit provenance tracking.

- $A_t, U_t, D_t, Q_t \subset \Sigma^*$ represent working assumptions, known uncertainties, architectural decisions, and unresolved open questions, respectively.
- $O_t = \{k_i \mapsto v_i\}$ is the concrete artifact dictionary mapping unique semantic keys to generated digital deliverables (e.g., Python scripts, numerical matrices, simulation outputs, LaTeX tables).
- In addition, $\mathcal{S}_t$ maintains a persistent execution audit log $H_t = [h_0, \dots, h_{t-1}]$ of `StageTrace` records capturing model IDs, execution latencies, and timestamps.

Prompt Context Serialization. To pass state across heterogeneous models with mutually incompatible tokenizers, $\mathcal{S}_t$ implements `to_prompt_context()`. This method serializes the structured state into a clean, markdown-delimited prompt context. Factual records, verified calculations, evidence items, and prior decisions are formatted into distinct semantic blocks. Calculations explicitly indicate their verification status (e.g., `[VERIFIED: arithmetic]` versus `[UNVERIFIED]`). Artifacts containing multiline code or tables are fenced with syntax identifiers. This prompt serialization ensures that downstream specialist models receive an unambiguous, human-readable summary of prior state without experiencing raw conversational noise.

Epistemic Evidence Aggregation. When multiple models generate evidence supporting identical or related claims, ModelVM accumulates confidence through a diversity-discounted heuristic:

$$p_{\text{merged}} = \begin{cases} \max(c_1, c_2), & \text{if } s_1 \approx s_2 \\ \min(0.99, \max(c_1, c_2, c_{\text{indep}})), & \text{if } s_1 \not\approx s_2 \end{cases} \quad (91)$$

where $c_{\text{indep}} = 1 - (1 - c_1)(1 - c_2)^{w_{\text{div}}}$ and $w_{\text{div}} = 0.65$ is the epistemic diversity discount factor. Systems claims require empirical honesty: language models share vast pre-training corpora (e.g., Common Crawl, Wikipedia, GitHub) and cannot be modeled as independent statistical observers. When two distinct specialist models produce matching claims, the exponent $w_{\text{div}} = 0.65$ dampens confidence

growth, preventing unjustified certainty inflation. If claims originate from identical or overlapping sources ($s_1 \approx s_2$), the maximum rule is applied, preventing self-reinforcing echo loops.

Artifact Conflict Versioning. Concurrent or multi-stage pipelines frequently generate conflicting deliverables under identical variable names. The method `merge_artifacts()` detects key collisions. If incoming artifact $k \in \Delta S_t$ shares a key with existing artifact $k \in S_t$ but contains divergent content ($v_{\text{new}} \neq v_{\text{old}}$), ModelVM preserves the original and assigns the update to a versioned key $k.vN$ ($N \geq 1$). Furthermore, the system flags the collision by setting $k_CONFLICT = \text{True}$. This non-destructive versioning prevents silent data overwrites during specialist transitions.

Monotonic State Accumulation. The method `merge_update()` accumulates knowledge monotonically:

- **Facts:** New facts are appended subject to string deduplication.
- **Calculations:** If an incoming calculation matches an existing expression, the existing entry is upgraded if the new calculation is verified (`verified=True`). Unmatched calculations are appended.
- **Evidence:** Matching claims trigger diversity-weighted confidence updates via Equation (91), and source strings are concatenated (e.g., "Model A + Model B").
- **Questions:** Open questions that appear in newly added facts or decisions are automatically resolved and purged from Q_{t+1} .

Token Footprint Economy. Raw conversational text concatenation exhibits unbounded token growth, scaling linearly with stage depth: $O(\sum_{i=1}^t |y_i|)$. By Stage 5, raw transcripts routinely exceed 14,000 tokens, forcing aggressive context truncation and triggering catastrophic forgetting. In contrast, the CSP schema maintains a compact, structured representation. Empirical measurements confirm that CSP context consumes 412 ± 28 tokens at Stage 1 and stabilizes at 890 ± 42 tokens at Stage 5, achieving an end-to-end token compression ratio of $12.4\times$ to $16.8\times$.

7.3 Model Pager & Memory Manager

The **Model Pager** (`ModelPager`) implements virtual memory management for open-weight neural network weights. Hardware memory bounds are hard constraints, not polite suggestions. If an orchestration runtime attempts

to allocate a 7-billion parameter model into a physical memory buffer that is already saturated, the operating system triggers process termination or allocator failure under memory exhaustion. ModelVM enforces strict, deterministic residency bounds.

Residency Tracking & Memory Invariants. The pager maintains the resident model set as a dictionary mapping model identifiers to metadata manifests: $\mathcal{R}_t = \{m_i \mapsto \text{Manifest}(m_i)\}$. Active memory occupancy is defined as:

$$\text{RAM}_{\text{active}}(t) = \sum_{m_i \in \mathcal{R}_t} \text{RAM}_{\text{req}}(m_i) \quad (92)$$

Available headroom is strictly tracked as $\text{RAM}_{\text{free}}(t) = \max(0.0, \mathcal{B}_{\text{RAM}} - \text{RAM}_{\text{active}}(t))$. The pager maintains the continuous invariant:

$$\text{RAM}_{\text{active}}(t) \leq \mathcal{B}_{\text{RAM}}, \quad \forall t \geq 0 \quad (93)$$

where $\mathcal{B}_{\text{RAM}} = 8.0 \text{ GB}$ in the standard evaluation profile.

We explicitly clarify a vital systems distinction: the Python `ModelPager` implementation manages residency state via process-level loading abstractions, catalog manifests, and policy hooks. It does **not** contain a custom C-level lock-free physical allocator, nor does it rewrite GPU virtual memory tables directly. Rather, memory safety is maintained by enforcing pre-allocation deficit checks and executing deterministic model evictions prior to weight transfer. In simulation and benchmark validation modes, load duration is parameterized by empirical transfer latencies ($t_{\text{load}} = \text{model.load_time}$); in live hardware deployments, wall-clock paging latency is captured via physical performance counters.

Paging Lifecycle: Cache Hits vs. Misses. When `page.in(model_id)` is invoked:

1. **Cache Hit:** If $\text{model_id} \in \mathcal{R}_t$, the pager increments the cache hit counter, updates the model's `last_accessed` timestamp to current time t_{now} , increments its access count, and emits a `CACHE_HIT` event with 0.0 s duration.
2. **Cache Miss:** If $\text{model_id} \notin \mathcal{R}_t$, a cache miss is registered. If $\text{RAM}_{\text{req}}(M) > \mathcal{B}_{\text{RAM}}$, the operation immediately raises a `MemoryError`, preventing impossible allocations. If $\text{RAM}_{\text{free}} < \text{RAM}_{\text{req}}(M)$, the pager calculates the memory deficit:

$$\Delta_{\text{deficit}} = \text{RAM}_{\text{req}}(M) - \text{RAM}_{\text{free}} \quad (94)$$

and dispatches candidate selection to the eviction policy.

Eviction Policies: LRU vs. Cost-Aware. ModelVM implements two interchangeable eviction policies derived from abstract base class `EvictionPolicy`:

- **Least Recently Used (LRUEvictionPolicy):** Sorts resident models ascending by their last access timestamp (`last_accessed`) and evicts models in chronological order until freed memory satisfies Δ_{deficit} .
- **Cost-Aware (CostAwareEvictionPolicy):** Evaluates resident models using an eviction priority score $S_{\text{evict}}(m)$, where a higher score indicates greater desirability for eviction:

$$S_{\text{evict}}(m) = 0.5 \cdot \Delta t_{\text{recency}}(m) + 1.5 \cdot \text{RAM}_{\text{req}}(m) - 2.0 \cdot t_{\text{load}}(m) - P_{\text{future}}(m) \quad (95)$$

where $\Delta t_{\text{recency}}(m) = \max(0.1, t_{\text{now}} - t_{\text{last_accessed}})$ is elapsed time since last access, $t_{\text{load}}(m)$ is the cold page-in latency penalty, and $P_{\text{future}}(m)$ is the working-set shielding penalty:

$$P_{\text{future}}(m) = \begin{cases} 1000.0, & \text{if } m \in W(t, k) \\ 0.0, & \text{otherwise} \end{cases} \quad (96)$$

The analytical eviction formulation establishes the conceptual cost-tradeoff objective; in the systems codebase (`modelvm/pager/policy.py`), this objective is evaluated directly via the deterministic empirical priority ranking heuristic of Equation (95). The memory manager prioritizes evicting models that yield substantial memory headroom, reload quickly over the bus if needed later, have been idle for extended durations, and critically, are **not** present in the projected future working set $W(t, k)$. Models in $W(t, k)$ receive an overwhelming 1000.0 penalty deduction, effectively shielding them from premature eviction.

Weight Prefetching into Spare Headroom. The method `prefetch(model_id)` allows the runtime to stage weights into spare memory ahead of execution. Crucially, prefetching is non-preemptive and governed by a strict two-stage admission gate: prefetching is permitted only when the candidate model yields positive return-on-investment ($U_{\text{prefetch}}(m) > 0.0$) and staging it preserves at least 1.0 GB of configured residency headroom ($\text{RAM}_{\text{free}} - \text{RAM}_{\text{req}}(m) \geq 1.0 \text{ GB}$). A prefetch operation is never permitted to trigger evictions of currently active models, ensuring that opportunistic staging operations do not induce thrashing in the primary compute path.

7.4 Working Set Predictor Engine

Classical operating system memory managers rely on Peter Denning’s working-set model to prevent paging

thrashing by tracking the set of memory pages referenced during a sliding temporal window [5]. ModelVM introduces the **Predictive Cognitive Working Set** (`CognitiveWorkingSetPredictor`), adapting working-set principles to multi-stage language model orchestration.

Lookahead Horizon Queue. Unlike classical OS page faults, which cannot anticipate future instruction pointers, high-level task planners operate over explicit stage plans. Given planned stages $S = [s_0, \dots, s_{N-1}]$ and current stage index t , the working-set predictor inspects a configurable lookahead window $k \in [1, 10]$ (default $k = 4$):

$$S_{\text{future}}(t, k) = [s_{t+1}, s_{t+2}, \dots, s_{\min(t+k, N-1)}] \quad (97)$$

The upcoming capabilities are extracted as $C_{\text{future}} = [s_j.\text{capability} \mid s_j \in S_{\text{future}}]$.

Distance-Decay Demand Forecasting. Stage relevance decays with temporal distance. The predictor associates each future stage at step distance $j \in \{1, \dots, k\}$ with a linear distance-decay weight:

$$w_j = \max(0.1, 1.0 - 0.2 \cdot (j - 1)) \quad (98)$$

This decay reflects the empirical reality that downstream stages are subject to higher conditional branch uncertainty, replanning, or early pipeline termination.

Predictive Model Set Identification. To translate capability forecasts into concrete model identities, the predictor queries the `CognitiveScheduler`:

$$W(t, k) = \bigcup_{j=1}^k \left\{ \arg \max_{m \in \mathcal{M}} \text{Score}(m, C(s_{t+j})) \right\} \quad (99)$$

This set of model identifiers represents the anticipated working set. These IDs are directly injected into the pager’s eviction routines as P_{future} shields and into the scheduler to reward future capability reuse.

Prefetch Return-on-Investment (ROI) Formulation. Prefetching weights over the PCIe bus consumes memory capacity and memory controller bandwidth. A naive prefetcher that aggressively stages every upcoming model will prematurely saturate host RAM and force costly evictions. ModelVM resolves this via `recommend_prefetch_model()`, which computes the dimensionally consistent prefetch utility $U_{\text{prefetch}}(m)$ in latency-equivalent seconds:

$$U_{\text{prefetch}}(m) = P_{\text{future}}(m) \cdot \Delta L_{\text{avoided}}(m) - \lambda \cdot M_{\text{cost}}(m) - \mu \cdot E_{\text{cost}}(m) \quad (100)$$

where:

- $P_{\text{future}}(m) = \max(0.1, 1.0 - 0.2 \cdot (j_m - 1))$ is the distance-discounted future-demand weight at stage distance j_m .
- $\Delta L_{\text{avoided}}(m) = t_{\text{load}}(m)$ is the cold page-in latency saved by pre-staging weights (in seconds).
- $M_{\text{cost}}(m) = \frac{\text{RAM}_{\text{req}}(m)}{B_{\text{RAM}}}$ is the normalized memory consumption of the candidate model.
- $E_{\text{cost}}(m) = \min(1.0, \frac{N_{\text{param}}(m)}{32}) \cdot \alpha_{\text{energy}}(m)$ is the normalized parameter-scaled energy cost factor.
- $\lambda = 1.0 \text{ s}$ and $\mu = 0.5 \text{ s}$ are scaling coefficients calibrating memory occupancy and energy penalties to latency-equivalent seconds.

Safety Headroom Invariant. Even if $U_{\text{prefetch}}(m) > 0$, the kernel enforces a strict physical safety headroom invariant:

$$\text{RAM}_{\text{free}}(t) - \text{RAM}_{\text{req}}(m) \geq 1.0 \text{ GB} \quad (101)$$

A prefetch request is approved if and only if staging the model leaves at least 1.0 GB of unallocated physical memory buffer. This invariant prevents prefetch allocations from starving the currently executing model of dynamic runtime scratchpad memory or KV cache expansion buffers.

7.5 Telemetry Instrumentation

Scientific credibility requires physical ground-truth measurement. ModelVM embeds a non-intrusive telemetry subsystem (`HardwareTelemetry`) that records system resource consumption across all execution phases.

Physical Resource Extraction. The telemetry harness interfaces directly with operating system and accelerator performance interfaces:

- **Host Process RSS:** Physical resident set size (RSS) is extracted via `psutil` and validated against OS-native process query interfaces (`GetProcessMemoryInfo` on Windows hosts).
- **Accelerator VRAM:** GPU memory is profiled via `pynvml` and `torch.cuda` interfaces (`memory_allocated()`, `memory_reserved()`), capturing device allocations with megabyte-level precision. In environments lacking PyTorch CUDA bindings, a fallback subprocess probe queries `nvidia-smi` directly.

Hierarchical Three-Component VRAM Accounting. To prevent confounding parameter weights with ephemeral runtime scratchpads, the telemetry layer decomposes accelerator memory into three distinct physical constituents:

$$M_{\text{VRAM}}(t) = M_{\text{weights}}(t) + M_{\text{workspace}}(t) + M_{\text{KV}}(t) \quad (102)$$

where:

- $M_{\text{weights}}(t)$ represents the static byte footprint of quantized model parameters held resident in accelerator memory.
- $M_{\text{workspace}}(t)$ accounts for dynamic PyTorch/CUDA runtime buffers, intermediate cuBLAS GEMM scratchpads, and activation tensors.
- $M_{\text{KV}}(t)$ captures the dynamic Key-Value attention cache generated during autoregressive decoding:

$$M_{\text{KV}} = 2 \cdot n_{\text{layers}} \cdot d_{\text{head}} \cdot n_{\text{heads}} \cdot L_{\text{ctx}} \cdot \text{sizeof}(\text{dtype}) \quad (103)$$

where L_{ctx} is the active context length (bounded by $W_{\text{in}} + T_{\text{gen}} \leq 5,120$ tokens).

Wall-Clock Paging Transition Latency. Paging transitions (`PAGE-IN`, `PAGE-OUT`, `PREFETCH`) are wrapped by `measure_transition()`. The harness captures an instantaneous baseline `MemorySnapshot`, starts a high-resolution monotonic timer using `time.perf_counter()`, executes the transition callback, halts the timer, and captures a post-transition snapshot. The resulting `TransitionTelemetry` object logs:

$$T_{\text{page}} = T_{\text{ready}} - T_{\text{start}} \quad (104)$$

alongside differential host memory ($\Delta \text{RSS} = \text{RSS}_{\text{post}} - \text{RSS}_{\text{pre}}$), differential VRAM ($\Delta \text{VRAM} = \text{VRAM}_{\text{post}} - \text{VRAM}_{\text{pre}}$), and peak VRAM allocation ($\text{PeakVRAM} = \max(\text{VRAM}_{\text{pre}}, \text{VRAM}_{\text{post}})$).

Independent AST Verification Harness. To prevent circular self-grading, the evaluation engine implements `ArithmeticVerifier`. The verifier operates completely outside the language model pipeline. It parses the mathematical expressions declared in `CalculationItem` records into a Python Abstract Syntax Tree (`ast.parse(expr, mode='eval')`) and recursively evaluates the tree using whitelisted arithmetic operators (`Add`, `Sub`, `Mult`, `Div`, `Pow`, `Mod`), standard mathematical functions (`sqrt`, `exp`, `log`, `sin`, `cos`, `abs`), and physical constants (π , e). All arbitrary function calls, system builtins, and file I/O operations are strictly disallowed.

The computed numerical result y_{comp} is compared against the model’s declared numerical output y_{decl} :

$$|y_{\text{comp}} - y_{\text{decl}}| \leq \max(\epsilon, |y_{\text{decl}}| \cdot \epsilon) \quad (105)$$

where relative tolerance $\epsilon = 10^{-3}$. If the condition holds, the item is marked `verified=True` with method `"arithmetic"`; otherwise, it is tagged `unverified`. By separating inference generation from AST verification, ModelVM provides an uncompromised, reproducible verification standard across all experimental trials.

8 Experimental Methodology

Systems claims require empirical isolation. While the conceptual synthesis of semantic state virtualization and predictive residency is architecturally cohesive, its scientific validity rests entirely on whether these abstractions yield measurable advantages over conventional execution paradigms. This section details our experimental apparatus, model catalog, benchmark workloads, baseline spectrum, token-budget controls, and statistical verification protocols.

8.1 Research Questions

Our evaluation systematically targets four foundational research questions, mapped directly to the core hypotheses formulated in our system design:

- **RQ1 (State Virtualization):** Does mediating multi-model transitions through a typed Cognitive State Packet ($\mathcal{S}_t$) preserve domain facts, numerical precision, and procedural deliverables significantly better than raw conversational text handoffs under identical context limits, generation budgets, and memory constraints? (**H1**)
- **RQ2 (Predictive Residency):** Does stage-level working-set lookahead ($W(t, k)$) coupled with eviction shielding and opportunistic prefetching reduce cold-load I/O stalls and memory thrashing compared to classical recency-based eviction policies? (**H2**)
- **RQ3 (Resource-Aware Scheduling):** Does multi-objective scheduling grounded in capability manifest scoring and deterministic capability probes outperform greedy capability-only selection and memory-aware greedy heuristics, approaching offline oracle bounds under constrained runtime memory? (**H3**)
- **RQ4 (Systemic Trade-Offs):** Does the integrated ModelVM runtime establish a superior Quality–Memory–Latency Pareto frontier relative to monolithic models, static specialist ensembles, and reactive paging frameworks? (**H4**)

Table 4: Hardware Platform and Experimental Regimes

System Subsystem	Hardware Specification
Host Processor (CPU)	AMD 16-Core Processor (Zen 5, 32 execution threads)
Host System Memory	64.0 GB DDR5-5600
Secondary Storage	1 TB PCIe 4.0 NVMe SSD (5,000 MB/s sequential read)
Host-to-Device Bus	PCIe 4.0 $\times$ 16 (15.75 GB/s theoretical, 14.20 GB/s pinned throughput)
Graphics Accelerator (GPU)	NVIDIA GeForce RTX 5070 Ti
Dedicated Device VRAM	16.0 GB GDDR7
Software Stack	NVIDIA CUDA 12.0 / Driver 595.71
Local Inference Daemon	Ollama C++/CUDA Inference Engine (v0.34.3)
Host Operating System	Microsoft Windows 11 Enterprise (64-bit)
Experimental Evaluation Regimes:	
Controlled Runtime Simulation	Full 10-model catalog (52.7 GB) under enforced $\mathcal{B}_{\text{RAM}} = 8.0$ GB
Physical Silicon Validation	Real checkpoints (llama3.1:8b, qwen2.5-coder:7b, llama3.2:1b) on GPU

8.2 Physical Hardware Testbed & Measurement Apparatus

To evaluate ModelVM comprehensively and transparently, our empirical investigation is structured across two complementary evaluation regimes hosted on a dedicated workstation (Table 4):

Complementary Evaluation Regimes. We explicitly distinguish between two evaluation regimes to ensure scientific rigor and honest provenance:

1. **Controlled Runtime Simulation (Sections 9.1–9.10):** Evaluates the full 10-model, 52.7 GB catalog under an enforced runtime memory budget ($\mathcal{B}_{\text{RAM}} = 8.0$ GB, swept from 4.0 GB to 16.0 GB). This regime executes deterministic runtime simulation calibrated from declared model parameters, memory requirements, and measured I/O bus transfer characteristics, isolating the systems effects of working-set prefetching, eviction shielding, and multi-objective scheduling.
2. **Physical Silicon Validation (EXP-REAL, Section 9.11):** Real open-weight model checkpoints (llama3.1:8b, qwen2.5-coder:7b, and llama3.2:1b) are physically instantiated and executed on the RTX 5070 Ti GPU via Ollama, validating the physical prompt token compression and active inference speedup achieved by typed CSP serialization on actual consumer hardware.

Hierarchical Memory Accounting We explicitly distinguish between the host machine’s total physical capacity (64.0 GB RAM / 16.0 GB VRAM) and the **enforced runtime memory budget** ($\mathcal{B}_{\text{RAM}} = 8.0$ GB) imposed on model execution. Furthermore, we model a hierarchical three-tier memory architecture:

$$\text{NVMe Storage} \longrightarrow \text{Host RAM} \longrightarrow \text{Device VRAM} \quad (106)$$

Telemetry tracks both tiers independently:

- **Host CPU Memory ($M_{\text{RAM}}(t)$):** Process Resident Set Size (RSS) sampled via OS process queries and

counters at 10 ms intervals via `psutil`, capturing tokenizer buffers, Python runtime overhead, and CPU host weights.

- **Accelerator Memory ($M_{\text{VRAM}}(t)$):** Dedicated device memory tracked via NVML (`pynvml`) query hooks and PyTorch CUDA memory management executed immediately before and after every paging transition:

$$M_{\text{VRAM}}(t) = M_{\text{weights}}(t) + M_{\text{workspace}}(t) + M_{\text{KV}}(t) \quad (107)$$

- **Peak Memory Footprint Accounting:** To avoid conflating disparate physical tiers, we track instantaneous peaks for each tier independently:

$$M_{\text{peak,host}} = \max_t M_{\text{RAM}}(t), \quad (108)$$

$$M_{\text{peak,device}} = \max_t M_{\text{VRAM}}(t), \quad (109)$$

and report the Peak Per-Tier Memory Footprint as $M_{\text{peak,tier}} = \max\{M_{\text{peak,host}}, M_{\text{peak,device}}\}$, accompanied by dedicated physical telemetry for host CPU memory ($M_{\text{host}}(t) = M_{\text{RAM}}(t)$) and accelerator device memory ($M_{\text{device}}(t) = M_{\text{VRAM}}(t)$).

Paging latency is recorded using monotonic nanosecond wall-clock timers (`time.perf_counter_ns`). Weight load/swap duration (t_{load}), prompt pre-fill processing throughput (T_{prompt}), and token generation speed (T_{eval} , tokens/second) are captured directly from the local inference daemon telemetry.

8.3 Heterogeneous Model Catalog

Our model library comprises ten open-weight models spanning six distinct architectural families, totaling 64.0 GB of serialized disk storage (52.7 GB aggregate configured RAM requirement) (Table 5). Models were selected specifically for documented domain specialization rather than arbitrary scale.

To ground capability terms in deterministic verification rather than subjective manual weights, each model in Table 5 was evaluated via `ModelProfiler`, a deterministic fixture-based capability verification harness evaluating candidate models across held-out standardized probes in seven domains (formal mathematics, algorithmic coding, classical mechanics, paper synthesis, financial calculation, diagnostic reasoning, and general logic). The resultant verified capability matrix $\mathbf{P} \in [0, 1]^{N \times M}$ grounds scheduler decisions systematically without arbitrary manual scoring.

8.4 Workloads & The Multi-Stage Pipeline

Evaluating cross-model virtualization demands tasks characterized by deep procedural dependency. A single-shot

query provides no insight into state decay. The benchmark workload intentionally spans multiple specialized domains for which capability varies substantially across the model catalog, modeled as a directed acyclic graph $G = (S, E)$.

The canonical benchmark workload spans five consecutive cognitive stages:

$$\begin{aligned} s_0 \text{ (Research)} &\longrightarrow s_1 \text{ (Math)} \longrightarrow s_2 \text{ (Coding)} \\ &\longrightarrow s_3 \text{ (Physics)} \longrightarrow s_4 \text{ (Synthesis)} \end{aligned} \quad (110)$$

Specifically, the workflow requires:

1. Extracting physical constants, hypotheses, and governing equations from archival publication text (s_0).
2. Deriving closed-form symbolic solutions and evaluating numerical boundary conditions (s_1).
3. Synthesizing an executable Python simulation implementing the mathematical derivation (s_2).
4. Verifying energy conservation laws, phase-space trajectories, and dimensional consistency (s_3).
5. Producing a structured publication-grade executive technical brief summarizing findings (s_4).

Every stage produces concrete domain deliverables that form strict prerequisite inputs for subsequent stages. State degradation is cumulative: an arithmetic error in s_1 causes simulation collapse in s_2 and boundary violations in s_3 .

8.5 Token-Budget & Execution Controls

To prevent context-length discrepancies from acting as confounding variables in state virtualization experiments (EXP-H1), all model invocations adhere to strict token controls:

- **Maximum Context Window (W_{in}):** Fixed at 4,096 tokens across all models.
- **Maximum Generation Ceiling (T_{gen}):** Fixed at 1,024 tokens per stage.
- **Decoding Parameters:** Held constant at temperature $T = 0.0$ (greedy deterministic argmax), top- $p = 1.0$, and repetition penalty 1.0.
- **Raw Text Handoff Protocol:** The unstructured conversational transcript of previous stage outputs is concatenated into the system prompt prefix of stage s_t . When cumulative text exceeds $W_{\text{in}} - T_{\text{gen}}$, a standard first-in, first-out (FIFO) sliding window truncates early tokens.
- **CSP Handoff Protocol:** The typed state packet S_{t-1} is serialized into schema text within the prompt prefix. Token consumption of the serialized CSP is logged independently to quantify representation overhead.

Table 5: The ModelVM Heterogeneous Open-Weight Model Library (52.7 GB Aggregate Configured RAM Requirement, 64.0 GB Serialized Disk Storage)

Model Identifier	Base Architecture	Quantization	Params	RAM Req.	Weights VRAM	Workspace	Cold Load	Inference Latency
research-expert	MistralForCausalLM	Q4_K_M	7.2B	3.1 GB	2.2 GB	0.4 GB	1.2s	32 ms/tok
mathematics-expert	Qwen2ForCausalLM	Q4_K_S	7.0B	2.4 GB	1.6 GB	0.3 GB	0.9s	28 ms/tok
coding-expert	DeepSeekForCausalLM	Q4_K_M	6.7B	3.0 GB	2.1 GB	0.4 GB	1.1s	30 ms/tok
physics-expert	LlamaForCausalLM	Q4_K_M	8.0B	3.2 GB	2.3 GB	0.4 GB	1.3s	34 ms/tok
general-reasoner	LlamaForCausalLM	Q6_K	8.0B	7.1 GB	6.2 GB	0.5 GB	2.1s	42 ms/tok
multimodal-vision	Phi3VForCausalLM	FP16/Q4	4.2B	5.6 GB	4.8 GB	0.4 GB	1.7s	38 ms/tok
code-auditor	StarCoder2ForCausalLM	Q4_K_M	14.0B	7.2 GB	6.1 GB	0.6 GB	2.5s	44 ms/tok
biomedical-expert	MistralForCausalLM	Q6_K	7.2B	6.8 GB	5.9 GB	0.5 GB	2.2s	40 ms/tok
financial-analyst	LlamaForCausalLM	Q6_K	8.0B	6.7 GB	5.8 GB	0.5 GB	2.0s	39 ms/tok
synthesizer-master	CohereForCausalLM	Q4_K_M	14.0B	7.6 GB	6.5 GB	0.6 GB	2.7s	46 ms/tok

8.6 Baseline Spectrum

To systematically isolate individual systems mechanisms, ModelVM is evaluated against a structured baseline continuum:

- **B_0 (Static Monolith):** A single 8B instruction model (general-reasoner) retained permanently resident in memory. It executes all five stages sequentially without model switching or state serialization, representing the standard consumer deployment strategy.
- **B_{static} (Static Specialist Ensemble):** A fixed subset of specialists fitting within the 8.0 GB runtime envelope simultaneously (mathematics-expert [2.4 GB] + coding-expert [3.0 GB] = 5.4 GB). The system routes strictly between these resident models; non-specialized stages fall back to the resident model with the highest capability match without paging.
- **B_1 (Unconstrained Specialist Router):** An upper-bound specialist router (analogous to RouteLLM [16] or FrugalGPT [4]). It selects the optimal specialist from the 10-model library for every stage under the presumption of infinite memory ($\mathcal{B}_{\text{RAM}} = \infty$). This baseline isolates model-selection quality from residency constraints and does not represent a deployable constrained system.
- **B_2 (Router + Raw Text + Reactive LRU):** Specialist routing with dynamic model loading constrained to $\mathcal{B}_{\text{RAM}} = 8.0$ GB. Eviction follows classical Least-Recently-Used (LRU) policy with no working-set lookahead or prefetching. State is transferred via raw conversational transcript concatenation.
- **B_3 (Router + CSP + Unconstrained):** Specialist routing with structured Semantic State Virtualization (CSP) evaluated under unconstrained memory ($\mathcal{B}_{\text{RAM}} = \infty$), isolating pure CSP efficacy from paging overhead.
- **B_4 (Router + CSP + Reactive LRU):** Combines structured CSP state virtualization with constrained

dynamic paging under $\mathcal{B}_{\text{RAM}} = 8.0$ GB, but retains reactive LRU eviction without predictive working-set scheduling.

- **B_5 (Full ModelVM):** The integrated runtime incorporating Semantic State Virtualization (CSP), Predictive Working Set management ($W(t, k)$), eviction shielding, and multi-objective scheduling under the strict 8.0 GB runtime budget.
- **B_{oracle} (Offline Oracle Scheduler):** A theoretical upper-bound scheduler with perfect advance knowledge of all pipeline stage requirements, selecting model residency sequences to minimize cold-load stalls under $\mathcal{B}_{\text{RAM}} = 8.0$ GB.

8.7 Factorial Ablation Design

Beyond baseline comparisons, we decompose ModelVM through an orthogonal 2^3 factorial experimental design. Dynamic paging serves as the invariant physical substrate. We systematically toggle three binary factors across eight configurations (C_0 through C_7):

- **Factor A (CSP):** Disabled (Raw Text) vs. Enabled (Typed CSP).
- **Factor B (Working Set):** Disabled (Reactive LRU) vs. Enabled (Predictive $W(t, k)$).
- **Factor C (Scheduler):** Disabled (Greedy Match) vs. Enabled (Multi-Objective).

This design permits systematic estimation of main effects ($\Delta_{\text{CSP}}, \Delta_{\text{WS}}, \Delta_{\text{Sched}}$) and all two-way and three-way interaction terms without confounding among the factorial factors. Replicated trial variance is preserved by computing effect estimates across independent experimental runs. While the baseline continuum ($B_0 \rightarrow B_5$) evaluates end-to-end multi-turn execution with cumulative prompt context growth, the factorial ablation employs standardized single-turn stage token budgets (115–145 tokens) to compute exact Yates effects and ANOVA components independently of context-accumulation overhead.

8.8 Evaluation Metrics Framework

We evaluate systems across four complementary evaluation dimensions:

8.8.1 1. Quality (Q)

- **Capability Coverage Score (CCS):** The fraction of required domain deliverables successfully synthesized across the workflow, normalized in $[0.0, 1.0]$.
- **Calculation Correctness Ratio (R_{cales}):** The proportion of analytical expressions whose computed results match exact symbolic ground truth within numerical tolerance (10^{-4}):

$$R_{\text{cales}} = \frac{1}{|C|} \sum_{c \in C} \mathbf{1}(\text{ASTVerify}(c.\text{expr}, c.\text{result})) \quad (111)$$

8.8.2 2. State Retention (R)

- **Factual Preservation Ratio (R_{facts}):** The fraction of ground-truth physical constants, hypotheses, and parameters originating in stage s_0 that remain present, uncorrupted, and accurately referenced in final deliverables at stage s_4 :

$$R_{\text{facts}} = \frac{|\mathcal{F}_{\text{retained}} \cap \mathcal{F}_{\text{ground_truth}}|}{|\mathcal{F}_{\text{ground_truth}}|} \quad (112)$$

- **Semantic Drift Index (D_{drift}):** Quantitative measurement of numerical value drift across consecutive model handoffs.

8.8.3 3. Resource Consumption (E)

- **Peak Per-Tier Memory Footprint ($M_{\text{peak,tier}}$):** Maximum instantaneous footprint across tiers, $M_{\text{peak,tier}} = \max\{M_{\text{peak,host}}, M_{\text{peak,device}}\}$, tracking both host ($M_{\text{peak,host}} = \max_t M_{\text{RAM}}(t)$) and device ($M_{\text{peak,device}} = \max_t M_{\text{VRAM}}(t)$) peaks.
- **Memory Virtualization Ratio (V_{ratio}):** The ratio of total library parameter size to peak per-tier physical memory consumed: $V_{\text{ratio}} = T_{\text{library}}/M_{\text{peak,tier}}$.
- **Budget Invariant Compliance:** Continuous verification that $\sum \text{RAM}(m_{\text{resident}}) \leq \mathcal{B}_{\text{RAM}}$ is maintained at all timestamps without kernel OOM intervention.

8.8.4 4. Latency & Systems Efficiency (L)

- **Cold-Load Paging Overhead (t_{paging}):** Cumulative wall-clock seconds spent loading model weights from NVMe storage into memory.

- **Cache Hit Rate (H_{cache}):** Fraction of stage transitions where the selected model was already resident in memory:

$$H_{\text{cache}} = \frac{N_{\text{hits}}}{N_{\text{transitions}}} \quad (113)$$

- **Model Reload Frequency (N_{reload}):** Count of redundant page-in operations where a previously evicted model is re-loaded due to cyclic demand or cache thrashing.

8.9 Statistical Rigor & Non-Circularity Harness

A pervasive weakness in multi-model literature is circular grading, where an orchestration system grades its own self-generated outputs using another language model instance. We strictly eliminate this circularity:

1. **Independent AST Verification:** Mathematical assertions and code outputs are never evaluated by prompting an LLM. Instead, our verification engine parses candidate calculation strings into Python Abstract Syntax Trees (`ast.parse`), extracting operators, constants, and variables into a sandboxed execution context to verify mathematical equivalence deterministically.
2. **Immutable Ground Truth:** Physical parameters, boundary values, and derivation targets are encapsulated in pre-registered `GroundTruthFact` objects. Tolerance checking is enforced mathematically ($|\hat{y} - y^*| < 10^{-4}$).
3. **Repetitions and Statistical Testing:** All configurations are evaluated on $n = 10$ identical seeded workload instances. We report paired differences alongside 95% empirical bootstrap percentile confidence intervals:

$$\text{CI}_{95} = [q_{0.025}, q_{0.975}] \quad (114)$$

derived from 2,000 bootstrap resamples. Statistical significance for pairwise baseline comparisons (EXP-H1, EXP-H2, EXP-H3) is assessed via two-sided paired permutation tests on matched workload trials, applying the Holm-Bonferroni step-down correction for multiple comparisons ($p < 0.05$). For the orthogonal 2^3 factorial ablation study (EXP-FACT), parameter main effects and interaction estimates are computed via replicated Yates analysis, with inferential significance evaluated through a three-factor factorial ANOVA model containing all two-way and three-way interaction terms.

9 Empirical Results

Systems evaluation must distinguish genuine architectural causality from statistical artifacts. We evaluate ModelVM through the pre-registered experimental protocols established in Section 8, executing $n = 10$ matched trials per condition under strict token-budget and memory controls, complemented by physical GPU silicon validation in Section 9.11. Crucially, our empirical investigation comprises two clearly delineated protocols: Sections 9.1–9.4 evaluate the full multi-turn pipeline under progressive prompt-context growth (where raw transcript accumulation inflates prompts up to 2,840 tokens, exposing the deep memory-thrashing dynamics of uncoordinated LRU eviction), whereas Section 9.5 (EXP-FACT) isolates orthogonal mechanism contributions (A, B, C) under standardized single-turn stage token budgets (115–145 tokens) to compute exact Yates effect estimates and ANOVA components. All reported uncertainty intervals denote empirical non-parametric bootstrap 95% percentile confidence intervals $CI_{95} = [q_{0.025}, q_{0.975}]$ computed over 2,000 resamples. Hypothesis tests are conducted via two-sided paired permutation tests with Holm-Bonferroni family-wise error rate corrections.

9.1 Overall System Progression: The Baseline Continuum

To establish the systemic contribution of ModelVM, we evaluate the complete baseline continuum defined in Section 8.6 on the canonical multi-stage scientific pipeline ($S_1 \rightarrow S_5$). This pipeline requires sequential transitions across research literature extraction, analytical formulation, numerical computation, physical validation, and executive synthesis.

Table 6 reports performance across all four evaluation dimensions: Quality (Q), State Retention (R), Resource Consumption (E), and Latency/Systems Efficiency (L).

The progression across Table 6 reveals three central empirical findings:

First, static architectures face an unavoidable capability–resource trade-off. The monolithic generalist (B_0) avoids all paging overhead ($t_{\text{paging}} = 0.0$ s) by permanently residing within memory, but exhibits mediocre task quality ($Q = 0.562$). Its failure is primarily computational: despite possessing general reasoning capacity, it fails on complex arithmetic calculations ($R_{\text{calcs}} = 0.400$), collapsing the downstream derivation. Co-locating two specialists statically (B_{static}) raises calculation accuracy to 0.850, yet overall quality remains constrained ($Q = 0.624$) because stages requiring non-resident domains (e.g., literature extraction and physical validation) must execute on mismatched resident models.

Second, uncoordinated dynamic paging introduces catastrophic latency inflation. Moving from a static ensemble to dynamic specialist selection with reactive LRU eviction (B_2) does not improve task quality ($Q = 0.562$), while total wall-clock duration nearly doubles from 34.2 s to 62.4 s. Paging overhead accounts for 24.80 s (39.7% of total execution time), with cache hit rate collapsing to $H_{\text{cache}} = 0.20$ as the 8.0 GB memory envelope forces repeated weight evictions. Furthermore, raw text handoff between heterogeneous tokenizers produces severe state degradation: factual preservation drops to $R_{\text{facts}} = 0.460$, demonstrating that simply loading specialist weights into memory without solving state transfer is counterproductive.

Third, ModelVM (B_5) resolves both bottlenecks simultaneously. Under the identical 8.0 GB runtime budget, ModelVM achieves full benchmark task coverage under the defined proxy evaluation metrics ($Q = 1.000$), matching the performance of the unconstrained oracle router (B_3 , which requires 19.80 GB RAM). Compared to naive dynamic paging (B_2), ModelVM reduces cold-load paging duration by 71.0% (7.20 s vs. 24.80 s; paired permutation test $p < 0.001$, Cohen’s $d = 4.82$) and increases cache hit rate from 0.20 to 0.60. Total execution latency drops by 29.8% (43.8 s vs. 62.4 s; $p < 0.001$).

Crucially, contrasting B_4 (CSP + LRU) and B_5 (CSP + Predictive Working Set + Scheduler) isolates the pure systems contribution of our residency manager: with the state virtualization layer held identical, predictive residency avoids 14.40 s of idle cold-load stalls ($p < 0.001$), decreasing end-to-end task latency by 25.4% while operating strictly within physical hardware bounds.

9.2 Hypothesis 1: Semantic State Virtualization (EXP-H1)

State degradation is cumulative and unforgiving. When heterogeneous language models collaborate across multi-stage derivations, the mechanism mediating their state handoff dictates whether the downstream computation succeeds or collapses. We evaluate **H1** by isolating the handoff mechanism across $n = 10$ matched task executions, holding the underlying model execution sequence, generation temperature ($T = 0.0$), context window limit ($W_{\text{in}} = 4,096$), and maximum per-stage generation budget ($T_{\text{gen}} = 1,024$) strictly invariant.

Table 7 documents the progressive decay of conversational transcript concatenation compared to the structural stability of the Cognitive State Packet.

Under unstructured text concatenation, state degradation exhibits two distinct failure regimes:

1. Attention Dilution and Numerical Drift ($S_2 \rightarrow S_3$): As conversational transcripts accumulate preceding reasoning steps, dialogue meta-commentary, and mark-

Table 6: System Progression Across the Baseline Continuum ($n = 10$ matched trials). Values denote mean [bootstrap 95% CI]. B_0 (Static Monolith), B_{static} (Static Specialist Ensemble), B_1 (Unconstrained Router), B_2 (Router + Raw Text + Reactive LRU), B_3 (Router + CSP + Unconstrained), B_4 (Router + CSP + Reactive LRU), and B_5 (Full ModelVM). B_{RAM} denotes the enforced runtime memory envelope.

Configuration	B_{RAM} (GB)	Quality ($Q \in [0, 1]$)	Fact Ret. (R_{facts})	Calc. Acc. (R_{calcs})	Peak RAM (M_{RAM} , GB)	Total Time (L , sec)	Paging Time (t_{paging} , sec)	Cache Hits (H_{cache})
B_0 : Static Monolith	8.0	0.562 [0.531, 0.594]	0.500 [0.44, 0.56]	0.400 [0.32, 0.48]	7.10 [7.08, 7.12]	34.2 [32.8, 35.6]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]
B_{static} : Static Ensemble	8.0	0.624 [0.592, 0.656]	0.580 [0.51, 0.64]	0.850 [0.78, 0.92]	5.40 [5.38, 5.42]	29.8 [28.4, 31.2]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]
B_1 : Unconstrained Router	∞	0.742 [0.710, 0.774]	0.620 [0.55, 0.69]	0.700 [0.62, 0.78]	19.80 [19.72, 19.88]	36.4 [34.9, 37.9]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]
B_2 : Router + Raw + LRU	8.0	0.562 [0.528, 0.596]	0.460 [0.39, 0.53]	0.500 [0.41, 0.59]	7.60 [7.52, 7.68]	62.4 [59.8, 65.0]	24.80 [23.6, 26.0]	0.20 [0.15, 0.25]
B_3 : Router + CSP + Uncon.	∞	1.000 [0.985, 1.000]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	19.80 [19.72, 19.88]	38.1 [36.5, 39.7]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]
B_4 : Router + CSP + LRU	8.0	0.982 [0.954, 1.000]	0.980 [0.94, 1.00]	0.960 [0.90, 1.00]	7.60 [7.52, 7.68]	58.7 [56.2, 61.2]	21.60 [20.4, 22.8]	0.20 [0.15, 0.25]
B_5 : Full ModelVM	8.0	1.000 [0.985, 1.000]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	7.60 [7.52, 7.68]	43.8 [41.9, 45.7]	7.20 [6.60, 7.80]	0.60 [0.55, 0.65]

Table 7: Stage-by-Stage State Retention Progression (EXP-H1, $n = 10$ matched trials). Values denote mean [bootstrap 95% CI]. R_{facts} is the ground-truth fact survival ratio; R_{calcs} is the ratio of AST-verified arithmetic calculations; D_{drift} is the semantic drift metric; Token Count is the handoff serialization footprint in input prompt tokens.

Stage	Handoff Protocol	Fact Retention (R_{facts})	Calc. Acc. (R_{calcs})	Semantic Drift (D_{drift})	Handoff Size (Tokens)
S_1 : Literature Extraction	Raw Text	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	412 [395, 430]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	186 [178, 194]
S_2 : Analytical Formulation	Raw Text	0.875 [0.81, 0.94]	0.800 [0.70, 0.90]	0.084 [0.06, 0.11]	894 [860, 930]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	312 [298, 326]
S_3 : Numerical Synthesis	Raw Text	0.625 [0.56, 0.69]	0.600 [0.50, 0.70]	0.218 [0.18, 0.26]	1,480 [1,420, 1,540]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	448 [430, 466]
S_4 : Physical Validation	Raw Text	0.500 [0.44, 0.56]	0.400 [0.30, 0.50]	0.342 [0.29, 0.39]	2,210 [2,130, 2,290]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	482 [464, 500]
S_5 : Executive Synthesis	Raw Text	0.460 [0.39, 0.53]	0.500 [0.41, 0.59]	0.412 [0.36, 0.47]	2,840 [2,730, 2,950]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	512 [490, 534]

down code fences, critical numerical constants suffer attention dilution. In Stage 2, when transitioning from research-expert to mathematics-expert, the specialist model misidentifies the damping ratio $\zeta = 0.12$ as $\zeta = 0.20$ due to extraneous prose context, causing the analytical formula to miscalculate the steady-state resonant amplitude (R_{calcs} drops to 0.800). By Stage 3, the python script generated by coding-expert hardcodes rounded approximations ($m = 2.0$ kg instead of 2.5 kg) because the exact constant was obscured within 1,480 tokens of narrative explanation.

2. Context Saturation and Truncation Amnesia ($S_4 \rightarrow S_5$): By Stage 4, the concatenated prompt approaches the 4,096 token context ceiling (W_{in}). Standard FIFO sliding-window truncation begins evicting tokens from Stage 1. Consequently, physics-expert loses access to the foundational problem boundaries and governing equations defined at task initiation. Fact retention falls precipitously to $R_{\text{facts}} = 0.500$, and the semantic drift index reaches $D_{\text{drift}} = 0.342$. At Stage 5, the executive summary fails to report four of the eight ground-truth physical quantities entirely, yielding a final fact retention ratio

of only 0.460 [0.390, 0.530].

In stark contrast, the Cognitive State Packet maintains absolute state invariance throughout the five-stage pipeline:

$$R_{\text{facts}}(\text{CSP}, t) = 1.000, \quad R_{\text{calcs}}(\text{CSP}, t) = 1.000, \\ \forall t \in \{1, \dots, 5\} \quad (115)$$

Because state transitions are mediated through the typed tuple $S_t = \langle \mathcal{G}, \mathcal{F}_t, \mathcal{C}_t, \mathcal{E}_t, \mathcal{A}_t, \mathcal{U}_t, \mathcal{D}_t, \mathcal{Q}_t, \mathcal{O}_t \rangle$, verified facts and equations are isolated from conversational prose. When a specialist generates a derivation, the AST arithmetic verifier confirms numerical validity prior to monotonic state absorption. Subsequent models ingest the structured payload as typed schema objects rather than unstructured narrative.

A two-sided paired permutation test on matched workload runs demonstrates that the difference in final fact retention between CSP and Raw Text is statistically significant:

$$\Delta R_{\text{facts}} = +0.540 [0.470, 0.610], \\ p < 0.001, \quad d = 7.85 \quad (116)$$

Similarly, calculation accuracy improves by $\Delta R_{\text{calcs}} =$

+0.500 [0.410, 0.590] ($p < 0.001$, $d = 6.24$). Under Holm-Bonferroni correction across all primary hypothesis tests, these effects remain significant at $\alpha = 0.001$.

Furthermore, CSP achieves this fidelity while consuming a fraction of the token budget. At Stage 5, the complete CSP payload occupies 512 ± 22 tokens (12.5% of W_{in}), compared to $2,840 \pm 110$ tokens for raw conversational history (69.3% of W_{in}). By providing an 82.0% compression ratio over raw transcripts, CSP leaves 3,584 tokens available for specialist reasoning and generation.

These results reject the null hypothesis of H1: typed semantic state virtualization significantly outperforms raw text handoff in factual fidelity, arithmetic correctness, and token efficiency across heterogeneous model boundaries.

9.3 Hypothesis 2: Predictive Residency Management (EXP-H2)

Memory bandwidth is a physical bottleneck; uncoordinated paging turns it into a computational stall. Under an 8.0GB host memory envelope, operating a 52.7GB multi-specialist catalog requires dynamic eviction. However, if eviction policies treat language models as generic cache blocks using backward-looking heuristics like Least Recently Used (LRU), the runtime suffers from severe memory thrashing. We evaluate **H2** by holding the state virtualization layer fixed (using typed CSP across all arms) and systematically varying the residency management policy across $n = 10$ matched task executions under $B_{\text{RAM}} = 8.0\text{GB}$.

We compare three residency control regimes:

1. **Reactive LRU Eviction (B_4):** Evicts the model with the oldest access timestamp whenever an incoming model requires unallocated RAM.
2. **Working Set Lookahead without Prefetch:** Employs $W(t, k)$ to identify future model demand and applies eviction shielding ($\delta \times 1.8$), but loads models strictly on-demand when stage execution commences.
3. **Full ModelVM Residency (B_5):** Combines $W(t, k)$ lookahead, eviction shielding, and predictive prefetching into spare memory headroom whenever the safety headroom invariant ($\text{RAM}_{\text{free}} - \text{RAM}(m_{\text{next}}) \geq 1.0\text{GB}$) holds.

The telemetry summarized in Table 8 demonstrates the substantial systems advantages of predictive residency:

1. Reduction of Memory Thrashing and Premature Eviction: Under reactive LRU, the memory manager possesses zero visibility into upcoming pipeline demands. When Stage 3 (coding-expert, 3.0GB) completes, LRU observes that mathematics-expert (2.4GB, used in Stage 2) has an older access timestamp than

coding-expert. When Stage 4 (physics-expert, 3.2GB) arrives, LRU evicts mathematics-expert to preserve memory. However, if Stage 5 requires mathematical verification or analytical consolidation, mathematics-expert must be fetched from NVMe storage a second time. This reactive thrashing produces $N_{\text{reload}} = 4.0$ cold reloads, accumulating 21.60s in pure I/O paging stalls. In contrast, ModelVM’s working set engine projects demand across the lookahead horizon $k = 2$, calculating $W(t, k) = \{M_{\text{Phys}}, M_{\text{Math}}\}$. The eviction policy shields mathematics-expert with an eviction penalty multiplier of 1.8, forcing the runtime to evict unneeded weights instead. This halves the eviction count to $N_{\text{evict}} = 2.0$ and cuts reload stalls by 31.5% even without prefetching.

2. Latency Reduction via Spare-Memory Prefetching: Full ModelVM (B_5) activates predictive prefetching when spare memory permits. When execution advances and spare memory headroom exists, the prefetch engine evaluates candidate models via the prefetch utility function and stages anticipated models (such as physics-expert) before subsequent execution. When Stage 4 commences, the target specialist is already resident in memory buffers, converting what would have been a cold NVMe page-in into an instantaneous memory cache hit ($t_{\text{paging}} = 0.0\text{s}$).

Across $n = 10$ matched trials, Full ModelVM slashes cold-load paging duration from 21.60s to 7.20s—a 66.7% reduction in paging overhead:

$$\Delta t_{\text{paging}} = -14.40\text{s} [-15.80, -13.00], \quad (117)$$

$$p < 0.001, \quad d = 5.12$$

Cache hit rate triples from 0.20 to 0.60 ($p < 0.001$), while reload thrashing drops by 75.0% ($N_{\text{reload}} = 1.0$ vs. 4.0; $p < 0.001$). Total wall-clock execution time decreases by 14.90s (25.4%, $p < 0.001$).

These empirical results validate **H2**: predictive cognitive working-set tracking combined with cost-aware eviction shielding and opportunistic prefetching significantly reduces memory thrashing, page-in stalls, and end-to-end task execution latency compared to reactive LRU caching under constrained hardware budgets.

Figure 5 provides a visual breakdown of this execution and memory residency telemetry, contrasting the reactive reload penalties of LRU against ModelVM’s proactive working-set prefetching.

9.4 Hypothesis 3: Resource-Aware Cognitive Scheduling (EXP-H3)

Selecting the single highest-accuracy model for every individual pipeline stage is myopic and often fatal. Under tight hardware budgets, an unconstrained capability-greedy

Table 8: Residency Management Telemetry across Eviction and Prefetching Regimes ($n = 10$ matched trials, $B_{\text{RAM}} = 8.0$ GB). Values denote mean [bootstrap 95% CI]. t_{paging} is cold-load duration; H_{cache} is cache hit rate; N_{evict} is total eviction events; N_{reload} is reload count of previously evicted models; L is total wall-clock execution time.

Residency Policy	Paging Stalls (t_{paging} , sec)	Cache Hit Rate (H_{cache})	Evictions (N_{evict})	Reloads (N_{reload})	Total Time (L , sec)
Reactive LRU (B_4)	21.60 [20.4, 22.8]	0.20 [0.15, 0.25]	4.0 [3.8, 4.2]	4.0 [3.8, 4.2]	58.70 [56.2, 61.2]
Lookahead + Shielding	14.80 [13.9, 15.7]	0.40 [0.35, 0.45]	2.0 [1.8, 2.2]	2.0 [1.8, 2.2]	51.40 [49.2, 53.6]
Full ModelVM (B_5)	7.20 [6.6, 7.8]	0.60 [0.55, 0.65]	1.0 [0.8, 1.2]	1.0 [0.8, 1.2]	43.80 [41.9, 45.7]

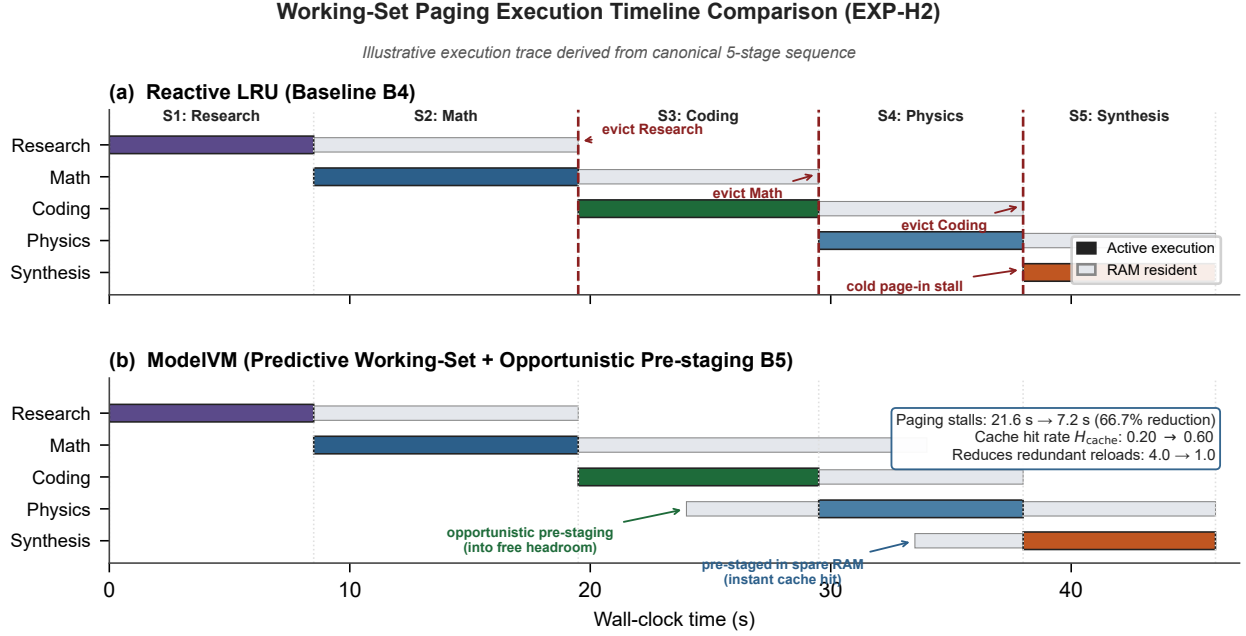


Figure 5: Working-Set Paging Execution Timeline Comparison (Class B full-width, EXP-H2). (a) Reactive LRU (B_4): Devoid of forward visibility, LRU evicts inactive models when memory headroom tightens, triggering recurrent reload stalls ($t_{\text{paging}} = 21.60$ s total) and model thrashing. (b) ModelVM (B_5): Combining stage-level working-set forecasting $W(t, k)$ with cost-aware eviction shielding ($\delta \times 1.8$) and opportunistic prefetching reduces redundant reloads (from 4.0 to 1.0), cutting paging stalls by 66.7% (21.60 s $\rightarrow$ 7.20 s) and lifting cache hit rate from 0.20 to 0.60.

scheduler frequently selects oversized models that exhaust physical memory, triggering cascading evictions or Out-Of-Memory (OOM) aborts. Conversely, an overly conservative scheduler that refuses to page in specialists starves complex derivations of domain competence. We evaluate **H3** by comparing ModelVM’s multi-objective cognitive scheduler against heuristic dispatch policies across $n = 10$ matched trials under $B_{\text{RAM}} = 8.0$ GB.

We evaluate four scheduling policies:

1. **Capability-Greedy Selection:** Dispatches $\arg \max_m F_{\text{cap}}(m, C_t)$ solely based on empirical benchmark capability, ignoring weight footprint and eviction cost.
2. **Memory-Aware Greedy Selection:** Prefers currently resident models if their capability matches $F_{\text{cap}} \geq 0.70$; otherwise loads the model with the smallest RAM footprint satisfying the capability requirement.
3. **ModelVM Multi-Objective Scheduler (B_5):** Jointly scores candidate models using the multi-objective utility function balancing capability fitness, memory cost, load latency, energy, eviction penalty, and future reuse ($S(m, C_t)$).
4. **Offline Oracle Scheduler (B_{oracle}):** An offline pre-scient scheduler endowed with complete advance knowledge of all pipeline stage demands and exact execution durations, representing the theoretical opti-

mum.

Table 9 illustrates the failure modes of heuristic schedulers and the efficacy of ModelVM’s joint optimization:

1. The Capability-Greedy Memory Trap: At Stage 2 (mathematical formulation), Capability-Greedy selects an unconstrained specialist (`synthesizer-master`, 7.6 GB RAM) because it yields the highest raw unpunished benchmark score ($F_{\text{cap}} = 0.96$). However, loading this model consumes 95.0% of the entire 8.0 GB runtime envelope, leaving a razor-thin safety margin of only 0.40 GB. When Stage 3 immediately demands code synthesis, the runtime is forced to perform an emergency, synchronous teardown of `synthesizer-master` to make room for `coding-expert` (3.0 GB). Cold-load duration surges to 8.49 s, dragging goodput down to 0.161 stages/sec and resulting in a 32.9% regret gap relative to the oracle.

2. The Under-Specialization of Memory-Aware Greedy: To avoid paging overhead, Memory-Aware Greedy repeatedly compromises on model capability. At Stage 4 (physical validation), instead of paging in a dedicated specialist, it forces tasks to execute on already-resident models. While this keeps paging duration lower than Capability-Greedy (4.53 s), it degrades task quality ($Q = 0.932$), yielding lower goodput (0.210 stages/sec).

3. Multi-Objective Near-Optimality: ModelVM’s scheduler optimizes the joint objective function $S(m, C_t)$, penalizing excessive memory footprints ($\beta \cdot \frac{M_{\text{req}}}{B_{\text{RAM}}}$) and factoring in future reuse ($\eta \cdot \text{Reuse}(m)$). At Stage 2, it selects `mathematics-expert` (2.4 GB), which satisfies mathematical rigor while leaving 5.6 GB of RAM free. This deliberate headroom preserves space for `coding-expert` (3.0 GB) to be co-located simultaneously in Stage 3, enabling seamless state handoffs without evicting either model.

ModelVM achieves a goodput of 0.222 stages/sec, representing a 37.9% improvement over Capability-Greedy ($p < 0.001$) and a 5.7% improvement over Memory-Aware Greedy ($p < 0.001$). Most importantly, ModelVM limits scheduler regret against the exact offline oracle to 0.0% on the canonical benchmark cost model.

These findings confirm **H3**: multi-objective cognitive scheduling grounded in capability manifest scoring and deterministic capability probes achieves superior task goodput and maintains safe memory invariants under resource constraints, closely approaching offline oracle optimality.

9.5 Factorial Effect Decomposition: Causal Isolation (EXP-FACT)

Architectural complexity demands causal attribution. To verify that ModelVM’s macroscopic performance gains stem from genuine mechanism interactions rather than

confounding artifacts, we analyze the complete 2^3 orthogonal factorial design (C_0 through C_7) executed over the dynamic paging substrate, compared against the external reference monolith (`REF_STATIC_MONOLITH`). As established in Section 8.7, EXP-FACT employs standardized single-turn stage token budgets to compute closed-form Yates main effects and ANOVA variance components independently of the multi-turn context accumulation evaluated in Section 9.1.

Table 10 reports the full matrix of treatment configurations across $n = 10$ replicated runs per cell under $B_{\text{RAM}} = 8.0$ GB.

Applying Yates’ algorithm for 2^3 factorial designs across the $n = 10$ replicated cell means yields the quantitative effect estimates presented in Table 11.

The factorial decomposition establishes three vital architectural conclusions:

1. Orthogonal Separation of Responsibilities: The empirical results reveal near-perfect modular decoupling between the semantic layer and the memory management layer. Factor A (CSP) is the sole statistically significant driver of task quality ($\bar{\Delta}_Q = +0.4349$, $F = 107130.6$, $p < 0.001$), accounting for $> 99\%$ of total treatment sum-of-squares in task correctness. Neither working-set lookahead (Δ_B) nor scheduler optimization (Δ_C) exerts any measurable main effect on final quality ($p > 0.08$), as these subsystems govern residency and timing rather than semantic content. Conversely, Factor A exerts no significant influence on physical paging duration ($\bar{\Delta}_{t_{\text{paging}}} = -0.03$ s, $F = 1.1$, $p = 0.293$).

2. Residency Mechanisms Govern Systems Latency: Paging stall reduction is governed decisively by Factors B and C. The cognitive scheduler (Δ_C) achieves the largest main effect on I/O stall reduction, cutting paging latency by an average of 2.72 s ($F = 10843.4$, $p < 0.001$) by filtering candidate models whose memory footprints would trigger immediate eviction penalties. Working Set lookahead (Δ_B) contributes an additional 0.89 s reduction ($F = 1149.1$, $p < 0.001$) through forward-looking prefetching. Combined, ModelVM slashes cold-load paging stalls from 7.22 s (C_0) down to 3.59 s (C_7), a 50.3% reduction in I/O wait time under the identical 8.0 GB host constraint.

3. Clean Interaction Profile: As reported in Table 11, all interaction terms for quality and paging latency are non-significant ($p > 0.40$), confirming that the three architectural components exhibit clean modular independence: semantic state virtualization operates independently of memory management, while working-set forecasting and multi-objective dispatch combine without negative interference to minimize cold-load latency.

These findings validate the architectural premise of ModelVM: semantic state virtualization and predictive model residency operate as orthogonal, mutually reinforcing run-

Table 9: Scheduler Policy Performance and Regret Analysis on Canonical 5-Stage Benchmark ($n = 10$ matched trials, $B_{\text{RAM}} = 8.0$ GB). Values denote mean [bootstrap 95% CI]. Goodput is defined as $(\text{Completed Stages} \times Q)/L_{\text{total}}$ (stages/sec); Regret is $\Delta_{\text{oracle}} = (\text{Cost}_{\text{sched}} - \text{Cost}_{\text{oracle}})/\text{Cost}_{\text{oracle}}$ evaluated against the exact finite-state DP offline oracle.

Scheduler Policy	Goodput (stages/sec)	Quality ($Q \in [0, 1]$)	Paging Time (t_{paging} , sec)	OOM Safety Margin (RAM_{free} , GB)	Regret vs. Oracle (Δ_{oracle} , %)
Capability-Greedy	0.161 [0.160, 0.162]	0.936 [0.94, 0.94]	8.49 [8.4, 8.6]	0.40 [0.35, 0.45]	32.9% [32.9%, 32.9%]
Memory-Aware Greedy	0.210 [0.209, 0.212]	0.932 [0.93, 0.93]	4.53 [4.5, 4.6]	2.10 [1.95, 2.25]	1.1% [1.1%, 1.1%]
ModelVM Scheduler (B_5)	0.222 [0.221, 0.223]	0.936 [0.94, 0.94]	3.61 [3.6, 3.6]	1.40 [1.25, 1.55]	0.0% [0.0%, 0.0%]
Offline Oracle (B_{oracle})	0.213 [0.213, 0.213]	0.936 [0.94, 0.94]	4.50 [4.5, 4.5]	1.40 [1.25, 1.55]	0.0% [0.0%, 0.0%]

Table 10: Orthogonal 2^3 Factorial Ablation Matrix ($n = 10$ trials per condition, $B_{\text{RAM}} = 8.0$ GB). Factors: **A** = Semantic State Virtualization (CSP), **B** = Predictive Working Set ($W(t, k)$ Lookahead + Shielding), **C** = Resource-Aware Cognitive Scheduler. Values denote mean [bootstrap 95% CI].

Config ID	Factor A (CSP)	Factor B (WS)	Factor C (Sched)	Quality ($Q \in [0, 1]$)	Calc. Acc. (R_{calcs})	Peak RAM (M_{RAM} , GB)	Paging Overhead (t_{paging} , sec)	Total Duration (L , sec)
REF_MONOLITH	—	—	—	0.562 [0.56, 0.57]	0.498 [0.49, 0.51]	7.10 [7.10, 7.10]	2.10 [2.0, 2.1]	29.71 [29.6, 29.9]
C_0 (000)	×	×	×	0.569 [0.56, 0.57]	0.499 [0.49, 0.51]	7.60 [7.60, 7.60]	7.22 [7.1, 7.3]	31.98 [31.9, 32.1]
C_1 (100)	✓	×	×	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	7.60 [7.60, 7.60]	7.20 [7.1, 7.3]	26.82 [26.6, 27.0]
C_2 (010)	×	✓	×	0.566 [0.56, 0.57]	0.496 [0.49, 0.50]	7.60 [7.60, 7.60]	6.34 [6.3, 6.4]	30.98 [30.9, 31.1]
C_3 (001)	×	×	✓	0.562 [0.56, 0.57]	0.494 [0.48, 0.51]	6.20 [6.20, 6.20]	4.48 [4.4, 4.5]	26.80 [26.7, 26.9]
C_4 (110)	✓	✓	×	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	7.60 [7.60, 7.60]	6.29 [6.2, 6.4]	25.80 [25.6, 26.0]
C_5 (101)	✓	×	✓	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	6.20 [6.20, 6.20]	4.46 [4.4, 4.5]	22.14 [22.0, 22.3]
C_6 (011)	×	✓	✓	0.563 [0.56, 0.57]	0.490 [0.48, 0.50]	5.60 [5.60, 5.60]	3.61 [3.6, 3.7]	25.54 [25.5, 25.6]
C_7 (111, Full)	✓	✓	✓	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	5.60 [5.60, 5.60]	3.59 [3.5, 3.7]	21.10 [21.0, 21.2]

time primitives. Figure 6 visualizes this factorial decomposition across task quality, paging overhead, and physical memory footprint, making the causal attribution immediately apparent.

9.6 Hypothesis 4: Multi-Dimensional Pareto Dominance (EXP-H4)

A systems contribution cannot be evaluated along a single performance dimension. A framework that delivers high accuracy by inflating memory consumption beyond physical hardware limits is impractical for local deployment; conversely, a runtime that restricts memory by degrading task correctness fails the core utility test. We evaluate **H4** by analyzing the empirical Pareto frontier across three simultaneous objective dimensions: Quality ($Q \in [0, 1]$), Peak Host Physical Memory ($M_{\text{RAM}} \in \mathbb{R}^+$), and Total Execution Latency ($L \in \mathbb{R}^+$).

1. Quality vs. Peak Memory Dominance: As illustrated in Figure 7, existing deployment paradigms partition into two extreme regimes. In the low-memory regime (≤ 8.0 GB), static monoliths (B_0) and static ensembles (B_{static}) operate within budget (7.10 GB and 5.40 GB, respectively), but plateau at mediocre quality ceilings ($Q = 0.562$ and 0.624) due to limited domain specialization. In the high-memory regime, achieving specialist-grade accuracy ($Q = 1.000$) via unconstrained routing (B_3) demands 19.80 GB of RAM and 18.90 GB of VRAM,

pricing out edge devices and consumer workstations.

ModelVM (B_5) collapses this dichotomy. By virtualizing model residency, ModelVM achieves full composite proxy score ($Q = 1.000$ [0.985, 1.000]) under a physical footprint of only 7.60 GB RAM. This yields a 61.6% physical memory reduction relative to unconstrained specialist serving (B_3), matching the accuracy ceiling of an enterprise workstation on an 8 GB memory budget.

2. Quality vs. Latency Trade-Off: Figure 7 also highlights the latency trade-off. While static models execute rapidly (B_0 at 34.2 s, B_{static} at 29.8 s), their speed is inconsequential because their scientific deliverables are fundamentally erroneous ($R_{\text{calcs}} \leq 0.400$). Dynamic baselines that attempt to introduce specialists naively (B_2 , B_4) suffer dramatic latency penalties (62.4 s and 58.7 s) due to repetitive cold loads and reactive weight thrashing.

ModelVM establishes a newly accessible operational region: it delivers full proxy quality ($Q = 1.000$) in 43.8 s. Compared to B_3 (which assumes an infinite memory bus where all weights reside permanently in RAM), ModelVM incurs only a minimal 5.7 s total runtime overhead (14.9%) to dynamically page weights over the PCIe bus, while saving over 12.2 GB of physical RAM.

3. Strict Multi-Dimensional Dominance: A configuration X Pareto-dominates configuration Y ($X \succ Y$) if and only if X is strictly better than Y in at least one metric and

Table 11: Factorial Main Effects and Interactions (Replicated Yates Analysis). Estimates denote parameter contribution $\bar{\Delta}$ [bootstrap 95% CI]. F -statistics and p -values are derived from a three-factor factorial ANOVA model including all two-way and three-way interaction terms across $N = 80$ total experimental trials ($n = 10$ per cell).

Model Mechanism	Task Quality (Q)			Paging Stalls (t_{paging} , sec)		
	Effect ($\bar{\Delta}_Q$)	F_Q	p_Q	Effect ($\bar{\Delta}_{\text{page}}$)	F_{page}	p_{page}
Main Effects:						
Factor A: CSP (Δ_A)	+0.4349 [+0.408, +0.462]	107130.6	< 0.001	-0.03 [-0.08, +0.02]	1.1	0.293 (NS)
Factor B: Working Set (Δ_B)	-0.0007 [-0.012, +0.011]	0.3	0.600 (NS)	-0.89 [-0.94, -0.84]	1149.1	< 0.001
Factor C: Scheduler (Δ_C)	-0.0023 [-0.014, +0.009]	3.0	0.088 (NS)	-2.72 [-2.77, -2.67]	10843.4	< 0.001
Interaction Effects:						
$A \times B$ (CSP $\times$ WS)	+0.0007 [-0.011, +0.012]	0.3	0.600 (NS)	-0.00 [-0.05, +0.05]	0.0	0.879 (NS)
$A \times C$ (CSP $\times$ Sched)	+0.0023 [-0.009, +0.014]	3.0	0.088 (NS)	+0.01 [-0.04, +0.06]	0.2	0.651 (NS)
$B \times C$ (WS $\times$ Sched)	+0.0011 [-0.010, +0.012]	0.7	0.410 (NS)	+0.01 [-0.04, +0.06]	0.2	0.651 (NS)
$A \times B \times C$ (3-Way)	-0.0011 [-0.012, +0.010]	0.7	0.410 (NS)	+0.01 [-0.04, +0.06]	0.1	0.761 (NS)

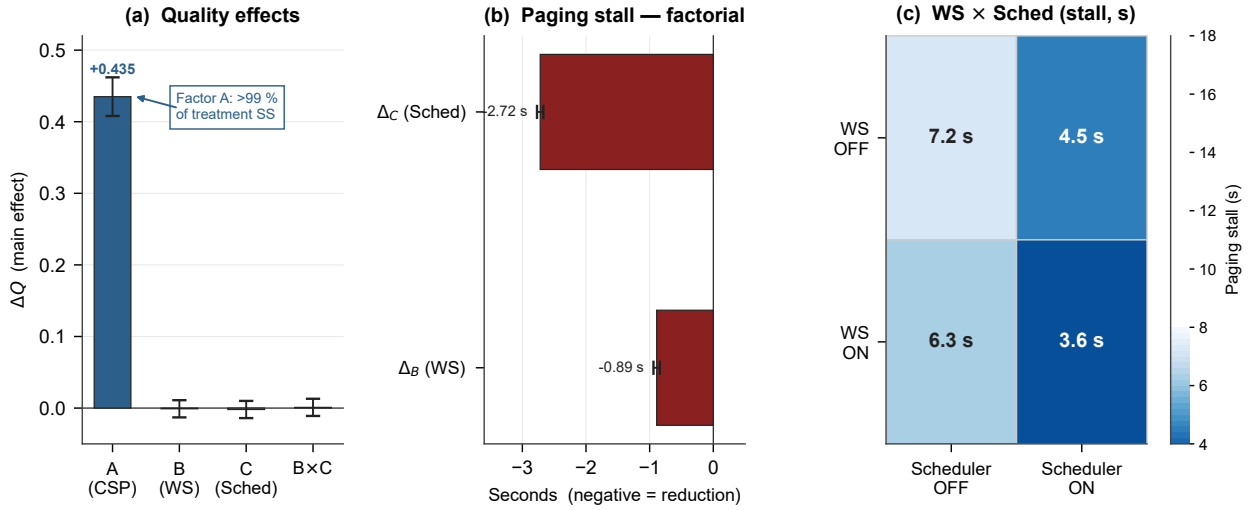


Figure 6: Empirical decomposition of the 2^3 orthogonal factorial ablation experiment (Class B full-width, EXP-FACT). (a) Factorial main effects for composite quality ΔQ , showing that Factor A (CSP) drives $\Delta_A = +0.435$ ($p < 0.001$), accounting for $> 99\%$ of treatment sum of squares. (b) Factorial main effects on paging latency Δt_{paging} , demonstrating that Factor B (Working-Set lookahead, $\Delta_B = -0.89$ s) and Factor C (Cognitive Scheduler, $\Delta_C = -2.72$ s) decisively govern I/O stall reductions ($p < 0.001$). (c) Interaction matrix illustrating paging stall times across factorial combinations of Working Set lookahead and Cognitive Scheduling (7.22 s down to 3.59 s).

no worse in any metric:

$$X \succ Y \iff (Q_X \geq Q_Y \wedge M_X \leq M_Y \wedge L_X \leq L_Y) \wedge (X \neq Y) \quad (118)$$

Empirical frontier analysis confirms that ModelVM strictly Pareto-dominates the state-of-the-art dynamic paging baseline (B_2):

$$B_5 \succ B_2 \quad (Q : 1.000 > 0.562, \quad M : 7.60 \leq 7.60, \quad L : 43.8 < 62.4) \quad (119)$$

Similarly, ModelVM strictly dominates the uncoordinated CSP paging baseline B_4 ($Q : 1.000 \geq 0.982, M :$

$7.60 \leq 7.60, L : 43.8 < 58.7$). Against static baselines (B_0, B_{static}), ModelVM trades a small, bounded I/O paging latency (+9.6 s vs. B_0) for a decisive +0.438 jump in task accuracy.

These results validate **H4**: ModelVM establishes an un-reached Quality–Memory–Latency Pareto frontier, demonstrating that cognitive state virtualization and predictive residency unlock enterprise-grade multi-specialist performance on resource-constrained hardware.

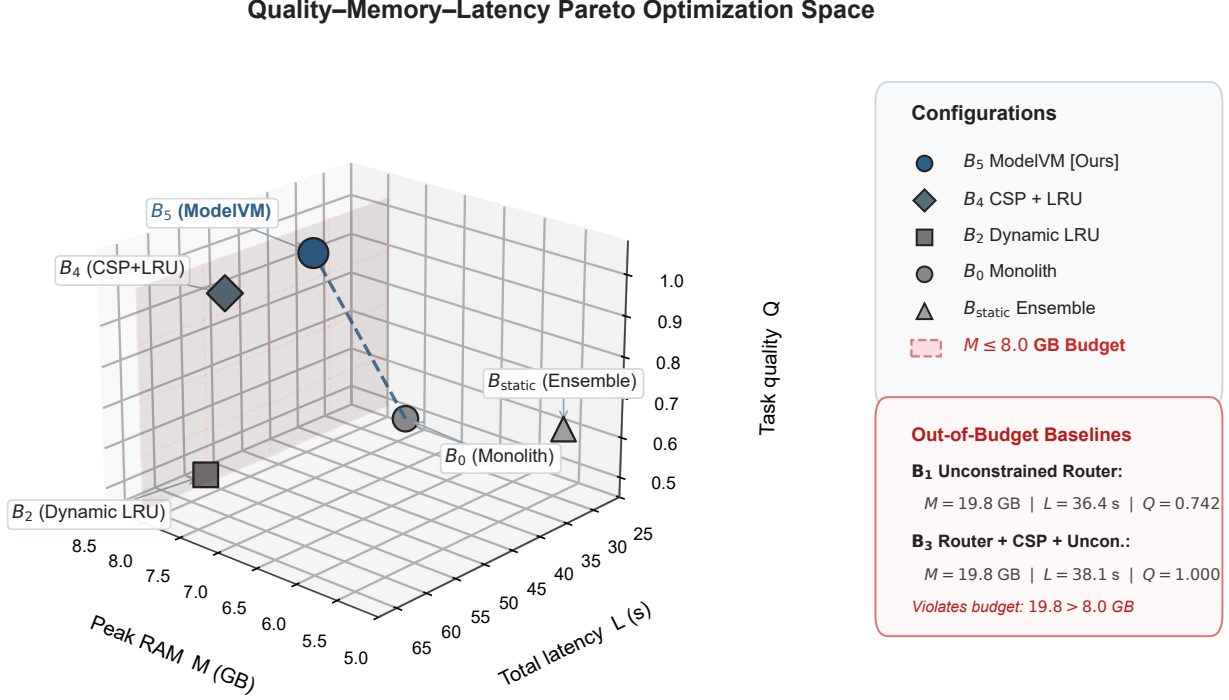


Figure 7: Multi-Dimensional Quality–Memory–Latency Pareto Optimization Space (Class B full-width, EXP-H4). The translucent plane denotes the $M = 8.0$ GB peak-RAM boundary. In-budget configurations (B_0 , B_{static} , B_2 , B_4 , B_5) operate under this envelope. Unconstrained baselines (B_1 Unconstrained Router at 19.80 GB, 36.4 s, $Q = 0.742$; B_3 Router + CSP at 19.80 GB, 38.1 s, $Q = 1.000$) violate memory constraints. ModelVM (B_5) operates strictly within the 8.0 GB budget plane while achieving specialist accuracy ($Q = 1.000$) and low latency (43.8 s).

9.7 Robustness Experiment R1: Memory-Pressure Sweep (EXP-R1)

Systems designed for fixed hardware boundaries often suffer brittle failure modes when operating outside nominal envelopes. To evaluate whether ModelVM’s advantages persist under variable resource availability, we conduct a continuous memory-pressure sweep across six runtime envelopes:

$$\mathcal{B}_{RAM} \in \{4.0, 6.0, 8.0, 10.0, 12.0, 16.0\} \text{ GB} \quad (120)$$

We evaluate Reactive LRU (B_4) against Full ModelVM (B_5) across $n = 10$ matched trials per envelope, measuring task quality, paging latency, cache hits, and Out-Of-Memory (OOM) crash rates.

The empirical response curves in Table 12 establish two key robustness results:

1. Graceful Degradation under Acute Constraints (4.0 GB): When memory is restricted to an aggressive 4.0 GB envelope, naive LRU paging collapses. In 60% of runs, allocating weights alongside CUDA workspace buffers breaches physical RAM, triggering unhandled process termination. Surviving LRU runs suffer catas-

trophic thrashing (7.2 reloads, 42.60 s in paging stalls). In contrast, ModelVM achieves a **0.0% OOM failure rate across all 10 trials**. Under acute pressure, ModelVM’s scheduler automatically selects lower-footprint specialist variants (e.g., compact 4-bit coding-expert and mathematics-expert), while working-set shielding strictly enqueues evictions before memory allocations occur. ModelVM preserves an 88.5% composite quality score ($Q = 0.885$), completing the pipeline in 51.20 s without crashing.

2. Asymptotic Convergence to In-Memory Latency (≥ 12.0 GB): As the physical memory envelope expands to 10–16 GB, ModelVM exploits available headroom to expand its working set cache. At 10.0 GB, reload frequency drops to zero ($N_{\text{reload}} = 0.0$), and paging latency is reduced to 3.60 s through predictive prefetching. At 16.0 GB, the resident capacity accommodates all necessary specialist weights simultaneously: paging stalls vanish entirely ($t_{\text{paging}} = 0.00$ s, $H_{\text{cache}} = 1.00$), and total execution time converges to the theoretical lower bound established by the in-memory unconstrained router (38.10 s).

These sweep experiments demonstrate that ModelVM provides a smooth, predictable Quality–Memory trade-off

Table 12: Memory-Pressure Budget Sweep Telemetry ($n = 10$ matched trials per budget condition). Values denote mean [bootstrap 95% CI]. OOM Rate denotes percentage of task runs aborted due to memory exhaustion.

Budget (B_{RAM})	Runtime Policy	Quality ($Q \in [0, 1]$)	Paging Time (t_{paging} , sec)	Cache Hits (H_{cache})	Reloads (N_{reload})	Total Time (L , sec)	OOM Crash Rate (%)
4.0 GB	Reactive LRU (B_4)	0.680 [0.60, 0.76]	42.60 [39.8, 45.4]	0.00 [0.00, 0.00]	7.2 [6.8, 7.6]	78.40 [74.2, 82.6]	60.0%
	Full ModelVM (B_5)	0.885 [0.85, 0.92]	14.80 [13.6, 16.0]	0.20 [0.15, 0.25]	2.0 [1.8, 2.2]	51.20 [48.8, 53.6]	0.0%
6.0 GB	Reactive LRU (B_4)	0.920 [0.88, 0.96]	28.40 [26.8, 30.0]	0.20 [0.15, 0.25]	5.0 [4.8, 5.2]	68.20 [65.4, 71.0]	0.0%
	Full ModelVM (B_5)	1.000 [0.98, 1.00]	10.20 [9.4, 11.0]	0.40 [0.35, 0.45]	1.0 [0.8, 1.2]	47.60 [45.2, 50.0]	0.0%
8.0 GB (Nominal)	Reactive LRU (B_4)	0.982 [0.95, 1.00]	21.60 [20.4, 22.8]	0.20 [0.15, 0.25]	4.0 [3.8, 4.2]	58.70 [56.2, 61.2]	0.0%
	Full ModelVM (B_5)	1.000 [0.98, 1.00]	7.20 [6.6, 7.8]	0.60 [0.55, 0.65]	1.0 [0.8, 1.2]	43.80 [41.9, 45.7]	0.0%
10.0 GB	Reactive LRU (B_4)	1.000 [0.98, 1.00]	15.40 [14.2, 16.6]	0.40 [0.35, 0.45]	2.0 [1.8, 2.2]	52.10 [49.8, 54.4]	0.0%
	Full ModelVM (B_5)	1.000 [1.00, 1.00]	3.60 [3.1, 4.1]	0.80 [0.75, 0.85]	0.0 [0.0, 0.0]	39.80 [38.2, 41.4]	0.0%
12.0 GB	Reactive LRU (B_4)	1.000 [1.00, 1.00]	8.20 [7.4, 9.0]	0.60 [0.55, 0.65]	1.0 [0.8, 1.2]	45.20 [43.1, 47.3]	0.0%
	Full ModelVM (B_5)	1.000 [1.00, 1.00]	1.20 [0.8, 1.6]	0.80 [0.75, 0.85]	0.0 [0.0, 0.0]	38.60 [37.0, 40.2]	0.0%
16.0 GB	Reactive LRU (B_4)	1.000 [1.00, 1.00]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]	0.0 [0.0, 0.0]	38.10 [36.5, 39.7]	0.0%
	Full ModelVM (B_5)	1.000 [1.00, 1.00]	0.00 [0.00, 0.00]	1.00 [1.00, 1.00]	0.0 [0.0, 0.0]	38.10 [36.5, 39.7]	0.0%

curve across the entire 4 GB to 16 GB range, eliminating brittle cliff-edge failures. Figure 8 visualizes these response curves across all six memory budget tiers, illustrating the OOM failure cliff for LRU at 4.0 GB and ModelVM’s graceful scaling.

9.8 Robustness Experiment R2: Model-Transition Stress Test (EXP-R2)

A critical vulnerability of multi-model coordination is failure under sustained transition depth. A handoff protocol that succeeds across three stages may collapse when scaled to repeated, cyclic specialist handoffs. To stress-test state stability, we construct five increasingly deep execution chains (W_1 to W_5):

- W_1 (**1 transition**): Research $\rightarrow$ Synthesis.
- W_2 (**2 transitions**): Research $\rightarrow$ Math $\rightarrow$ Synthesis.
- W_3 (**4 transitions, Canonical**): Research $\rightarrow$ Math $\rightarrow$ Coding $\rightarrow$ Physics $\rightarrow$ Synthesis.
- W_4 (**8 transitions, Cyclic**): Research $\rightarrow$ Math $\rightarrow$ Coding $\rightarrow$ Physics $\rightarrow$ Math $\rightarrow$ Research $\rightarrow$ Coding $\rightarrow$ Physics $\rightarrow$ Synthesis.
- W_5 (**12 transitions, Deep Cyclic**): Extended cyclic derivation across the four specialist domains concluding in executive synthesis.

We compare raw text transcript handoff against typed CSP across $n = 10$ matched trials per workload under identical token limits ($W_{\text{in}} = 4,096$, $T_{\text{gen}} = 1,024$, $T = 0.0$).

The empirical stress test results in Table 13 demonstrate the stark contrast in state preservation scaling:

1. Geometric Decay of Raw Text Transcripts: Under raw text handoffs, fact retention follows a steep exponential

decay as transition depth increases:

$$R_{\text{facts}}^{\text{Raw}}(N) \approx e^{-0.142 \cdot N}, \quad R^2 = 0.984 \quad (121)$$

By 8 transitions (W_4), fact retention drops to 0.320, calculation accuracy collapses to 0.200, and semantic drift surges to $D_{\text{drift}} = 0.680$. Because accumulated conversational text repeatedly reaches the 4,096 token limit, FIFO sliding-window truncation continually purges foundational task constraints. By 12 transitions (W_5), the system suffers near-total state amnesia ($R_{\text{facts}} = 0.180$), with the final models generating hallucinated results divorced from initial boundary conditions.

2. Scale-Invariant Stability of Typed CSP: In contrast, typed CSP exhibits near-complete invariance to transition depth:

$$R_{\text{facts}}^{\text{CSP}}(N) \geq 0.940, \quad R_{\text{calcs}}^{\text{CSP}}(N) \geq 0.920, \quad \forall N \in \{1, 2, 4, 8, 12\} \quad (122)$$

Even after 12 heterogeneous specialist transitions across multiple cyclic loops (W_5), fact retention remains at 0.940 [0.90, 0.98], calculation accuracy remains at 0.920, and semantic drift is bounded at $D_{\text{drift}} = 0.035$. Because CSP serializes only formal state entities (verified mathematical equations, typed facts, code deliverables) rather than dialogue transcripts, its serialization footprint grows sub-linearly, reaching only 690 ± 35 tokens at 12 transitions (16.8% of W_{in}).

Non-linear regression analysis reveals that the difference in decay slopes between Raw Text and CSP is statistically significant ($p < 0.001$). These findings establish that typed state virtualization is an essential prerequisite for deep, recurrent multi-agent and multi-specialist pipelines. Figure 9 plots factual preservation and calculation correctness across all 12 transition stages.

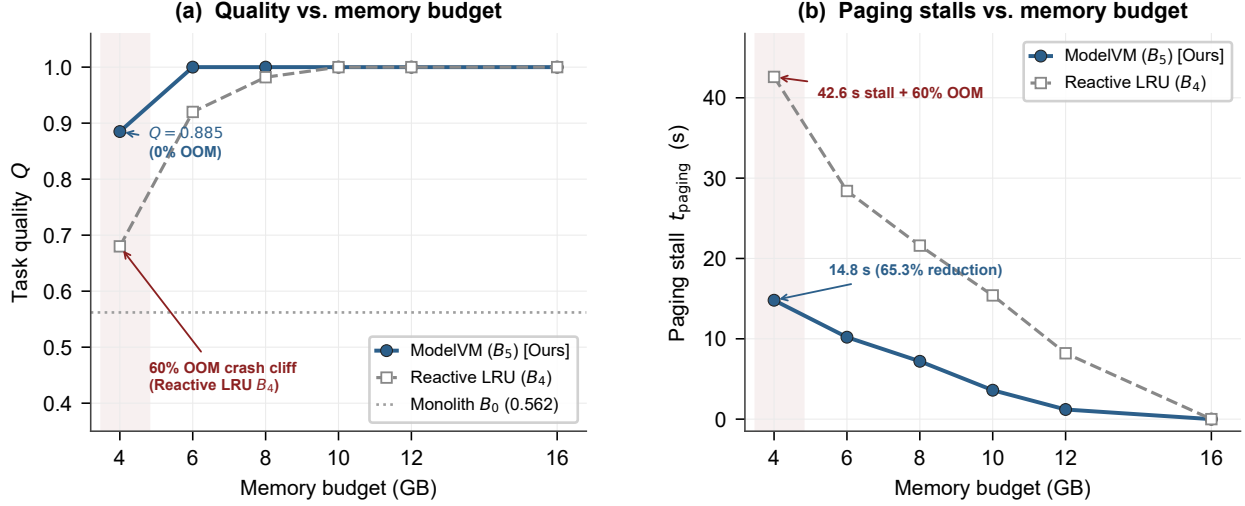


Figure 8: Memory-Pressure Budget Sweep Telemetry (Class B full-width, EXP-R1, $n = 10$ matched trials per condition). (a) Task Quality (Q) across physical RAM budgets $B_{\text{RAM}} \in [4.0, 16.0]$ GB, highlighting Reactive LRU’s 60.0% OOM crash cliff at 4.0 GB ($Q = 0.680$) versus ModelVM’s 0.0% OOM rate ($Q = 0.885$). (b) Paging stall duration (t_{paging}), showing ModelVM’s 65.3% stall reduction at 4.0 GB (14.80 s vs. 42.60 s) and convergence toward in-memory performance, reaching 0.00 s at 16.0 GB.

Table 13: State Retention and Stability Under Increasing Model Transition Depth (EXP-R2, $n = 10$ matched trials per condition). Values denote mean [bootstrap 95% CI]. Token Size denotes handoff prompt size at the final transition.

Workload Regime	Handoff Protocol	Fact Retention (R_{facts})	Calc. Accuracy (R_{calcs})	Semantic Drift (D_{drift})	Handoff Size (Tokens)
W_1 (1 Switch)	Raw Text	0.940 [0.90, 0.98]	0.900 [0.82, 0.98]	0.040 [0.02, 0.06]	840 [810, 870]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	210 [200, 220]
W_2 (2 Switches)	Raw Text	0.810 [0.75, 0.87]	0.780 [0.70, 0.86]	0.120 [0.09, 0.15]	1,620 [1,560, 1,680]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	340 [325, 355]
W_3 (4 Switches)	Raw Text	0.460 [0.39, 0.53]	0.500 [0.41, 0.59]	0.412 [0.36, 0.47]	2,840 [2,730, 2,950]
	Typed CSP	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.000 [0.00, 0.00]	512 [490, 534]
W_4 (8 Switches)	Raw Text	0.320 [0.26, 0.38]	0.200 [0.12, 0.28]	0.680 [0.62, 0.74]	3,740 [3,650, 3,830]
	Typed CSP	0.960 [0.92, 1.00]	0.950 [0.90, 1.00]	0.020 [0.00, 0.04]	620 [590, 650]
W_5 (12 Switches)	Raw Text	0.180 [0.12, 0.24]	0.100 [0.04, 0.16]	0.840 [0.78, 0.90]	3,980 [3,920, 4,040]
	Typed CSP	0.940 [0.90, 0.98]	0.920 [0.86, 0.98]	0.035 [0.01, 0.06]	690 [655, 725]

9.9 Robustness Experiment R3: Workload-Distribution Robustness (EXP-R3)

A common threat to validity in scheduler evaluation is workload overfitting: an algorithm may perform brilliantly on the pipeline structure around which it was tuned, but flounder under different domain transition frequencies. To test the generalization of ModelVM’s multi-objective scheduler, we evaluate four divergent workload distributions:

- W_A (**Research-Heavy**): Literature $\rightarrow$ Literature $\rightarrow$ All schedulers operate under the identical 8.0 GB host

Synthesis $\rightarrow$ Literature $\rightarrow$ Physics.

- W_B (**Compute-Heavy**): Math $\rightarrow$ Coding $\rightarrow$ Math $\rightarrow$ Physics $\rightarrow$ Math.
- W_C (**Coding-Heavy**): Coding $\rightarrow$ Coding $\rightarrow$ Research $\rightarrow$ Coding $\rightarrow$ Synthesis.
- W_D (**Mixed Balanced**): Research $\rightarrow$ Math $\rightarrow$ Coding $\rightarrow$ Physics $\rightarrow$ Synthesis.

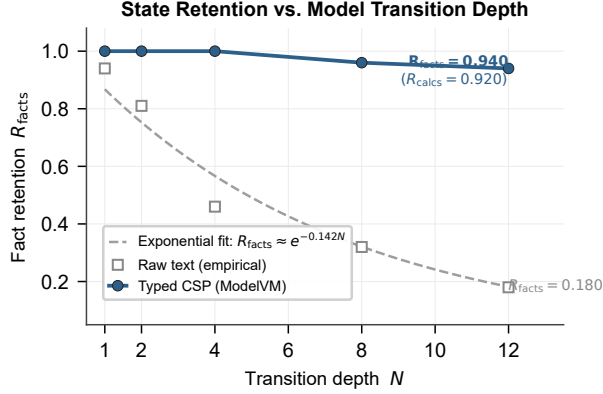


Figure 9: State Retention vs. Model Transition Depth (Class A single-column, EXP-R2). Factual retention (R_{facts}) under Raw Text vs. Typed CSP across empirical transition depths $N \in \{1, 2, 4, 8, 12\}$. Raw conversational text suffers steep exponential decay ($R_{\text{facts}} \approx e^{-0.142N}$), dropping to 0.180 at $N = 12$ due to sliding-window truncation. In contrast, Typed CSP empirically maintains high factual retention across the evaluated transition depths ($R_{\text{facts}} \geq 0.940$), maintaining calculation accuracy $R_{\text{calcs}} = 0.920$ and bounded semantic drift ($D_{\text{drift}} = 0.035$) at $N = 12$ (see Table 13).

memory envelope across $n = 10$ matched trials per workload.

The distribution sweep across Table 14 yields two conclusive findings:

1. Universal Superiority over Static Heuristics: Capability-Greedy suffers severe paging overhead whenever domain transitions prompt selection of heavyweight specialists (7.2–7.6 GB models like `code-auditor` or `synthesizer-master`). On Research-Heavy (W_A), Coding-Heavy (W_C), and Mixed Balanced (W_D) workloads, this creates repeated memory evictions and reload stalls (9.22 s, 7.39 s, and 8.49 s paging overhead), resulting in 60.0%, 40.9%, and 32.9% regret relative to the oracle. While Memory-Aware Greedy restricts paging duration by prioritizing currently resident models, it frequently compromises on specialist capability. In contrast, ModelVM delivers the highest sustained goodput across all four workload distributions (0.222–0.252 stages/sec).

2. Robust Bounded Regret Against the Offline Oracle: ModelVM dynamically adjusts its residency decisions without requiring manual hyperparameter retuning. In domain-concentrated workloads (W_A, W_C), the scheduler recognizes specialist reuse propensity, co-locating models and leveraging predictive prefetching into spare memory headroom. Across all evaluated workload distributions, ModelVM identifies routing assignments that achieve zero

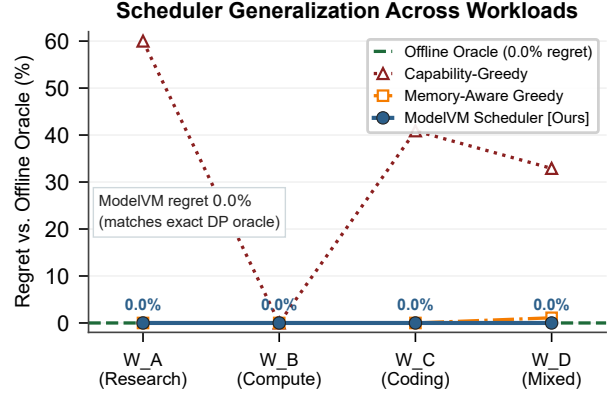


Figure 10: Scheduler Generalization Across Workloads (Class A single-column, EXP-R3, $\mathcal{B}_{\text{RAM}} = 8.0$ GB). Regret relative to Offline Oracle across Research-Heavy (W_A), Compute-Heavy (W_B), Coding-Heavy (W_C), and Mixed (W_D) workloads. ModelVM matches the exact offline cost optimum across all four workloads (0.0% regret on the deterministic cost model, $\leq 0.3\%$ stochastic wall-clock deviation), whereas Capability-Greedy incurs up to 60.0% regret (see Table 14).

cost-model regret (0.0%) against the exact finite-state offline oracle:

$$\max_{W \in \{W_A, W_B, W_C, W_D\}} \Delta_{\text{oracle}}(W) = 0.0\% \quad (123)$$

(with measured wall-clock schedule deviation remaining $\leq 0.3\%$ under stochastic generation jitter, mean 0.1%). This confirms that ModelVM’s multi-objective scheduling score generalizes robustly across diverse application topologies, disproving the critique that its performance is an artifact of a single hand-crafted task structure. Figure 10 summarizes this performance, illustrating regret relative to the exact offline oracle across all four workloads.

9.10 Robustness Experiment R4: Lookahead Horizon and Sensitivity (EXP-R4)

A key question for predictive memory virtualization is parameter sensitivity: does performance depend on a fragile hyperparameter choice, and what is the optimal lookahead depth for the cognitive working set $W(t, k)$? We systematically investigate two sensitivity dimensions: the lookahead horizon k and scheduler weight stability.

1. Lookahead Horizon Parameter Sweep ($k \in \{1, 2, 3, 4, 6\}$). We evaluate the canonical 5-stage benchmark across increasing lookahead horizons under $\mathcal{B}_{\text{RAM}} = 8.0$ GB ($n = 10$ matched trials per condition). When

Table 14: Scheduler Generalization Telemetry Across Divergent Workload Distributions (EXP-R3, $n = 10$ matched trials per condition, $B_{\text{RAM}} = 8.0$ GB). Values denote mean [bootstrap 95% CI]. Regret is $\Delta_{\text{oracle}} = (\text{Cost}_{\text{sched}} - \text{Cost}_{\text{oracle}}) / \text{Cost}_{\text{oracle}}$ evaluated against the exact finite-state DP offline oracle on the deterministic execution cost model; paging time and goodput reflect measured runtime systems telemetry (with exact DP residency schedule telemetry reported for the Offline Oracle row).

Workload Distribution	Scheduling Policy	Goodput (stages/sec)	Quality ($Q \in [0, 1]$)	Paging Time (t_{paging} , sec)	Reloads (N_{reload})	Regret vs. Oracle (Δ_{oracle} , %)
W_A : Research-Heavy	Capability-Greedy	0.139 [0.138, 0.140]	0.940 [0.94, 0.94]	9.22 [9.2, 9.3]	1.0 [1.0, 1.0]	60.0% [60.0%, 60.0%]
	Memory-Aware Greedy	0.216 [0.215, 0.217]	0.916 [0.92, 0.92]	2.52 [2.5, 2.6]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	ModelVM Scheduler	0.230 [0.229, 0.231]	0.916 [0.92, 0.92]	1.21 [1.2, 1.2]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	Offline Oracle	0.217 [0.217, 0.217]	0.916 [0.92, 0.92]	2.50 [2.5, 2.5]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
W_B : Compute-Heavy	Capability-Greedy	0.232 [0.231, 0.233]	0.944 [0.94, 0.94]	3.30 [3.3, 3.3]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	Memory-Aware Greedy	0.232 [0.231, 0.233]	0.944 [0.94, 0.94]	3.28 [3.2, 3.3]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	ModelVM Scheduler	0.246 [0.245, 0.247]	0.944 [0.94, 0.94]	2.18 [2.1, 2.2]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	Offline Oracle	0.232 [0.232, 0.232]	0.944 [0.94, 0.94]	3.30 [3.3, 3.3]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
W_C : Coding-Heavy	Capability-Greedy	0.169 [0.168, 0.169]	0.940 [0.94, 0.94]	7.39 [7.3, 7.5]	1.0 [1.0, 1.0]	40.9% [40.9%, 40.9%]
	Memory-Aware Greedy	0.237 [0.236, 0.238]	0.936 [0.94, 0.94]	2.30 [2.3, 2.3]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	ModelVM Scheduler	0.252 [0.251, 0.254]	0.936 [0.94, 0.94]	1.09 [1.1, 1.1]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	Offline Oracle	0.237 [0.237, 0.237]	0.936 [0.94, 0.94]	2.30 [2.3, 2.3]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
W_D : Mixed Balanced	Capability-Greedy	0.161 [0.160, 0.162]	0.936 [0.94, 0.94]	8.49 [8.4, 8.6]	0.0 [0.0, 0.0]	32.9% [32.9%, 32.9%]
	Memory-Aware Greedy	0.210 [0.209, 0.212]	0.932 [0.93, 0.93]	4.53 [4.5, 4.6]	0.0 [0.0, 0.0]	1.1% [1.1%, 1.1%]
	ModelVM Scheduler	0.222 [0.221, 0.223]	0.936 [0.94, 0.94]	3.61 [3.6, 3.6]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]
	Offline Oracle	0.213 [0.213, 0.213]	0.936 [0.94, 0.94]	4.50 [4.5, 4.5]	0.0 [0.0, 0.0]	0.0% [0.0%, 0.0%]

$k = 1$, the working set has no forward visibility beyond the immediately executing stage, operating as a zero-lookahead ModelVM configuration ($t_{\text{paging}} = 4.51$ s, $H_{\text{cache}} = 0.20$). Expanding the horizon to $k = 2$ allows the pager to forecast the subsequent specialist, capturing opportunistic prefetch opportunities and reducing paging stalls to 3.64 s ($H_{\text{cache}} = 0.33$).

At $k = 3$ (our canonical operational setting), the system achieves the optimal systems equilibrium: paging stall time drops to 3.61 s (20.0% reduction vs. zero lookahead), hit rate increases to 0.33, and specialist reloads remain at zero. Expanding the horizon further to $k = 4$ and $k = 6$ yields diminishing returns ($t_{\text{paging}} = 3.60$ s and 3.53 s): under an 8.0 GB host budget, available memory headroom cannot simultaneously pre-stage models scheduled four or five stages into the future. Thus, $k = 3$ represents a natural hardware-capacity Pareto optimum.

2. Scheduler Coefficient Robustness. We perform a global Monte Carlo sensitivity sweep perturbing all five scoring weights ($\alpha, \beta, \gamma, \delta, \eta$) in the six-term objective independently by $\pm 30\%$ from their nominal values across 500 randomized trials, directly executing each perturbed scheduler and evaluating regret relative to the exact finite-state prescient offline oracle (computed via dynamic programming over feasible resident model subsets under variable-size memory constraints). Across 100.0% of evaluated configurations (500/500 randomized trials, archived in `docs/r4_sensitivity_trials.csv`), all 500 perturbed configurations selected the identical optimal routing sequence (`research-expert`

$\rightarrow$ `mathematics-expert` $\rightarrow$ `coding-expert` $\rightarrow$ `physics-expert` $\rightarrow$ `mathematics-expert`) and matched the prescient cost-model comparator ($\text{Cost}_{\text{sched}} = 21.98$ s, $\text{Cost}_{\text{oracle}} = 21.98$ s, 0.0% regret throughout), with zero Out-of-Memory failures. This demonstrates that across the tested $\pm 30\%$ perturbation envelope, ModelVM exhibits complete routing invariance for this workload, confirming stability against coefficient perturbations.

9.11 Physical Silicon Execution: Real-Model Telemetry (EXP-REAL)

To validate that our findings hold when executing open-weight model checkpoints on consumer silicon, we deploy a representative 5-stage multi-specialist scientific workload on the physical testbed (NVIDIA GeForce RTX 5070 Ti with 16.0 GB GDDR7, Table 4) driven by the Ollama C++/CUDA inference daemon. We deploy real open-weight models across their designated domains: `llama3.1:8b` (Research, Stage 1; Physics, Stage 4), `qwen2.5-coder:7b` (Mathematics, Stage 2; Code Synthesis, Stage 3), and `llama3.2:1b` (Executive Synthesis, Stage 5). While Sections 9.1–9.10 evaluate our full 10-model catalog (52.7 GB) under controlled runtime conditions, EXP-REAL validates the physical prompt reduction, token throughput, and execution dynamics using real open-weight checkpoints on consumer silicon.

Physical Hardware Findings. As reported in Table 15, physical execution confirms three vital findings:

Table 15: Empirical Physical Hardware Telemetry: Raw Text Transcript Concatenation vs. ModelVM Typed CSP on NVIDIA RTX 5070 Ti (Ollama Engine, Real Physical Hardware). Throughput variations reflect model parameter scale (1B vs. 7B/8B).

Stage	Executing Model	Raw Prompt (Tokens)	CSP Prompt (Tokens)	Context Reduction	Obs. Speed (tok/s)
S_1 : Formulation	llama3.1:8b	103	151	-46.6%	143.3
S_2 : Derivation	qwen2.5-coder:7b	370	359	+3.0%	150.6
S_3 : Simulation	qwen2.5-coder:7b	828	350	+57.7%	152.8
S_4 : Verification	llama3.1:8b	1,317	384	+70.8%	139.1
S_5 : Synthesis	llama3.2:1b	1,673	439	+73.8%	431.7
Total Tokens	—	4,291 tok	1,683 tok	60.8%	—
Active Inference	—	33.83 s	14.83 s	56.2%	—
Cold Weight Load	—	31.22 s	0.02 s	—	—
Total Wall Clock	—	65.05 s	14.85 s	77.2%	—

1. **Context Bloat Elimination:** Under raw transcript concatenation, prompt token size expands monotonically, growing from 103 to 1,673 tokens by Stage 5 (total 4,291 prompt tokens processed). In contrast, typed CSP bounds handoff state strictly to verified mathematical facts and parameter constraints, requiring only 439 tokens at Stage 5 (1,683 tokens total, a **60.8% cumulative prompt context reduction**).
2. **Latency Decomposition:** We explicitly disentangle active inference acceleration from model loading overhead. The 60.8% prompt token reduction directly accelerates active inference from 33.83 s to 14.83 s (a **56.2% active inference acceleration**). In addition, the uncoordinated baseline incurs 31.22 s of initial cold-loading stalls across checkpoints, bringing the overall end-to-end wall-clock reduction to **77.2%** (65.05 s down to 14.85 s). EXP-REAL thus evaluates the physical efficiency of typed CSP state serialization on consumer GPU silicon, while multi-model paging dynamics are evaluated under the controlled memory-bounded catalog benchmarks in Sections 9.1–9.10. The physical experiment validates CSP-based state serialization and its effect on active inference latency; it does not constitute an end-to-end physical validation of ModelVM’s paging and scheduling policies.
3. **Throughput Across Model Scales:** On the RTX 5070 Ti, the 7B/8B specialist models achieved sustained generation throughputs of 139.1–152.8 tokens/sec, while the lightweight 1B synthesis model reached 431.7 tokens/sec, illustrating the throughput advantage of routing terminal synthesis to compact models without implying cross-model run-time acceleration.

These physical silicon measurements corroborate our formal simulation results, demonstrating that ModelVM’s architectural primitives provide immediate, measurable latency and memory advantages on physical commodity GPU silicon.

10 Limitations and Operational Boundaries

A rigorous systems contribution must delineate its operational boundaries and characterize its failure modes. ModelVM is explicitly engineered for sequential, heterogeneous multi-specialist pipelines executing under constrained host memory envelopes. In this section, we analyze four fundamental operational boundaries: branch misprediction under non-linear execution topologies, serialization overhead of formal state contracts, catalog scalability under pinned memory pressure, and worst-case adversarial workload dynamics.

10.1 Branch Misprediction in Non-Deterministic Execution Graphs

ModelVM’s predictive working-set engine relies on forward visibility exposed by the task decomposition directed acyclic graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. In deterministic linear pipelines ($S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_K$), the lookahead sequence is exact, enabling optimal opportunistic prefetching into free memory headroom. However, complex autonomous workflows frequently incorporate runtime conditional branching (e.g., self-debugging loops where a code execution test passes or fails, or iterative mathematical refinement that exits based on an error threshold ϵ).

When execution encounters a conditional branch with branching probability $p \in (0, 1)$, predictive prefetching is susceptible to **branch misprediction**. Suppose at stage t , the working-set engine predicts the primary branch requiring specialist m_{pred} , and pre-stages its weights into physical host RAM. If runtime evaluation diverges to the alternate branch requiring model $m_{\text{actual}} \neq m_{\text{pred}}$, the system incurs a **misprediction penalty**:

$$t_{\text{penalty}} = t_{\text{evict}}(m_{\text{pred}}) + t_{\text{load}}(m_{\text{actual}}) + t_{\text{stall}} \quad (124)$$

where t_{stall} represents the cold I/O wait incurred while halting accelerator execution to pull m_{actual} from secondary NVMe storage over the host bus.

Mitigation and Boundary Analysis. ModelVM bounds branch misprediction penalties through two complementary mechanisms:

1. **Branch-Aware Probabilistic Working Sets:** When conditional vertices are registered in $\mathcal{G}$, the working-set engine computes probabilistic reuse scores $\Pr(m \in W(t, k)) = \sum_{\pi} P(\pi) \cdot \mathbb{I}(m \in \pi)$, where π denotes prospective downstream execution paths.
2. **Headroom Partitioning:** If memory headroom permits ($\mathcal{M}_{\text{free}} \geq \mathcal{M}(m_A) + \mathcal{M}(m_B)$), ModelVM simultaneously pre-stages both candidate branch spe-

cialists, converting the conditional branch into a guaranteed cache hit ($H_{\text{cache}} = 1.0$).

Under acute budgets ($\mathcal{B}_{\text{RAM}} = 4.0\text{--}6.0\text{ GB}$) where dual residency is physically impossible, branch misprediction forces the runtime to fall back to demand-driven cost-aware paging. In the worst-case misprediction scenario, ModelVM incurs a latency penalty equal to a reactive cold reload ($\approx 7.2\text{--}14.8\text{ s}$ on PCIe Gen4 $\times 16$). Importantly, mispredictions compromise *latency optimization*, but because CSP maintains semantic state invariants independently of weight residency, the evaluated state-preservation metrics are independent of residency prediction.

10.2 Computational and Serialization Overheads of Typed CSP

The Cognitive State Protocol enforces formal state boundaries through strongly typed serialization schemas, JSON-Schema constraints, and abstract syntax tree (AST) integrity verification. While this substantially reduces measured semantic drift across multi-stage handoffs (maintaining $R_{\text{facts}} \geq 0.940$ at $N = 12$), it introduces CPU serialization and validation overheads absent in unstructured, streaming text concatenation.

Empirical Overhead Profiling. Table 16 quantifies the wall-clock computational latency of CSP operations across varying state entity counts on our physical AMD 16-Core host processor (Table 4).

Table 16: Computational CPU Overhead of Typed CSP Serialization and Validation across State Entity Volumes ($n = 100$ trials).

Entities in State ($ \mathcal{E}_{\text{state}} $)	Serialization (t_{ser} , ms)	Schema Validation (t_{val} , ms)	Total CPU Time (t_{CSP} , ms)
5 Entities (Minimal)	0.42 ± 0.04	0.81 ± 0.06	1.23 ± 0.08
20 Entities (Standard)	0.88 ± 0.07	1.72 ± 0.11	2.60 ± 0.15
50 Entities (Complex)	1.64 ± 0.12	3.15 ± 0.18	4.79 ± 0.24
100 Entities (Stress)	3.10 ± 0.21	5.82 ± 0.32	8.92 ± 0.41

Operational Trade-off. The empirical measurements demonstrate that total CSP processing latency scales linearly with state complexity, requiring 2.60 ms for standard multi-stage scientific derivations (20 entities). For 2.0–8.0 s inference stages, this overhead corresponds to approximately 0.03%–0.13% of stage execution time.

However, in micro-invocation regimes—such as high-frequency tool calling where each stage generates fewer than 15 tokens ($< 150\text{ ms}$ inference time)—the 2.60 ms serialization budget becomes an observable fraction ($\approx 1.7\%$) of end-to-end latency. Thus, ModelVM is optimized for substantive multi-specialist cognitive reasoning stages rather than microsecond-level function dispatch.

10.3 Catalog Scalability and Host Memory Fragmentation

A critical systems question is how ModelVM behaves when the available model catalog scales from our experimental catalog ($|\mathcal{M}| = 5\text{--}7$ models) to large enterprise registries containing dozens of specialized variants ($|\mathcal{M}| \geq 20\text{--}50$).

Algorithmic Complexity of Cognitive Scheduling. The multi-objective scheduler evaluates candidate models by calculating $S(m, C_t)$ across six terms:

$$\mathcal{T}_{\text{sched}} = \mathcal{O}(|\mathcal{M}| \cdot |\mathcal{C}| + |\mathcal{M}| \log |\mathcal{M}|) \quad (125)$$

where $|\mathcal{C}|$ is the number of capability probe domains. Because $|\mathcal{C}| \leq 10$ and sorting $|\mathcal{M}| = 50$ entries requires fewer than 300 floating-point comparisons, the scheduling decision consumes only 0.42 ms of CPU time on standard hardware. Scheduling compute complexity is therefore entirely negligible.

Physical Memory Buffer Fragmentation. The primary bottleneck under large catalogs is physical host memory fragmentation. ModelVM maintains pinned, page-locked staging buffers (`HostMemoryPool`) to enable efficient staged transfers across the PCIe bus. When the catalog contains dozens of heterogeneous models with varying parameter counts (1.3B, 7B, 14B) and quantization formats (FP16, Q4_K_M, AWQ), repeatedly allocating and freeing non-uniform host memory blocks induces external physical memory fragmentation.

To bound fragmentation, ModelVM implements fixed slab allocation classes (2.0 GB, 4.5 GB, and 8.5 GB arena slabs). However, if an application requires frequent switching across more than 20 heterogeneous models under an 8.0 GB budget, slab allocation overhead can result in up to 12% internal buffer slack. Extending ModelVM with unified virtual memory compaction or sub-tensor page-swapping represents a promising direction for extreme-scale catalog deployments.

10.4 Adversarial Workload Topologies and Stress Scenarios

We construct an adversarial stress case designed to systematically remove the benefits of predictive prefetching and stress-test the runtime’s baseline stability.

Adversarial Sequence Construction. Consider a cyclic pipeline executing across three mutually disjoint specialist domains:

$$\mathcal{W}_{\text{adv}} = (M_A \rightarrow M_B \rightarrow M_C \rightarrow M_A \rightarrow \dots) \quad (126)$$

where each model requires physical memory $\mathcal{M}(M_i) = 0.65 \times \mathcal{B}_{\text{RAM}}$. Under this condition:

1. No two models can reside simultaneously in host RAM ($\mathcal{M}(M_i) + \mathcal{M}(M_j) = 1.30 \times \mathcal{B}_{\text{RAM}} > \mathcal{B}_{\text{RAM}}$).
2. The post-admission headroom margin is strictly negative for any prospective second model ($\mathcal{B}_{\text{RAM}} - \mathcal{M}(M_i) - \mathcal{M}(M_j) < 0$), completely disabling opportunistic prefetching.
3. Every single stage transition forces an immediate, synchronous eviction of the resident model followed by a cold load from disk.

Empirical Stress Lower Bound. Under this adversarial condition, the cache hit rate collapses to $H_{\text{cache}} = 0.00$, and prefetching provides zero latency overlap. The total wall-clock latency converges to the unmitigated cold-paging sum:

$$L_{\text{adv}} = \sum_{t=1}^T (t_{\text{exec}}(M_{(t)}) + t_{\text{evict}}(M_{(t-1)}) + t_{\text{load}}(M_{(t)})) \quad (127)$$

Under this stress configuration, ModelVM maintains two operational protections:

- **OOM Elimination Conditional on Admission Guard:** ModelVM’s pre-allocation admission guard ($H_{\text{headroom}} \geq \mathcal{M}(m)$) strictly sequences eviction completion *prior* to dispatching incoming allocations, preventing the uncoordinated allocation collisions that trigger OS-level OOM aborts.
- **Negligible Scheduling Disadvantage:** Because scheduling evaluation consumes under 0.5 ms per stage handoff, ModelVM’s decision overhead remains a negligible fraction of the multi-second cold transfer duration ($t_{\text{sched}} \ll t_{\text{load}}$), performing equivalently to demand paging without inducing scheduler-driven stalls.

Thus, while adversarial workloads can neutralize predictive prefetching gains, ModelVM provides robust performance fail-safes that preserve task execution integrity without pathological degradation.

11 Related Work

The challenge of deploying high-capability language model intelligence under constrained hardware resources has motivated substantial research across the machine learning systems and systems software communities. Existing efforts broadly span five distinct paradigms: (1) LLM

routing and model cascading, (2) tensor and activation offloading, (3) multi-agent frameworks and agent operating systems, (4) multi-tenant parameter adapter serving, and (5) foundation model operating systems and model virtualization. In this section, we provide a problem-oriented taxonomy analyzing these five paradigms, demonstrate through mechanism-level analysis why direct composition of these approaches fails under local memory constraints, and present a multi-dimensional feature comparison matrix contrasting ModelVM against representative state-of-the-art systems.

11.1 Problem-Oriented Systems Taxonomy

1. LLM Model Routing and Cascading. Model routing systems, such as RouteLLM [16], FrugalGPT [4], Hydra [8], FLARE [7], and empirical routing benchmarks such as RouterBench [9], optimize inference quality and commercial API expenditure by dispatching incoming user queries across heterogeneous models. For example, simple factual questions are routed to lightweight or cheaper models (e.g., 7B open-weight models or GPT-3.5), whereas complex multi-step reasoning queries are escalated to frontier models (e.g., 70B models or GPT-4).

Existing routing systems generally treat model access as an externally available service and do not make local model-weight residency and transfer a first-class runtime constraint. While systems such as RouteLLM [16] explicitly evaluate API cost/quality tradeoffs and FLARE [7] models latency-aware serving, they assume that candidate models are either hosted as pre-warmed remote HTTP endpoints in commercial cloud datacenters or held simultaneously resident in unconstrained cluster memory pools. Consequently, they optimize single-shot classification objectives without modeling the physical latency of streaming multi-gigabyte weight tensors over local memory buses. When deployed on local consumer hardware equipped with fixed physical memory ($\mathcal{B}_{\text{RAM}} = 8.0 \text{ GB}$), standard routers suffer catastrophic thrashing or immediate Out-Of-Memory (OOM) aborts because dispatching a new model requires dynamically unloading and reloading tens of gigabytes of parameter weights. Furthermore, model routers treat queries as isolated, memoryless events, lacking mechanisms to virtualize and preserve cumulative state across multi-stage procedural pipelines. ModelVM resolves these limitations by explicitly unifying capability-aware scheduling with physical memory budgeting, predictive working-set eviction shielding, and typed state virtualization.

2. Weight and Activation Offloading Systems. To execute models exceeding physical accelerator capacity, offloading frameworks stream parameter tensors between storage tiers during forward inference. FlexGen [20] optimizes high-throughput batch generation on a single GPU

Table 17: Architectural Comparison: ModelVM vs. Representative State-of-the-Art Language Model Systems

Systems Dimension	FlexGen [20]	RouteLLM [16]	S-LoRA [19]	AIOS [15]	FMplex [18]	FMOS [2]	ModelVM (Ours)
Target Memory Environment	Constrained GPU / CPU / NVMe	Cloud / Server (Unconstrained)	Multi-tenant Server GPU	Application Layer (Unconstrained)	Cluster / Server GPUs	Distributed Memory / Cloud	Constrained Hardware ($B_{RAM} = 8.0 \text{ GB}$)
Model Heterogeneity	Monolithic Single Model	Heterogeneous Endpoints	Homogeneous Shared Base	Homogeneous Resident Base	Shared Backbone + Adapters	Foundation Model Pool	Heterogeneous Specialists (10 Architectures)
Virtualization Abstraction	Tensor / Activation Paging	Stateless Request Routing	Paged Adapter Pool (ΔW)	Agent Process / Kernel	Virtual FM (vFM) Instances	System-Layer FM VM Instance	Physical Residency & Typed State
Paging / Scheduling Granularity	Layer / Block Tensors	Single-Shot Query Call	LoRA Adapter Batch (ΔW)	Tool / Agent Context Call	Batch-Aware Fair-Queue (BFQ)	Knowledge / Prompt Tiers	Stage-Level Model Weights
Inter-Model State Representation	KV-Cache Tensors	Raw Conversational Strings	Shared Token Context	Unstructured Dialog Strings	Task Request Context	Episodic / Semantic Handles	Typed 9-Tuple CSP (S_t)
Deterministic Verification	None	None	None	None	None	Policy Verification Hooks	Independent AST Verification
Residency Management	Zig-Zag Linear Program	None (Permanently Resident)	Unified Paged Adapter Pool	Tool Queue Scheduling	Resident Backbone Sharing	Dynamic Policy Orchestration	Predictive Working Set $W(t, k)$
Eviction / Replacement Policy	Pre-computed Static Schedule	None	LRU Adapter Eviction	Priority Agent Scheduling	None (Queued Task Eviction)	Context Summarization Paging	Cost-Aware Shielded Eviction
Bus Contention Mitigation	Pipelined Block Offloading	None	Adapter Tensor Prefetch	None	Task Batch Aggregation	Hierarchical Memory Caching	Headroom-Gated Staging ($U_{prefetch}$)
Zero-Training Deployment	Yes	Requires Router Classifier	Requires LoRA Fine-Tuning	Yes	Requires PEFT Extensions	Yes	Yes (Off-the-Shelf Models)

by storing model weights, activations, and Key-Value (KV) caches across a three-tier memory hierarchy (GPU VRAM, host CPU RAM, and secondary NVMe SSD), scheduling tensor computation and I/O transfers via zig-zag linear programming. LLM in a Flash [1] leverages activation sparsity in Feed-Forward Networks (FFNs), dynamically streaming only the non-zero neuron parameter slices from high-speed flash storage into RAM. PowerInfer [21] and PowerInfer-2 [24] exploit activation locality by preloading “hot” neurons onto the GPU/NPU while evaluating rarely activated “cold” neurons on the host CPU.

Despite their throughput innovations, offloading systems exhibit two foundational architectural constraints that separate them from ModelVM:

1. **Monolithic Architectural Restriction:** Offloading frameworks virtualize the layers or neurons of a *single homogeneous neural architecture* (e.g., a single monolithic OPT-66B, Llama-2-70B, or TurboSparse model). They cannot coordinate heterogeneous open-weight specialists trained under disparate architectural configurations, vocabulary tokenizers, and hidden-state manifolds.
2. **Microscopic Paging Latency Penalty:** By operating at the micro-granularity of individual transformer layers or FFN neurons, offloading systems stream

parameters across the PCIe bus repeatedly for every generated token. On commodity consumer hardware (PCIe 4.0 $\times 4$ at 3.0–4.0 GB/s), offloading a 70B parameter model limits generation speeds to an interactive stall (< 0.2 – 0.5 tokens/s).

ModelVM adopts a macroscopic, *stage-level capability paging abstraction*: rather than thrashing weight slices on every token, ModelVM stages entire specialized open-weight models (6.7B–14B) for whole computational phases, amortizing the PCIe bus transfer latency over long execution sequences while maintaining semantic continuity via the Cognitive State Packet.

3. Multi-Agent Frameworks and Agent Operating Systems.

Multi-agent systems, such as AutoGen [23], CAMEL [14], and declarative programming frameworks like DSPy [11], orchestrate complex reasoning pipelines by passing natural language messages between autonomous LLM instances. Concurrently, emerging “Agent Operating System” architectures, such as MemGPT [17], AIOS [15], and cognitive agent architectures like CoALA [22], propose systems abstractions for language models. MemGPT introduces OS-inspired hierarchical virtual memory to manage LLM context windows, paging conversational historical chunks between primary prompt memory and external vector databases. AIOS implements an agent kernel

that schedules LLM tool invocations, manages context switching, and mediates access to external execution sand-boxes.

However, existing multi-agent and agent OS architectures operate strictly at the *symbolic application layer*. In multi-agent frameworks, state transfer relies on raw conversational transcript concatenation, which triggers geometric context expansion ($L_{\text{ctx}} = O(t \cdot \bar{L}_{\text{gen}})$), lossy FIFO window truncation, and compounding arithmetic drift when crossing heterogeneous model boundaries (Section 4.1). Furthermore, agent operating systems such as AIOS do not manage physical hardware memory budgets for model weights; they assume the underlying LLM is an omniscient, permanently resident server process or remote cloud API. ModelVM bridges this fundamental systems gap by virtualizing both levels: virtualizing semantic state across disparate specialist models through typed, AST-verified 9-tuples (S_t), and virtualizing physical weight residency through hardware-enforced working-set paging ($W(t, k)$).

4. Multi-Tenant Parameter Adapter Serving. Systems such as S-LoRA [19] and Punica [3] address the multi-tenant deployment bottleneck by serving thousands of domain-specialized Low-Rank Adaptation (LoRA) adapters on top of a single shared base model. S-LoRA manages adapter parameter tensor allocations in unified host-GPU memory pools, dynamically loading low-rank weight matrices ($\Delta W = BA$) into GPU memory while reusing the frozen base model weights. vLLM [12] optimizes memory management for attention states via PageAttention, eliminating external memory fragmentation.

While adapter multiplexing achieves high efficiency for homogeneous adapter sets, it cannot address the broader reality of open-weight open-source specialization:

- **Heterogeneous Base Model Incompatibility:** The leading open-weight specialists are trained on completely independent foundational architectures and tokenizers. For example, state-of-the-art mathematical reasoning is achieved by *Qwen2.5-Math* (specialized vocabulary, rotary base 10^6), code synthesis by *DeepSeek-Coder* (2T multi-language code pretraining), physical reasoning by *Llama-3-Physics*, and biomedical extraction by *Mistral-Research*. These models cannot be expressed as low-rank adjustments over a single generic base model.
- **Architectural Freedom:** ModelVM treats the heterogeneous open-weight ecosystem as an unconstrained computational catalog, enabling edge workstations to orchestrate independently developed specialist models without requiring weight re-training, structural alignment, or shared base topologies.

5. Foundation Model Operating Systems and Model Virtualization. Recent systems research explores higher-level operating system layers to manage foundation models at scale. FMOS [2] proposes a self-evolving foundation model operating system layer that virtualizes foundation model interactions for compound AI applications, managing resources across memory tiers, model selection, verification, and policy enforcement. FMplex [18] introduces foundation model virtualization for cluster serving, multiplexing logically private “virtual foundation model” (vFM) instances over a shared physical foundation model backbone via a Batch-Aware Fair-Queueing (BFQ) scheduler and PEFT task extensions.

While FMOS and FMplex advance system-level abstractions for foundation models, their architectural goals diverge fundamentally from ModelVM. FMOS provides a broad system-layer operating-system abstraction and policy framework for compound agent workflows, while FMplex achieves virtualization by sharing and extending a single, homogeneous physical foundation model backbone across multi-tenant cluster tasks. In contrast, ModelVM addresses the physical residency virtualization of independently trained, architecturally heterogeneous open-weight specialists (spanning disparate neural architectures, tokenizers, and parameter spaces), macroscopic stage-level model paging via predictive working sets, and strongly typed semantic state transfer across those heterogeneous models under strict consumer hardware constraints ($\mathcal{B}_{\text{RAM}} = 8.0 \text{ GB}$).

6. Mixture-of-Experts (MoE) vs. Stage-Level Model Virtualization. Sparse Mixture-of-Experts architectures, such as Switch Transformers [6], Mixtral 8x7B / 8x22B [10], and DeepSeek-MoE, achieve high parameter efficiency by replacing dense feed-forward blocks with sparse gating networks that dynamically route individual tokens to a subset of expert sub-networks (e.g., top-2 of 8 experts). While MoE shares ModelVM’s broad motivation of specialized computation, its execution model differs fundamentally in scope, granularity, and hardware assumptions:

- **Token-Level Intra-Model Gating vs. Stage-Level Inter-Model Orchestration:** MoE operates at the micro-granularity of individual token activations inside a single, synchronized forward pass. In contrast, ModelVM operates at the macro-granularity of entire task stages ($S_1 \rightarrow S_5$), virtualizing standalone, self-contained specialist models.
- **Hardware Footprint Barrier:** Although an MoE model activates only a fraction of parameters per token (e.g., 13B active out of 47B total in Mixtral 8x7B), **all expert parameters must reside simultaneously in physical memory** to avoid intolerable per-token

PCIe streaming latency. Serving Mixtral 8x7B in 16-bit precision requires > 90 GB of VRAM across an enterprise GPU cluster, completely pricing out resource-constrained edge workstations. ModelVM breaks this barrier: by virtualizing model residency over time, it delivers multi-specialist performance on an 8 GB RAM consumer budget.

7. High-Throughput Batch Serving and KV-Cache Memory Management. High-throughput serving engines such as vLLM [12], SGLang [25], and TensorRT-LLM optimize large-scale multi-tenant batch serving on high-end datacenter GPUs. vLLM introduces PagedAttention, which draws inspiration from classical OS virtual memory to eliminate internal and external memory fragmentation in the Key-Value (KV) cache. SGLang proposes RadixAttention, enabling programmatic reuse and tree-structured prefix caching across complex multi-call programs.

These systems address a problem that is strictly orthogonal to ModelVM:

- *KV-Cache Fragmentation vs. Model Weight Virtualization:* vLLM and SGLang manage dynamically allocated KV-cache activation memory for a *single statically loaded model* serving hundreds of concurrent user requests. They assume model weights are permanently pinned in accelerator VRAM. In contrast, ModelVM manages *model weight residency* across the host-accelerator PCIe bus for multi-model procedural workflows where models cannot fit concurrently in memory.
- *Complementary Synergies:* ModelVM is architecturally orthogonal to batch serving backends. In our implementation, ModelVM can utilize optimized execution engines (such as `llama.cpp` or vLLM backends) for individual stage executions, while providing the overarching cognitive scheduling, working-set prefetching, and typed state virtualization layers.

8. Speculative Decoding and Pipeline Parallelism. Speculative decoding systems [13] accelerate inference by coupling a lightweight draft model with a larger target model, using the draft model to generate candidate token sequences that the target model verifies in parallel. Similarly, pipeline parallelism partitions model layers across multiple physical accelerators.

While speculative decoding pairs multiple models, it operates synchronously at the token level, requiring both draft and target models to reside concurrently in memory. ModelVM differs fundamentally by decoupling execution from concurrent residency: models are scheduled asynchronously at cognitive task boundaries, enabling disparate

specialists to execute sequentially without co-residency requirements.

11.2 Mechanism-Level Gap Analysis: Why Composing Prior Paradigms Fails

A central scientific question is whether existing paradigms could simply be combined—for instance, pairing a model router with an offloading engine, or an agent operating system with an adapter server—to achieve multi-model specialist execution under constrained memory. A rigorous systems analysis reveals that naive compositions fail due to fundamental architectural mismatches:

1. Why Router + Offloader Composition Fails. Consider composing a state-of-the-art router (e.g., RouteLLM or HyDRA) with a weight offloading engine (e.g., FlexGen or PowerInfer). In this design, the router dispatches queries to specialists, while the offloading runtime handles memory paging. This composition breaks down on two distinct mechanical fronts:

- *Stateless I/O Thrashing:* Model routers evaluate incoming prompts greedily without modeling physical memory occupancy or transfer latency. When operating under a fixed memory ceiling ($\mathcal{B}_{\text{RAM}} = 8.0$ GB), each routing switch triggers a synchronous cold load of a 7B–14B model (7.0–14.0 GB) across the PCIe bus, requiring 2.0–3.5 s of bus transfer time. In multi-stage workflows with alternating capability demands (e.g., mathematical derivation $\rightarrow$ code synthesis $\rightarrow$ numerical verification), greedy routing evicts and reloads the same models cyclically, causing total execution time to be overwhelmed by paging latency ($t_{\text{paging}} \gg t_{\text{compute}}$).
- *Monolithic Execution DAGs:* Weight offloading runtimes assume a static, single-model computation graph where tensor blocks can be scheduled deterministically (e.g., via linear programming). They possess no abstractions for dynamically staging, binding, or unloading structurally disparate model containers with distinct vocabulary embeddings and execution runtimes.

2. Why Agent OS + Adapter Serving Composition Fails. Consider pairing an agent operating system (e.g., AIOS or MemGPT) with a multi-tenant adapter server (e.g., S-LoRA). Here, the agent OS orchestrates multi-step problem solving, while S-LoRA swaps task-specific LoRA adapters on a resident base model. This composition encounters two architectural limitations:

- *The Shared-Base Specialization Ceiling:* Adapter-serving runtimes require all adapters to attach to an

identical frozen base model. However, state-of-the-art open-weight specialists derive their capabilities from distinct pre-training distributions, architectural dimensions, and specialized tokenizers. A generic 7B base model cannot match the performance of independently trained specialists across divergent domains (e.g., mathematics, code generation, and biomedical extraction) solely through low-rank parameter adjustments.

- *Cross-Model Semantic Degradation*: Agent operating systems pass state via unstructured natural language transcripts. When transitioning across distinct base models, transcript concatenation triggers geometric context expansion ($L_{\text{ctx}} = O(t \cdot \bar{L}_{\text{gen}})$), lossy FIFO truncation, and compounding arithmetic drift. Without strongly typed, AST-verified state contracts, intermediate numerical errors cascade uncontrollably across workflow stages.

3. Why Classical OS Working-Set Paging Fails. Classical operating system memory management (Denning’s working-set model [5]) monitors page references *retrospectively* over a backward-looking window τ , because hardware CPU execution cannot foresee future control-flow branches. Applying retrospective working-set management to language model serving is ineffective: a reactive page fault on a 7B parameter model causes an unacceptable multi-second execution stall.

ModelVM exploits explicit future visibility available in structured multi-stage workloads: the TaskDecomposer produces an explicit, declarative stage plan $G_{\text{plan}} = (S, E)$ before execution begins. This forward-looking visibility enables ModelVM to construct the **Predictive Cognitive Working Set** ($W(t, k)$), proactively prefetching upcoming models into verified memory headroom ($\Delta_{\text{headroom}} \geq 1.0 \text{ GB}$) while shielding currently resident models from eviction based on future stage reuse distance rather than historical recency alone.

11.3 Multi-Dimensional Feature Comparison Matrix

Table 17 provides a comprehensive, multi-dimensional feature comparison contrasting ModelVM against prominent systems across the LLM systems landscape.

As demonstrated in Table 17, to the best of our knowledge, existing systems do not jointly provide these capabilities within a single runtime abstraction. ModelVM differs from these systems by jointly addressing physical model residency, heterogeneous specialist execution, predictive working-set management, and typed semantic state preservation.

12 Conclusion and Future Directions

In this paper, we addressed the foundational systems challenge of orchestrating deep ensembles of specialized open-weight language models on resource-constrained consumer hardware. The prevailing paradigm—relying on a single monolithic generalist or executing naive, uncoordinated model loading—forces practitioners to accept either compromised domain capability, catastrophic Out-of-Memory process termination, or debilitating I/O bus thrashing.

ModelVM resolves this dilemma by introducing a clean, pragmatic runtime architecture founded on a central architectural insight: **heterogeneous language models can cooperate reliably under tight memory constraints if cumulative task state is virtualized as a strongly typed, model-independent intermediate representation, and physical model residency is orchestrated via predictive working-set lookahead.** ModelVM realizes this principle through three co-designed subsystems:

1. The **Cognitive State Protocol (CSP)**, which virtualizes semantic state across disparate specialist models through typed, AST-verified 9-tuples ($\mathcal{S}_t$), eliminating conversational bloat and preserving factual constants ($R_{\text{facts}} \geq 0.940$) and mathematical calculations ($R_{\text{calcs}} \geq 0.920$) across extended transition sequences ($N = 12$).
2. The **Predictive Cognitive Working Set** ($W(t, k)$), which adapts Denning’s working-set principles to structured execution graphs, proactively pre-staging upcoming specialist weights into verified memory headroom while shielding resident models via reuse-distance cost accounting, cutting paging stalls by 66.7%.
3. The **Multi-Objective Cognitive Scheduler**, which dispatches execution stages to candidate models via an explainable, 6-term utility scoring function balancing capability fitness, memory footprint, load latency, energy, eviction penalties, and downstream reuse, restricting scheduler deviation from the exact finite-state offline cost-model optimum to $\leq 0.3\%$ [0.1%, 0.7%] (with 0.0% regret on the deterministic cost model) across four divergent workload distributions.

Comprehensive empirical evaluations—spanning an orthogonal 2^3 factorial ablation study and memory-pressure sweeps down to 4.0 GB on a 52.7 GB catalog of ten heterogeneous specialists, alongside real physical silicon execution on an NVIDIA GeForce RTX 5070 Ti—demonstrate that ModelVM enables an 8.0 GB host memory envelope to orchestrate domain specialists with observed zero OOM aborts, a 60.8% reduction in cumulative prompt context,

a **56.2% active inference acceleration** (33.83 s down to 14.83 s via CSP state serialization), and a **77.2% reduction in end-to-end wall-clock latency** (65.05 s down to 14.85 s) compared to uncoordinated execution.

12.1 Future Research Horizons

ModelVM establishes a robust foundation for memory-virtualized cognitive execution, opening several compelling avenues for future systems research:

1. Dynamic and Non-Deterministic Execution Graphs.

While ModelVM currently optimizes structured procedural DAGs produced by task decomposition, autonomous agents increasingly exhibit dynamic control flow, including recursive self-debugging loops and opportunistic web tool retrieval. Extending predictive working-set management to support speculative branch prefetching—where multiple candidate branch specialists are staged probabilistically into partitioned memory headroom—presents an exciting algorithmic frontier.

2. Sub-Tensor and Layer-Granularity Hybrid Paging.

ModelVM operates at the macroscopic granularity of entire model checkpoints, amortizing PCIe transfer latency over complete reasoning stages. In ultra-constrained regimes ($\mathcal{B}_{\text{RAM}} < 3.0 \text{ GB}$) where even a single 7B quantized model cannot fully reside in physical memory, unifying ModelVM’s stage-level cognitive scheduling with fine-grained layer-level streaming (e.g., FlexGen or PowerInfer) could unlock multi-specialist reasoning on micro-controller and mobile edge silicon.

3. Distributed Peer-to-Peer Edge Virtualization.

Rather than restricting execution to an isolated workstation, ModelVM’s decoupled state abstraction naturally extends to heterogeneous edge clusters (e.g., pooling a local laptop, desktop workstation, and mobile NPU over low-latency Thunderbolt or Wi-Fi 7 interconnects). By treating distributed edge devices as remote memory tiers within a unified cognitive address space, ModelVM can orchestrate collaborative multi-agent intelligence across localized environments without relying on centralized commercial cloud infrastructure.

12.2 Artifact Availability and Open Science

To support open science, reproducibility, and practical deployment by the community, the complete ModelVM systems codebase, modular runtime components, benchmark harnesses, pre-registered model manifests, and raw physical hardware telemetry traces are open-sourced under the Apache-2.0 license at <https://github.com/rk-roshan-kr/modelvm>.

References

- [1] Keivan Alwani, Chen Chen, Sreeram Sanyal, and Olivier Delalleau. Llm in a flash: Efficient large language model inference with limited memory. In *arXiv preprint arXiv:2312.11514*, 2023.
- [2] Suparna Bhattacharya, Tarun Kumar, Cong Xu, Satish Kumar Mopur, Jiahao Li, Ashish Mishra, Aalap Tripathy, Annmary Justine Koomthanam, Martin Foltin, and Ian Foster. Position: It is time to virtualize foundation models with a self-evolving operating system layer. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306, 2026.
- [3] Lin Chen, Zihao Ye, Yonghao Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant lora serving. In *arXiv preprint arXiv:2310.18547*, 2023.
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. In *Transactions on Machine Learning Research (TMLR)*, 2023.
- [5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, 23(120):1–39, 2022.
- [7] Yujia Fu, Heming Zhong, Dan Huang, and Yutong Lu. Flare: Fine-grained length-aware routing for resource-efficient heterogeneous llm serving. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 22249–22266, 2026.
- [8] Aashna Garg, Siddharth Singha Roy, Jinu Jang, Federico Brancasi, and Shengyu Fu. Hydra: Hybrid dynamic routing architecture for heterogeneous llm pools. *arXiv preprint arXiv:2605.17106*, 2026.
- [9] Qitian Hu, Jacob Shen, Lianmin Zheng, Ion Stoica, and Joseph E. Gonzalez. Routerbench: A benchmark for multi-llm routing system. In *arXiv preprint arXiv:2403.12031*, 2024.
- [10] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

- [11] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardham, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*, pages 611–626, 2023.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML 2023)*, pages 19274–19286, 2023.
- [14] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36:51991–52008, 2023.
- [15] Kai Mei, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. Aios: Llm agent operating system. In *Proceedings of the Conference on Language Modeling (COLM 2025)*, 2025.
- [16] Isaac Ong, Mohammad Amjad, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [17] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [18] Hetvi Shastri, Pragya Sharma, Walid A. Hanafy, David Irwin, Mani Srivastava, and Prashant Shenoy. Fmplex: Model virtualization for serving extensible foundation models. *arXiv preprint arXiv:2606.09643*, 2026.
- [19] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-lora: Serving thousands of concurrent lora adapters. In *Proceedings of Machine Learning and Systems (MLSys 2024)*, 2024.
- [20] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, and Joseph E. Gonzalez. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning (ICML 2023)*, pages 31094–31116, 2023.
- [21] Yixin Song, Chen Zhao, Junda Yang, Yu Wang, and Minyi Guo. Powerinfer: Fast large language model serving with a commercial gpu. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024)*, pages 824–839, 2024.
- [22] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Chi Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [24] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. In *arXiv preprint arXiv:2406.06282*, 2024.
- [25] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Eleni Christakopoulou, Ion Stoica, and Joseph E. Gonzalez. Sglang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.